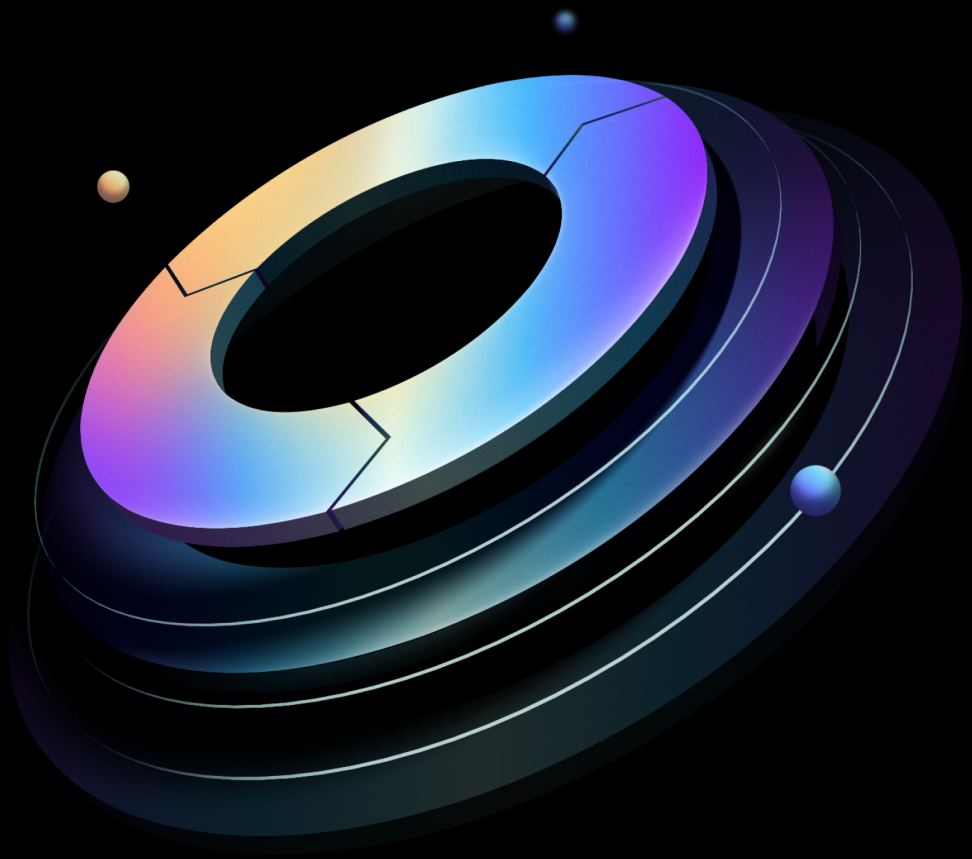
HashiCorp

Vault Administration Guide

Built from commit 3a6a26c83c

HashiCorp | Validated Designs

09 November 2026



HashiCorp

Contents

1 Introduction **8**
1.1 Why use HashiCorp Validated Designs? . . . 8
1.2 Audience . . . 8
1.3 Scope . . . 9
1.4 Prerequisites . . . 9

2 People and process **9**
2.1 Internal developer platform . . . 10
2.1.1 Producers . . . 11
2.1.2 Consumers . . . 11
2.1.3 Platform team . . . 12
2.1.4 Automation/tools team . . . 12
2.1.5 Golden workflows . . . 13
2.1.6 Consumer workflows . . . 13

3 Vault organizational concepts **14**
3.0.1 Configuring Vault using Terraform . . . 16
3.0.2 Set up Vault for consumption for various business units . . . 17

4 Authentication for people **20**
4.1 Configure admin policy . . . 21
4.2 Configure the first auth method . . . 24
4.2.1 OIDC auth method . . . 25
4.2.2 Configuring OIDC auth method . . . 26
4.2.3 LDAP auth method . . . 32
4.2.4 Configuring LDAP auth method . . . 32
4.3 Revoke the root token . . . 36

5 Authentication for applications **37**
5.1 Secure introduction approach . . . 37
5.1.1 Platform integration . . . 37
5.1.2 Trusted orchestrator . . . 39
5.1.3 Vault agent . . . 40

5.2 Configure machine auth method . . . 41
5.2.1 Cloud provider auth method . . . 41
5.2.2 AppRole auth method . . . 45

6 MFA administration 55
6.0.1 Vault MFA capabilities . . . 55
6.0.2 1. Login MFA via OpenID Connect (OIDC) or external identity provider . . . 55
6.0.3 2.Vault Enterprise Step Up MFA . . . 58

7 Static secrets management 60
7.1 Overview . . . 60
7.2 Static secrets management . . . 61
7.2.1 Key/Value (KV) secrets engine . . . 61
7.2.2 Mount and path structure . . . 63
7.2.3 Configure the KV secrets engine . . . 64
7.3 Vault policies . . . 68
7.3.1 Principle of least privilege . . . 69
7.3.2 Role-based policies . . . 69
7.3.3 Policy templates . . . 70
7.3.4 Version control . . . 70
7.3.5 Configure policies for KV Secrets . . . 70

8 Secrets sync management 78
8.1 Overview . . . 79
8.2 Secrets sync components . . . 79
8.3 Secret management challenges . . . 80
8.3.1 People . . . 80
8.3.2 Processes . . . 81
8.4 Example scenario . . . 81
8.5 Design considerations . . . 84
8.5.1 Infrastructure architecture . . . 84
8.5.2 Mount and namespace structure . . . 84
8.5.3 Policy management . . . 85
8.5.4 Lifecycle management . . . 87
8.5.5 Automation . . . 88
8.6 Best practices for sync destinations . . . 89

9 Dynamic secrets management 91
9.1 Dynamic secrets overview . . . 91
9.2 Architecture of dynamic secrets . . . 92
9.2.1 Recommendation on TTL . . . 92
9.2.2 Network considerations . . . 92
9.2.3 Dynamic secrets engine path . . . 95
9.2.4 Database secrets engine . . . 97
9.2.5 Config . . . 97
9.2.6 Database Roles . . . 99
9.2.7 Static Roles . . . 101
9.3 Other database types . . . 103
9.3.1 AWS DynamoDB - AWS Secrets Engine . . . 103
9.3.2 IBM DB2 . . . 103

10 PKI certificate management 103
10.1 Overview . . . 103
10.2 A pattern for automatic certificate renewal . . . 104
10.3 Producers and consumers of certificates . . . 104
10.4 People and process considerations . . . 105
10.4.1 Cross-team collaboration . . . 105
10.4.2 Change management . . . 105
10.4.3 Enablement and documentation . . . 106
10.4.4 Monitoring and alerting . . . 106
10.5 Validated architecture . . . 106
10.5.1 PKI hierarchy . . . 106
10.5.2 Securing CA key material . . . 110
10.5.3 Certificate storage . . . 111
10.5.4 Revocation considerations . . . 111
10.5.5 Unified CRL and OCSP with cross-cluster revocation . . . 112
10.5.6 Automatic CRL rebuilding and rotation . . . 112
10.5.7 Tidying operations . . . 113
10.6 Access controls . . . 114
10.6.1 PKI roles . . . 116
10.6.2 ACL policies and entity metadata . . . 119
10.6.3 Pre-creating entities for machine authentication . . . 121

10.6.4 Enforcing certificate constraints with Sentinel . . . 123
10.7 Vault Agent for VM-based workloads . . . 125
10.7.1 Introduction to Vault Agent . . . 125
10.7.2 PKI consumption workflow . . . 125
10.7.3 Implementation guide . . . 127
10.8 Conclusion . . . 136

11 Key lifecycle management administration 136
11.1 Overview . . . 136
11.1.1 Bring Your Own Key (BYOK) . . . 137
11.2 Key Management Secrets Engine . . . 138
11.2.1 Setup . . . 139
11.2.2 Usage . . . 139
11.3 General Best Practices . . . 141
11.4 KMIP Secrets Engine . . . 142
11.4.1 Supported Profiles . . . 142
11.4.2 Components . . . 142
11.4.3 KMIP Certificate Authority . . . 143
11.4.4 Scopes and Roles . . . 144
11.4.5 KMIP Supported Operations . . . 144
11.4.6 KMIP Debugging & Troubleshoot . . . 146
11.4.7 Implementation of Vault KMIP . . . 146
11.4.8 General Considerations . . . 147

12 Transit and Transform management 147
12.1 Prerequisites . . . 147
12.2 Scaling Vault for Transit and Transform operations . . . 148
12.2.1 Key scaling considerations . . . 148
12.2.2 Scaling strategy . . . 149

13 Monitoring and observability 149
13.1 General recommendations . . . 150
13.2 Telemetry . . . 151
13.2.1 Key metrics . . . 151
13.2.2 Performance monitoring . . . 167

13.3 Audit logs . . . 168
13.3.1 General . . . 169
13.3.2 Usage patterns . . . 169
13.3.3 Privileged endpoints . . . 170
13.3.4 Audit device metrics . . . 171
13.3.5 Comparing sensitive values . . . 172
13.4 API-based monitoring . . . 173
13.4.1 Health API . . . 173
13.4.2 Transactional monitoring . . . 173
13.5 Operational logs . . . 175
13.6 Incident response . . . 175
13.6.1 General steps . . . 175
13.6.2 Restrict access . . . 176

14 Resource quotas 177
14.1 Overview . . . 177
14.2 Prerequisites . . . 177
14.3 Choose the right quota type . . . 178
14.4 Rate limit quotas . . . 179
14.4.1 How rate limit quotas work . . . 179
14.4.2 Scope and precedence . . . 179
14.4.3 Blocking behavior with `block_interval` . . . 180
14.4.4 Identity-based and collective modes (Enterprise) . . . 181
14.5 Lease count quotas . . . 182
14.5.1 How lease count quotas work . . . 182
14.5.2 Protecting against lease explosions . . . 182
14.6 Operational guidance . . . 183
14.6.1 Audit logging for quota rejections . . . 183
14.6.2 Telemetry metrics . . . 184
14.6.3 Response headers . . . 185
14.6.4 Exempt paths . . . 185
14.7 Common scenarios . . . 186
14.7.1 Protect a shared auth mount . . . 186
14.7.2 Protect a noisy namespace or path . . . 186
14.7.3 Guard against lease growth on a dynamic secrets mount . . . 187

14.8 What not to do . . . 187
14.9 Related resources . . . 188

15 Backup and restore operations 189
15.1 Backup standard procedure . . . 189
15.1.1 Prerequisites . . . 189
15.1.2 Automated backup procedure for Vault with disaster recovery replication enabled . . . 189
15.2 Restoring a Vault Enterprise cluster . . . 190
15.2.1 Restore procedure for Vault with disaster recovery replication enabled . . . 190

16 Disaster recovery operations 191
16.1 DR cluster promotion . . . 191
16.1.1 When to initiate a DR process . . . 191
16.1.2 How to initiate a DR process . . . 195
16.1.3 How to failback . . . 197

17 Performance replication operations 199
17.1 Monitoring performance replication . . . 199

18 Upgrade procedures 200
18.1 Vault Enterprise upgrade . . . 200
18.1.1 Autopilot . . . 200
18.1.2 How to upgrade Vault in an AWS auto scaling group with automated upgrades 201

19 Upgrade procedure examples 204
19.0.1 HA cluster in place upgrade . . . 204
19.0.2 DR upgrade . . . 205

20 Support readiness 205

21 Changelog 205
21.1 2026-08 . . . 206
21.2 2026-06 . . . 206
21.3 2026-01 . . . 206
21.4 2025-12 . . . 206
21.5 2025-07 . . . 206

21.6 2025-06 206
21.7 2025-05 206
21.8 2025-04 207
21.9 2024-02 207

22 Credits **207**

1 Introduction

Note

These guides were reorganized in August 2026. The previous Solution Design Guide and Operating Guides (Adoption, Standardization, Scaling) have been replaced with three role-based guide types: Installation, Administration, and User. Read the HVD 2.0 release notes for full details on what changed and where to find the content you need.

1.1 Why use HashiCorp Validated Designs?

HashiCorp Validated Designs (HVD) provide practitioners with opinionated guidance for achieving production-grade deployments of HashiCorp products. These designs are purpose-built for delivering foundational use cases, with a baseline level of architectural and operational maturity. They draw on the field experiences of Solutions Engineers and Solutions Architects working with customers across a wide range of environments and organizational requirements.

Each guide provides access to an opinionated reference architecture, including key design decisions and the rationale behind them. Where applicable, guides identify modular design components that you can adjust to align with organizational or regulatory requirements without compromising the overall integrity of the implementation. For many deployments we include Terraform modules to automate large portions of infrastructure provisioning and software installation.

1.2 Audience

This Administration Guide is for platform, security, identity, and operations teams that run Vault Enterprise as a shared service after installation is complete. It assumes the Vault cluster is deployed, initialized, unsealed, and reachable by operators and automation.

Use this guide for recurring day-2 administration: operating model, namespaces, authentication, policies, secrets-engine administration, monitoring, backup and restore, disaster recovery operations, upgrades, and support readiness. For deployment architecture and first-run setup, use the Vault Installation Guide for virtual machines or the Vault Installation Guide for OpenShift. For application and service-team consumption workflows, use the Vault User Guide.

1.3 Scope

This guide covers day-2 platform operation of Vault Enterprise:

Area	Summary
People and process	Team responsibilities, producer and consumer workflows, and operating-model guidance.
Organizational concepts	Namespaces, secrets engines, policies, and Terraform-based platform administration patterns.
Identity and access	Ongoing authentication method, MFA, policy, and application-auth administration.
Secrets-engine administration	Operator setup and governance for static secrets, Secrets Sync, dynamic secrets, PKI, key management, Transit, and Transform.
Observability and response	Audit logs, operational logs, monitoring, incident response, and replication monitoring.
Operations	Backup, restore, disaster recovery, upgrades, and support readiness.

1.4 Prerequisites

Before using this guide, complete the relevant installation track and confirm that Vault is running in the target environment. Operators should also have administrative access to Vault, access to the organization's identity provider, and access to the infrastructure and monitoring systems used by the deployment.

2 People and process

Vault requires thoughtful considerations around people and processes in order to maximize its value. It is also necessary to rethink how an important platform technology such as Vault is positioned

and offered. Changing culture and processes can be difficult, as can organizational and political challenges. Our observations across the industry give us a glimpse into what good looks like, which we share below. Please keep in mind that no two organizations are the same, and your specific circumstances may require you to choose how you organize your teams.

2.1 Internal developer platform

The Internal developer platform (IDP) has emerged to address the challenge of reducing development cycles and organizational complexity, especially as cloud computing and IaC have drastically cut down infrastructure provisioning times.

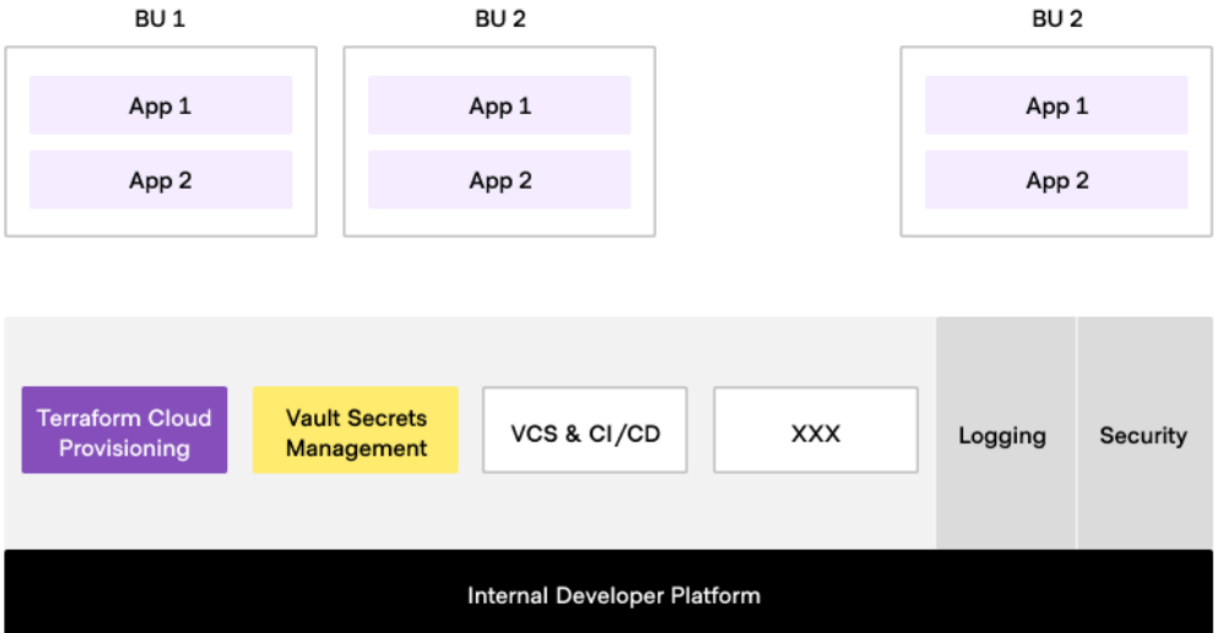


Figure 1: Internal developer platform

The IDP builds on the time-tested shared service model; improved by adding a product management approach. In this model, a platform team works with developers to build golden paths that development teams can consume in a self-service mode.

Because applications need secrets for operation, a secret management service is an integral component of any IDP. We recommend that the team with ownership of the IDP (aka The Platform Team) also take ownership of operating Vault, with the Security Team providing governance.

With this approach, two key roles are typically identified: producers and consumers. The terms *producer* and *consumer* are used to define the roles and responsibilities within a shared service or IDP provided by a platform team.

Tip

While we talk of the platform team as a singular team, it is possible that the platform team is composed of multiple sub-teams with separate areas of responsibility. For example, secret management components of the IDP might be owned by a separate team than infrastructure provisioning components. These sub teams will however collaborate closely to ensure that developers in the various business unit app teams can easily access these shared services.

2.1.1 Producers

In a shared service model, the producers have a role in configuring the system to meet the needs of the different consumers. Their responsibilities include:

- Operating Vault and ensuring it is available as a tier-0, mission-critical service.
- Ensuring workflows are clearly defined for application teams.
- Offering necessary enablement to the consumers.

The producers ensure that the shared services are readily accessible and efficiently utilized by the consumers, empowering them to leverage the benefits of the Vault platform effectively.

2.1.2 Consumers

In a shared service model, consumers refer to any team within the organization that utilizes the shared services provided by the Platform Team. These teams have responsibilities that include:

- Initiating requests for access to store secrets in Vault.
- Configuring applications to integrate with Vault.

Consumers play a crucial role in leveraging the shared services to meet their specific needs and requirements while adhering to the guidelines and standards set by the Platform Team.

2.1.3 Platform team

The platform team is responsible for overseeing the overall implementation and management of the shared tools and services to enable golden workflows, and driving its adoption within the organization. When a platform team matures, they will tend toward providing their consumers an integrated user experience (that implements golden workflows) using an internal developer platform (IDP).

In the context of Vault adoption, the platform team ensures efficiency and standardization by establishing and enforcing standards and best practices, promoting consistency across project teams and minimizing the risk of errors and inconsistencies that can disrupt operations. They enable scaling and reusability by enabling automated golden workflows for common user workflows that streamline operations and reduce redundant efforts.

By providing valuable knowledge and expertise in Vault to the consumer community, they serve as a centralized resource for expert guidance. This reduces the learning curve for project teams and ensures consistent application of knowledge throughout the organization.

Automation and integration are facilitated by the platform team, automating common tasks using IaC and integrating it with other DevOps tools. This streamlines processes, increases speed, and reduces manual errors. They also address the security aspect by ensuring adherence to security best practices in infrastructure setup and maintenance, reducing vulnerabilities to cyber threats.

Moreover, the platform team significantly reduces the time to market for new features and applications. With a mature IaC/Terraform service, they expedite the deployment of new infrastructure, enabling faster time to market.

2.1.4 Automation/tools team

The automation/tools team is a tactical component of the platform team. They manage and maintain the automation and tools required for systems and services to operate effectively. They enable/implement golden workflows for consumers to leverage the shared services and tools that are part of the platform. As this team matures, they become the product owners and implementers of the IDP.

There could be multiple teams within the automation/tools team, each responsible for a different tool or group of tools. The CCoE provides overall strategic guidance for this team, and they collaborate with the consumer community to ensure that the services or workflows they enable meet their needs.

In simpler terms, the automation/tools team is responsible for making sure that the tools and automation that are used by the Platform Team are working properly. They also work with the consumer community to make sure that the services and workflows that are enabled by these tools meet their needs.

2.1.5 Golden workflows

A golden workflow is a standardized, repeatable process for completing a specific task or achieving a specific outcome. It is typically well-documented and tested, and it is designed to be efficient and effective. Golden workflows are often used in software development, IT operations, and other business processes.

There are many benefits to using golden workflows. They can help to:

- Improve efficiency and productivity
- Reduce errors and mistakes
- Ensure consistency and quality
- Improve communication and collaboration
- Accelerate onboarding and training
- Facilitate automation

With Vault, the platform team enables golden workflows for federating identity and managing secrets. The platform team empowers appdev and other teams to independently store and modify secrets while maintaining centralized management and control. The platform team oversees and governs these workflows, ensuring compliance with policies, security requirements, and best practices.

2.1.6 Consumer workflows

These workflows are enabled by the platform team but used by various consumers including but not limited to various application/development teams.

1. App onboarding Workflows: This foundational workflow involves onboarding an application to Vault. This involves determining the location within the vault where secrets for a particular application will be stored and ensuring the application can consume those secrets.
2. Developer Workflow: In this workflow the developer is enabled to make modifications/updates to the secrets stored in Vault. This also includes adding new types of secret engines.

3 Vault organizational concepts

At the highest level, we have a Vault cluster. There are two deployment options: self-managed, where you install and run Vault Enterprise manually, and HCP Vault Dedicated, where we handle the deployment and management of Vault on your behalf, including seamless upgrades.

In Vault, a cluster begins with a root namespace. For HCP Vault Dedicated, this namespace is reserved for HCP control plane management and is not accessible to end users. In self-managed Vault Enterprise clusters, the root namespace should be reserved for Vault operators only.

Under this root namespace, in HCP Vault Dedicated, a namespace is automatically created and called *admin*. We recommend that operators of self-managed Vault clusters create the same namespace and chroot this in line with the Vault Namespace and Mount Structuring Guide which we recommend considering as part of your planning. Reserve the root namespace for operators with root-level access. This is to secure cluster-level configurations, as some configurations can only be made within the root namespace.



Figure 2: Vault Enterprise namespace and mounting recommendations

A Vault operator can create subordinate namespace structures just like a folder structure, i.e., namespaces can be nested (as shown in the diagram above). Each namespace can store:

- Secret engines: Provide an interface for storing, retrieving, and controlling access to sensitive data, with each type of secret engine handling specific kinds of secrets or workflows, such as key/value storage, database credential issuance, and more.
- Auth methods: The method that users, machines, or applications can use to authenticate to a Vault cluster.

- Policies: The permissions assigned to authenticated users, machines, or applications specifying what secrets they can access.

While it is important to understand that the eventual namespace structure can depend on the circumstances, we recommend deploying tenant namespaces under the admin namespace. This is where the *tenant administrators* define the policies necessary for granting access to the different sub-namespaces which could map applications or environments in the tenant. The rationale behind this recommendation is to consolidate all human access points into a single, centralized location within the cluster. Typically, organizations utilize an identity management platform like Active Directory or Okta for managing human user access. Should use cases for mapping auth methods for human users to tenant namespaces subsequently arise, the Vault operator can develop the solution in a controlled manner accordingly.

3.0.1 Configuring Vault using Terraform

We recommend using Terraform to automate the configuration of Vault, for example, setting up namespaces, secret mounts, policies, etc. On a small scale, manually configuring Vault may be manageable. However, as the scale and complexity of your use case increases, you will need some form of automation to help keep up with the increasing management overhead.

For example, you will most likely need to map many OIDC provider groups to Vault external groups across multiple environments (development, test, staging, production, etc.). This can quickly become a significant operational overhead without an automated process.

The Vault Terraform provider allows Terraform to read from, write to, and configure HashiCorp Vault. See here for more details on the Vault provider. HashiCorp provides a validated pattern for using Terraform to manage Vault policies.

The Vault Terraform provider enables you to take an infrastructure as code (IaC) approach to your Vault configurations. This provides many benefits including increased productivity, promoting repeatable processes, and reducing human errors. While other infrastructure automation tools such as Ansible can be used to automate Vault configuration, we recommend the Terraform provider because it features testing and support from HashiCorp as well as regular updates to support new features in Vault. Terraform also has a robust state management system which tracks the current configuration on your Vault clusters. For more information on adopting Terraform, please refer to the HashiCorp Validated Design for Terraform.

We recommend that the Vault admin team set up a `vault-admin-config` repository in a version control system (VCS) such as Git, where Vault configuration is stored as Terraform files. Any change to the configuration is applied to the repo and the changes are then made to Vault.

This allows configuration changes to be migrated between the various application lifecycle environments.

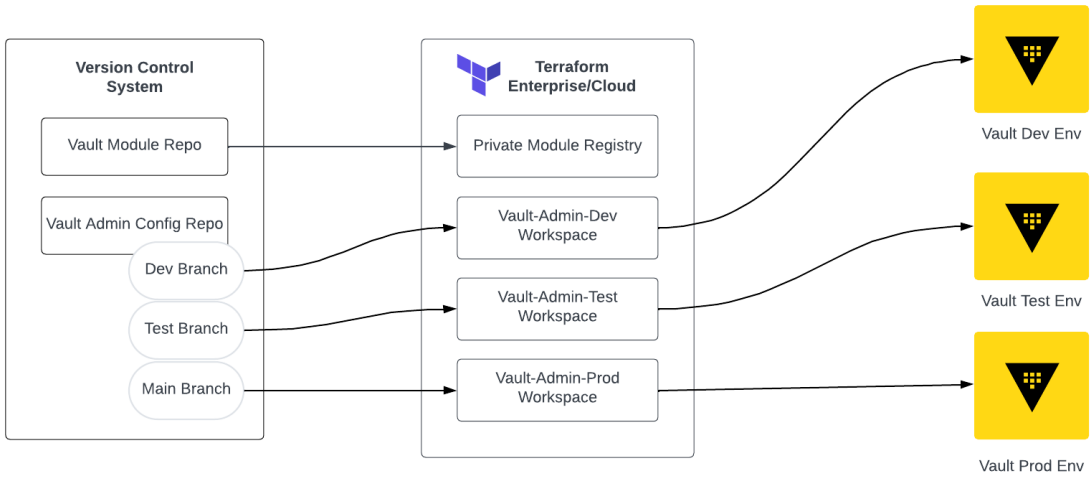


Figure 3: Vault Terraform configuration diagram

3.0.2 Set up Vault for consumption for various business units

When Vault is configured as recommended in the Vault organizational concepts section, all users of various business units will authenticate via a single top level auth method. Their membership to groups will dictate which namespace and which secret engine paths within the namespace(s) they will have access to.

Environments

Separate from your production Vault environment, we recommend deploying an additional cluster as a testing instance. A test Vault instance should be able to withstand longer periods of downtime without business impacts, allowing for the safe examination and troubleshooting of new configurations, policies, and updates without putting your business at risk. The configuration of the testing

environment should mirror the configuration of the production environment as closely as possible in order to ensure that any changes being tested will produce the same behavior in both environments.

You may also wish to provision additional Vault clusters for additional environments separate from production. For example, you may wish to have a separate Vault cluster for a development or staging environment. This decision should be based on your organization's own availability requirements and security policies.

For example, one organization may have CI/CD pipelines for a development environment which are still critical to producing and deploying new software, and therefore may need to be associated with the production Vault cluster to guarantee high-availability and security. Another organization may have extremely strict security requirements for secrets associated with their production environment, and therefore may decide to run a separate Vault cluster for their development environment to prevent any co-mingling of production and development secrets. These considerations can help you decide how many distinct Vault environments are right for your organization.

Important workflows

1. Application team onboards a new app to Vault.
2. Application team adds a new secret.
3. Application team edits/rotates a secret.
4. Application consumes a secret.

Onboarding workflow

Aim to fully shift your onboarding process left, so that the application teams adopting the platform have an end-to-end, automated workflow that is integrated into the organisation's identity and access management (IAM) system. This will ensure that the application teams can onboard themselves to the platform without needing to involve the platform team. To not do this risks the situation where manual tasks to onboard internal teams pile up in the platform team's backlog, this team then becoming a bottleneck to the projects success. We recommend ensuring your project manager has scoped time in the project plan to ensure platform team engineers can complete the necessary engineering so that it can be tested meaningfully in UAT by your early-adopters.

Adopt a namespace structure in line with the Vault Namespace and Mount Structuring Guide so that appropriate policies can be used to scope specific access while allowing correct sharing of

secrets engines and auth methods. This will allow the organisation to avoid limits in the system which may adversely impact the cluster in the future.

We recommend that you codify Vault configurations using an IaC tool such as Terraform. Below is an example of an onboarding workflow built around Terraform Enterprise or HCP Terraform.

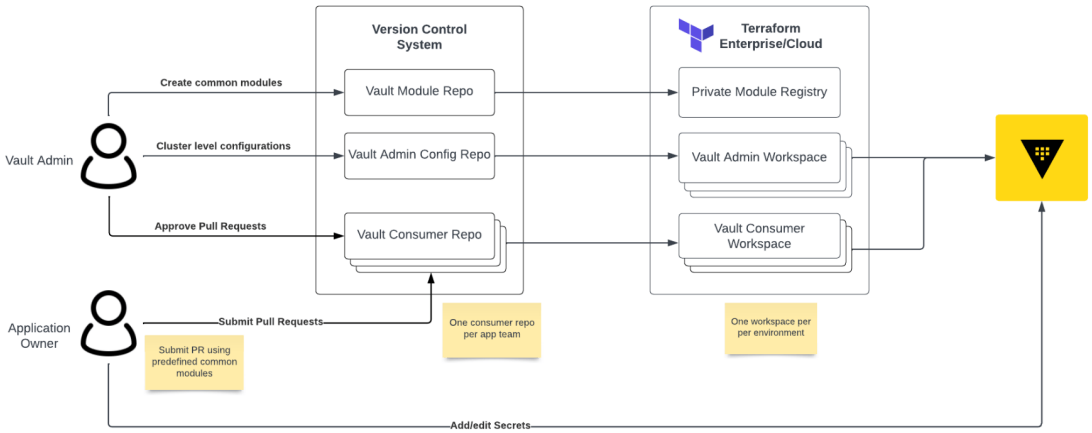


Figure 4: Vault IaC Workflow

There are two personas in this workflow: Vault administrators and application owners. Vault administrators are part of the platform team, they are responsible for:

1. Vault cluster level configuration such as setting up audit logs, mounting auth methods and secret engines etc.
2. Creating common Terraform modules for application owners to consume.
3. Approving pull requests in the Vault consumer repositories.

Vault administrators manage cluster level configurations using Terraform through a Vault admin configuration Git repository. The Vault admin configuration repository maps to multiple Terraform Enterprise or HCP Terraform workspaces using the VCS workflow - one workspace per environment (i.e. development, staging, production). When a pull request is merged into the repository, Terraform will kick off a workflow to apply the configuration changes. For more information on this workflow, please refer to the Terraform validated design.

In addition to cluster level configuration, the Vault administrators are also responsible for creating common Terraform modules to be consumed by application owners. Each module should align to

a pre-approved Vault use case and provisions the resources necessary for consumers to access and work with Vault. For example, a module for consuming Vault static secrets would provision the appropriate Vault policies, identity groups, auth method roles, and the required paths within the KV secret engine. These modules serve as an interface for Vault, abstracting its configuration details from application owners. They should offer sensible defaults and expose only the necessary input parameters.

Application owners request access to Vault by using common Terraform modules and making a pull request to a Vault consumer repository. Each application team requesting access to Vault should have its own consumer repository. The creation of the consumer repository should be initiated by an onboarding request form through ServiceNow, Jira, or equivalent. The consumer repository is where Vault configuration is managed long-term for the application. Similar to the Vault admin configuration repository, each consumer repository maps to multiple environment specific workspaces in Terraform Enterprise or HCP Terraform. Pull requests submitted to the consumer repository are reviewed and approved by the Vault administration team. Once the pull request is approved and merged, Terraform Enterprise or HCP Terraform will kick off a workflow to apply the changes to Vault. Application owners should then use the Vault UI, CLI, or API to manage secrets to ensure that secrets are not committed into the repository.

4 Authentication for people

Authentication in Vault is the process by which user- or machine-supplied information is verified against an internal or external system. Vault supports multiple auth methods including LDAP, OIDC, and more. Each auth method has a specific use case. In this section, you will configure your first auth method to allow human users to login to Vault.

Clients must authenticate against an auth method to interact with Vault. After successful authentication, a token is returned to the client. The token is associated with policies which control the resources and operations that the client can access.

The mapping between the token and policy is configured in the auth method. This process is described in detail in the policies concepts documentation.

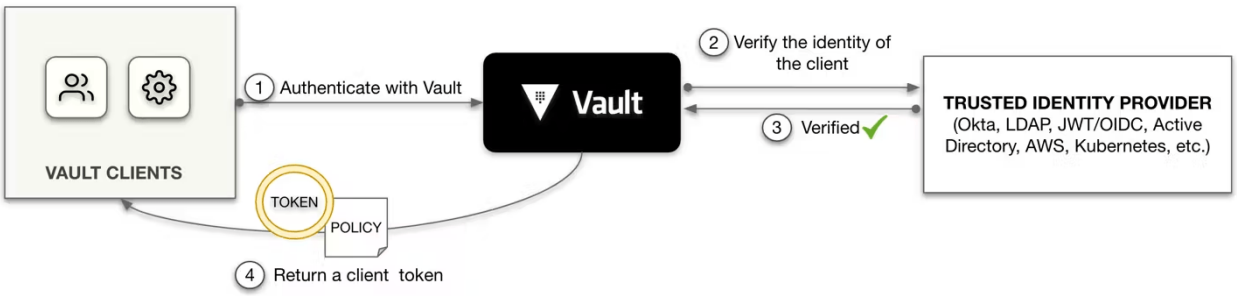


Figure 5: Vault authentication workflow

Figure 1: Vault's authentication workflow

The token auth method is built-in and is at the core of client authentication. A client can authenticate with Vault through the token auth method. For example, a Vault admin logs in with Vault via token auth method using the initial root token (or admin token if you are running HCP Vault) so that the admin can configure other auth methods.

> **Note**
>
> It is considered a best practice not to persist root tokens. Use the root token only for just enough initial setup. Once you enable an auth method with appropriate policies allowing Vault admins to log in and perform operational tasks, the admins should use the auth method to authenticate instead of using the root token (or admin token for HCP Vault).

4.1 Configure admin policy

Policies form the basis of the role-based access control (RBAC) system for Vault. Logical operations in Vault are path-based, and policies provide a declarative way to grant or forbid access to certain paths and operations in Vault. If you are not familiar with the concept of Vault policies, we strongly encourage you to review the policies concept documentation and the getting started guide to policy writing.

The first group of users to grant access to Vault are your operators who have full access to Vault. They are responsible for cluster level configurations and any requests to the restricted API that requires the root namespace. By default Vault does not include any administrative policies. In a new Vault installation, there is only a *default* policy that does not grant access to any functional

resources in Vault. You will need to create an appropriate operator policy for this subset of users. Below is an example.

```
# * (glob) character matches any prefixes in the path and can only be used at the end
path "*" {
  capabilities = ["create", "read", "update", "delete", "list", "sudo"]
}
```

> Note
>
> You do not need to create an operator policy for HCP Vault since the root namespace is not accessible.

Login to Vault using your root token and run the command below to create an operator policy under the root namespace.

```
$ export VAULT_TOKEN=<your root token>
$ vault policy write operator operator-policy.hcl

Success! Uploaded policy: operator
```

The second group of users to grant access to Vault are your tenant administrators. Tenant administrators are responsible for day to day Vault operations for their Vault tenant (a parent namespace and a number of subordinate application/environment namespaces). They only have access to their tenant namespace and below and do not have access to the root namespace. The paths that the tenant administrators can access are governed by the tenant admin policy. Below is an example of such a policy.

```
# List existing policies
path "sys/policies/acl"
{
  capabilities = ["list"]
}

# Create and manage ACL policies
path "sys/policies/acl/*"
{
  capabilities = ["create", "read", "update", "delete", "list"]
}

# Manage auth methods broadly across Vault
path "auth/*"
{
  capabilities = ["create", "read", "update", "delete", "list", "sudo"]
}

# Create, update, and delete auth methods
path "sys/auth/*"
{
  capabilities = ["create", "update", "delete", "sudo"]
}

# List auth methods
path "sys/auth"
{
  capabilities = ["read"]
}

# Managing identity
path "identity/*"
{
  capabilities = ["create", "read", "update", "delete", "list"]
}

# Enable and manage the key/value secrets engine at `secret/` path
path "secret/*"
{
  capabilities = ["create", "read", "update", "delete", "list"]
}

# Allow managing leases
path "sys/leases/*"
{
  capabilities = ["read", "update", "list"]
}

# Manage namespaces
path "sys/namespaces/*" {
  capabilities = ["create", "read", "update", "delete", "list"]
}

# Manage secrets engines
path "sys/mounts/*"
{
  capabilities = ["create", "read", "update", "delete", "list"]
```

This example policy is only a subset of Vault's API. If additional administrative capabilities are necessary, edit the admin policy when the need arises.

Login to Vault using your root token and run the commands below to create an admin policy under the org-level namespace.

```
$ export VAULT_NAMESPACE=<org level namespace>
$ vault policy write admin admin-policy.hcl

Success! Uploaded policy: admin
```

> Tip
>
> Setting the environment variable `VAULT_NAMESPACE` tells the CLI to execute all subsequent commands under the specified namespace.

4.2 Configure the first auth method

With the admin policies created, you are ready to configure your first auth method to allow your Vault super administrators to login.

There are many auth methods available in Vault, some auth methods are targeted towards human users while others are targeted towards machines. Which auth methods you enable depends on the type of clients (human or machine), and the existing processes and systems in place for identity management. For example, if an organization uses Microsoft Active Directory for user identity management, then it makes sense to enable the LDAP auth method for user authentication.

Depending on the use case, multiple auth methods can be enabled in Vault to provide access to the same data. For the initial configuration, you will enable one auth method for the operators in the root namespace and one auth method for all other users in the tenant admin namespaces.

For human user access, if your organization already has a user authentication system tied into the user lifecycle and Vault supports it (e.g. OIDC, LDAP), you should consider using this as an auth method. In this case, you only have one place to manage your users, and users are less likely to get stranded in Vault either in a group they have moved from or as a legacy user that no longer works in the organization.

Certain auth methods are more popular due to the widespread use of their associated technologies in modern organizations. This section focuses on the following common auth methods for human

user access and how to use them.

- OIDC
- LDAP

4.2.1 OIDC auth method

The OIDC auth method allows authentication via a configured identity provider such as Okta or Azure AD, providing a single sign-on experience to Vault services. Authentication requests can be initiated using the Vault UI, CLI, or the API. The OIDC auth method allows a user's browser to be redirected to a configured identity provider, complete login, and then be routed back to Vault's UI with a newly-created Vault token. This workflow is depicted in Figure 2.

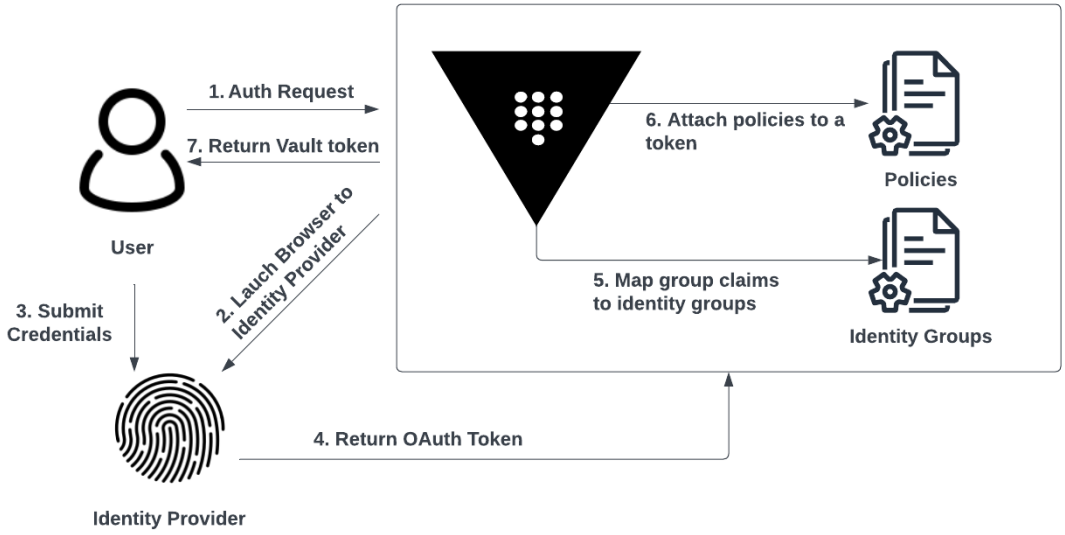


Figure 6: OIDC Auth Workflow

Figure 2: OIDC Auth Workflow

1. User requests authentication using the OIDC auth method via Vault's UI or CLI.
2. Vault launches a browser to the configured discovery URL for the OIDC provider.
3. User submits credentials to the OIDC provider.

4. OIDC provider validates credentials and returns an OAuth token to Vault. The OAuth token contains information about the user such as security groups that the user is a member of, which could map to an identity group within Vault.
5. Vault maps the security group claims in the OAuth token to pre-configured identity groups within Vault, which is attached to Vault policies.
6. Vault attaches policies associated with the identity groups to a Vault token.
7. A Vault token is returned.

4.2.2 Configuring OIDC auth method

The steps outlined below are the general workflow for configuring an OIDC auth method. You should perform these steps first in the root namespace for your Vault operators, then configure another OIDC auth method or a different human auth method in the org level namespace for tenant administrators and all other users. The code snippets in these steps will highlight distinctions between the configuration in the root namespace versus the org level namespace.

Step 1: Enable the auth method

Before you can use an auth method in Vault, you must enable it. You can enable and configure the auth method using the UI, CLI, or the API. We will focus on CLI commands in this document.

Configuration for root namespace

Using the Vault CLI, log in to Vault with your root token and enable the OIDC auth method under the root namespace using the code snippet below.

```
$ vault auth enable oidc
```

Configuration for tenant namespaces

Using the Vault CLI, log in to Vault with your root token and enable the OIDC auth method under the tenant level namespace using the code snippet below.

```
$ export VAULT_NAMESPACE=<org level namespace>
$ vault auth enable oidc
```

> **Tip**
>
> This could be reconfigured for different use cases. For example, OIDC could be enabled multiple times on different paths, for different OIDC providers or different environments. Enabling this auth method at a different path can be achieved using the `-path` flag.

Step 2: Configure the auth method

Once the OIDC auth method has been enabled, you are now ready to configure the auth method itself. Vault needs to know how to connect to your identity provider and what credentials it should use. These settings include the `oidc_discovery_url`, `oidc_client_id`, and the `oidc_client_secret`. You can find more details about these settings in the JWT/OIDC auth method API. Your Vault nodes will need the ability to connect to the specified `oidc_discovery_url` typically over TCP port 443 (HTTPS) to retrieve OIDC metadata. You must also provide a default Vault role name as part of the configuration. The role does not have to exist at this stage as it will be created in the next step.

Configure the OIDC auth method using the CLI command below. Set the `oidc_discovery_ca_pem` parameter to point to the CA certificate used to validate the connections to the OIDC discovery URL. If not set, the system certificates will be used.

```
$ vault write auth/oidc/config \
    oidc_discovery_url="https://sso.example.com" \
    oidc_client_id="vault" \
    oidc_client_secret="passw0rd" \
    oidc_discovery_ca_pem=@ca.pem \
    default_role="default"
```

> **Tip**
>
> The `@` prefix can be used to read data from a file on disk. See the CLI documentation for more information.

Step 3: Create the default role

With the OIDC auth method configured, you can now create the default role named in the previous step.

Multiple roles can be created under a single auth method. Each role can be associated with different Vault policies, granting multiple levels of access. The default role created in this step is only used when the user does not specify a role during authentication.

The role contains information that Vault uses to uniquely identify users and the groups to which they are a member of, using claims within the OAuth token sent by the identity provider. The role also specifies which Vault policies should be attached to the tokens issued by this role when a user's OAuth token matches the claims defined within the role. The default role is commonly associated with Vault's default policy, which is a built-in policy that cannot be removed and provides a set of limited capabilities. Below is an example CLI command to create the default role.

```
$ vault write auth/oidc/role/default -<<EOF
{
 "user_claim": "email",
 "groups_claim": "groups",
 "allowed_redirect_uris": [ "https://vault.example.com:8200/ui/vault/auth/oidc/oidc/
    callback", "https://vault.example.com:8250/oidc/callback" ],
 "token_policies": "default",
 "token_ttl": "1h",
 "token_max_ttl": "1h"
}
EOF
```

The `user_claim` parameter is the claim in the OAuth token used to uniquely identify the user. This will be used as the name for the identity entity alias created due to a successful login. The identity entity alias is a representation of a client for a specific auth method within Vault's identity system. Please refer to the Identity Secrets Engine documentation for more information.

The `group_claim` parameter is an optional field containing the claim used to uniquely identify the set of groups to which the user belongs. Vault will attempt to match the value from this claim to an external group created within Vault. If a match is found, Vault will add the entity to the external group as a member.

The `allowed_redirect_uris` is a list of allowed values where users are redirected after authenticating with the OIDC provider. The URI with port 8200 is for login via the UI whereas the URI with port 8250 is for login via the CLI. These URIs must match the configuration on the OIDC provider configuration.

The `token_policies` is the list of Vault policies that would be attached to the token upon a successful login.

The `token_ttl` is the duration of time before the generated token expires if not renewed. The format of this parameter follows the duration string format. We recommend that you always provide a token TTL value to override the default token TTL of 32 days (768 hours). See our Vault Tokens documentation for more information.

The `token_max_ttl` is the maximum lifetime for the generated token. The token cannot be renewed past the allowed max TTL.

Step 4: Create an external group and policy mapping

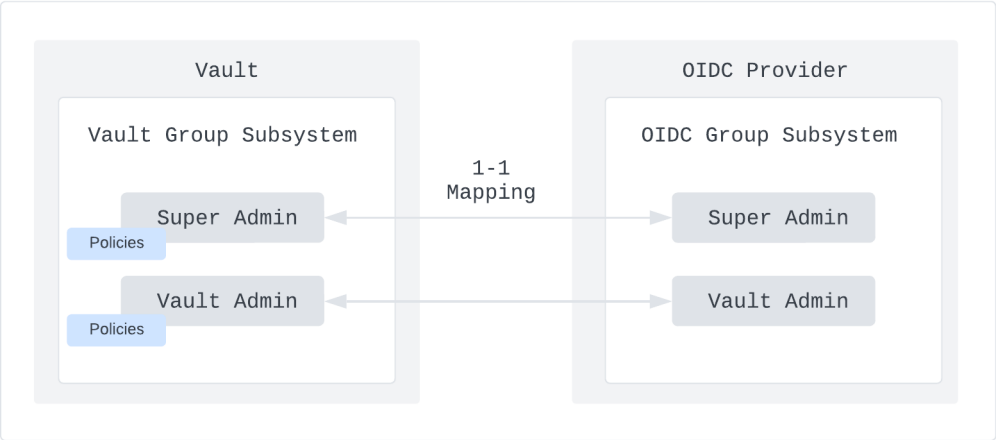


Figure 7: External group mapping

Figure 3: Mapping of OIDC provider group to Vault identity group

The last configuration for OIDC auth is to map groups in your OIDC provider to Vault identity groups. This allows you to centrally manage group membership within your OIDC provider. Vault uses this information to assign the correct policies for each group within your OIDC provider.

The first step in implementing this mapping is to create a Vault external group, which is a Vault representation of a group outside of its identity store. The external group is associated with a set of Vault policies that define the access allowed. The next step is to create a Vault group alias, which ties the Vault external group to a group in your OIDC provider. During authentication, Vault will attempt to match an OIDC provider group to an external group using the group claim specified in step 3. If a match is found, an entity is added as a member of the external group and a token is returned with the associated policy. If a user is removed from the group in your OIDC provider, that change gets reflected in Vault only upon a subsequent login or token renewal operation.

Configuration for root namespace

The commands below create an external group called vault-operators for the Vault operators team. This external group is tied to an OIDC group called vault-operators by the group alias created in the last command. You can find more details on each parameter in our identity secrets engine API.

```
$ GROUP_ID=$(vault write -format=json identity/group \
  name="vault-operators" \
  type="external" \
  policies="operator" | jq -r ".data.id")

$ MOUNT_ACCESSOR=$(vault read -field=accessor sys/mounts/auth/oidc)

$ vault write identity/group-alias \
  name="vault-operators" \
  mount_accessor=$MOUNT_ACCESSOR \
  canonical_id=$GROUP_ID
```

Configuration for org namespace

Before enabling the below configuration, ensure that your Vault CLI is configured to point to the org namespace. Create an external group called vault-admins for your regular Vault administrators.

```
$ GROUP_ID=$(vault write -format=json identity/group \
  name="vault-admins" \
  type="external" \
  policies="admin" | jq -r ".data.id")

$ MOUNT_ACCESSOR=$(vault read -field=accessor sys/mounts/auth/oidc)

$ vault write identity/group-alias \
  name="vault-admins" \
  mount_accessor=$MOUNT_ACCESSOR \
  canonical_id=$GROUP_ID
```

Step 5: Validate OIDC auth configuration

You can validate the OIDC auth configuration with either the Vault UI or CLI. Login using the CLI with the following command in the next sections.

Configuration for root namespace

Login to the root namespace as a super administrator.

```
$ vault login -method=oidc
```

Configuration for tenant namespace

Login to the tenant namespace as a tenant administrator.

> Note
>
> In order to access the org namespace, super administrators must first login via the root namespace. Once authenticated, the token granted can be used to operate on the org namespace.

```
$ export VAULT_NAMESPACE=<tenant level namespace>
$ vault login -method=oidc
```

This command launches a browser window to your OIDC provider login page. The default role is used since no role is specified. Enter your credentials and you should be redirected back to Vault indicating that you have successfully signed in via OIDC. You can validate the policies attached to the returned token with the token information displayed on your CLI.

```
Waiting for OIDC authentication to complete...
Success! You are now authenticated. The token information displayed below
is already stored in the token helper. You do NOT need to run "vault login"
again. Future Vault requests will automatically use this token.

Key                    Value
---                    -----
token                  hvs.root.token
token_accessor         Igcq6WRvsPk8ViO9rUpEGlrk
token_duration         1h
token_renewable        true
token_policies         ["default"]
identity_policies      ["admin"]
policies               ["default" "admin"]
token_meta_email       nick@domain.com
token_meta_role        default
token_meta_username    nick
```

Note

Users who are mapped in the tenant namespace can still login to the root namespace. However, their access is restricted by the default policy since there is no external group and policy mapping for these users in the root namespace.

Useful resources

- OIDC configuration troubleshooting
- High-level configuration steps for various OIDC providers
- Azure AD with OIDC auth method tutorial
- OIDC authentication with Okta tutorial

4.2.3 LDAP auth method

The LDAP auth method allows authentication using an existing LDAP server and user/password credentials. This enables users to login to Vault using their existing LDAP credentials without having to create new user accounts in Vault. LDAP groups can be directly mapped to Vault policies, which control the paths in Vault that members of the LDAP group can access.

The LDAP auth method connects directly to your organizations' LDAP servers. Vault must be able to communicate with the LDAP server over TCP and UDP port 389, or on port 636 for LDAPS. When an authentication request is made through the auth method, Vault verifies the provided credentials with the backend LDAP server through a service account. After successful validation of the credentials, Vault issues a token, associates it with the appropriate Vault policy, and returns the token to the user.

4.2.4 Configuring LDAP auth method

The steps outlined below are the general workflow for configuring an LDAP auth method. You should perform these steps first in the root namespace for your super administrators, then configure another LDAP auth method or a different human auth method in the tenant namespace for regular administrators and all other users. The code snippets in these steps will highlight distinctions between the configuration in the root namespace versus the tenant namespace.

Step 1: Enable the auth method

First step is to enable the auth method. You can enable and configure the auth method using the UI, CLI, or the API (including via Terraform). We will focus on CLI commands in this document.

Configuration for root namespace

Using the Vault CLI, login to Vault with your root token and enable the LDAP auth method under the root namespace using the code snippet below.

```
$ vault auth enable ldap
```

Configuration for tenant namespace

Using the Vault CLI, login to Vault with your root token and enable the LDAP auth method under the tenant namespace using the code snippet below.

> Tip
>
> Setting the environment variable `VAULT_NAMESPACE` tells the CLI to execute all subsequent commands under the specified namespace.

```
$ export VAULT_NAMESPACE=<tenant namespace>
$ vault auth enable ldap
```

Step 2: Configure the auth method

Next step is to configure the auth method to allow Vault to communicate with the backend LDAP server(s).

The command below shows an example of a basic LDAP configuration.

- LDAP server running on ldap.example.com, port 389.
- Server supports `STARTTLS` command to initiate encryption on the standard port.
- CA Certificate stored in file named `ldap_ca_cert.pem`
- Server does not allow anonymous binds for performing user search (`discoverdn=false`).
- Bind account used for searching is `cn=vault,ou=users,dc=example,dc=com` with password `My$ecrt3tP4ss`.
- User objects are under the `ou=Users,dc=example,dc=com` organizational unit.

- Username passed to Vault when authenticating maps to the `sAMAccountName` attribute.
- Group membership will be resolved via the `cn` attribute of group objects. That search will begin under `ou=Groups,dc=example,dc=com`.
- Generated Vault tokens expire 1 hour after creation.
- Generated Vault tokens can be renewed for up to 8 hours after creation.

```
vault write auth/ldap/config \
 url="ldap://ldap.example.com" \
 userattr="sAMAccountName" \
 userdn="ou=Users,dc=example,dc=com" \
 groupdn="ou=Groups,dc=example,dc=com" \
 groupattr="cn" \
 binddn="cn=vault,ou=users,dc=example,dc=com" \
 bindpass="My$ecr3tP4ss" \
 certificate=@ldap_ca_cert.pem \
 insecure_tls=false \
 starttls=true \
 discoverdn=false \
 token_ttl=1h \
 token_max_ttl=8h
```

For more example configurations, see the LDAP Auth Method documentation. For details on specific parameters, see the LDAP Auth API.

Step 3: Assign Vault policies to LDAP groups

Next, you will map existing LDAP groups to the appropriate Vault policies. This allows you to manage group membership and Vault access within LDAP, removing the need to create and manage groups within Vault.

Configuration for root namespace

The command below shows an example where the `vault-operators` group in LDAP is mapped to the operator policy created earlier in this document.

```
$ vault write auth/ldap/groups/vault-operators policies=operator
```

Configuration for tenant namespace

The command below shows an example where the `vault-operators` group in LDAP is mapped to the operator policy created earlier in this document.

```
$ vault write auth/ldap/groups/vault-operators policies=operator
```

Step 4: Validate LDAP auth configuration

You can validate the LDAP auth configuration with either the Vault UI, CLI, or API. Login using the CLI with the following command.

Configuration for root namespace

Login to the root namespace as a Vault cluster operator.

```
$ vault login -method=ldap username=my-operator password=passw0rd
```

> Note
>
> In order to access the tenant namespace, operators must first login via the root namespace. Once authenticated, the token granted can be used to operate on the tenant namespace.

Configuration for tenant namespace

Login to the tenant namespace as a tenant administrator and validate that the admin policy is attached to the returned token.

```
$ vault login -method=ldap username=nwong password=passw0rd

Success! You are now authenticated. The token information displayed below
is already stored in the token helper. You do NOT need to run "vault login"
again. Future Vault requests will automatically use this token.

Key                    Value
---                    -----
token                  hvs.root.token
token_accessor         EbPflE8su5iRb2qD8KydXsUf.7fKdt
token_duration         1h
token_renewable        true
token_policies         ["default" "admin"]
identity_policies      []
policies               ["default" "admin"]
token_meta_username    nwong
```

> **Note**
>
> Users who are mapped in the tenant namespace can still login to the root namespace. However, their access is restricted by the default policy since there is no external group and policy mapping for these users in the root namespace.

Reference material

- LDAP Auth Method documentation

4.3 Revoke the root token

> **Note**
>
> This section is not necessary for HCP Vault.

For Vault Enterprise, the root token is attached to the root policy, which is capable of performing every operation for all paths in Vault. After you have validated that you can login using your first auth method, revoke the root token to eliminate the risk of exposure.

Revoke the root token using the following CLI command.

```
$ vault login <your root token>

Success! You are now authenticated. The token information displayed below
is already stored in the token helper. You do NOT need to run "vault login"
again. Future Vault requests will automatically use this token.

Key                  Value
---                  -----
token                hvs.root.token
token_accessor       JlM24NC1dk6MZLtup1XrKHYM
token_duration ∞
token_renewable      false
token_policies       ["root"]
identity_policies    []
policies             ["root"]


$ vault token revoke <your root token>
Success! Revoked token (if it existed)
```

Tip

In case of an emergency when a root token is absolutely necessary (for example, loss of auth method preventing admin access requiring break glass access to Vault), you should generate a new root token using the `operator generate-root` command. See this tutorial that demonstrates the steps to regenerate a root token.

5 Authentication for applications

In the previous section, we discussed the considerations for choosing an authentication method for human users. Since a human user will likely not authenticate with Vault in the same manner as a programmatic access request or application, there is a different set of considerations to take into account when choosing an authentication strategy for machine access.

In order for a client to interact with Vault's API, it must pass an authentication token along with the request. An authentication token is granted after the client proves its identity to Vault via an authentication method. For a human user, providing a proof of identity is straightforward using a user ID and password. However, this is more challenging for applications because you need a secure way to provide that initial credential for the application to authenticate with Vault. This problem is often referred to as the *secret zero* or secure introduction challenge. How does a secret consumer, an application or machine, prove that it is the legitimate recipient for a secret so that it can acquire a token? How can you avoid persisting raw token values during your secure introduction?

5.1 Secure introduction approach

There are two approaches to securely authenticate a secret consumer.

- Platform integration
- Trusted orchestrator

5.1.1 Platform integration

In the platform integration model, Vault trusts the underlying platform (e.g. AWS, Azure, GCP, Kubernetes etc.) which assigns a token or cryptographic identity (such as IAM token, signed JWT) to virtual machine, container, or serverless function.

Vault uses the provided identifier to verify the identity of the client by interacting with the underlying platform. After the client identity is verified, Vault returns a token to the client that is bound to their identity and policies that grant access to secrets.

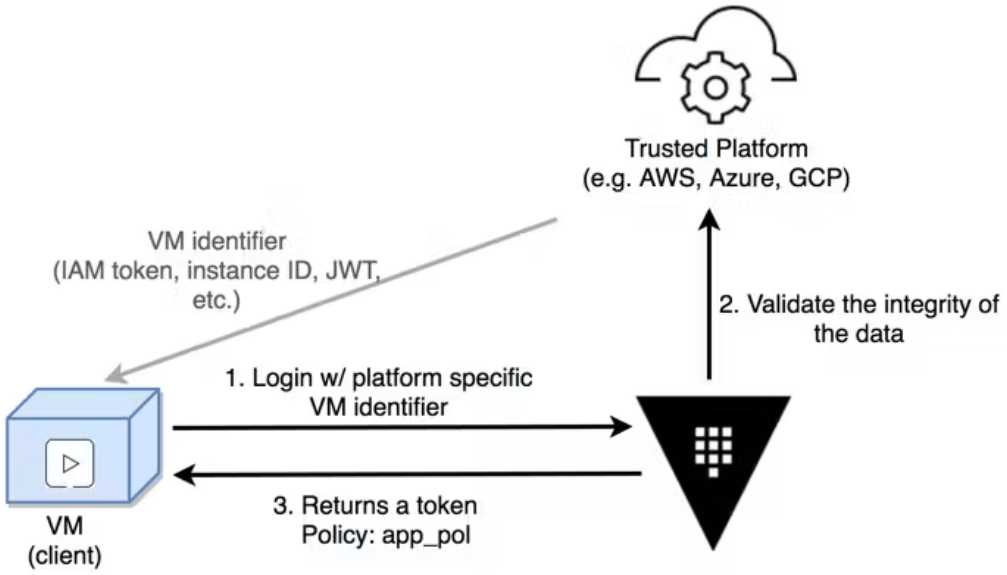


Figure 8: Platform integration

For example, suppose we have an application running on a virtual machine in AWS EC2. When that instance is started, an IAM token is provided via the machine local metadata URL. That IAM token is provided to Vault, as part of the AWS auth method, to login and authenticate the client. Vault uses that token to query the AWS API and verify the token validity and fetch additional metadata about the instance (account ID, VPC ID, AMI, region, etc). These properties are used to determine the identity of the client and to distinguish between different roles (e.g. a web server versus an API server).

Once validated and assigned to a role, Vault generates a token that is appropriately scoped and returns it to the client. All future requests from the client are made with the associated token, allowing Vault to efficiently authenticate the client and check for proper authorizations when consuming secrets.

Use case

When the client app is running on a compute construct (e.g. container, VM, or serverless function)

hosted on a supported cloud provider, you can leverage the corresponding auth method to authenticate with Vault.

Reference materials

- AWS Auth Method
- Azure Auth Method
- GCP Auth Method
- Kubernetes Auth Method

5.1.2 Trusted orchestrator

In the trusted orchestrator model, you have an orchestrator which is already authenticated against Vault with privileged permissions. The orchestrator launches new applications and injects a mechanism they can use to authenticate (e.g. PKI cert, token, AppRole etc.) with Vault.

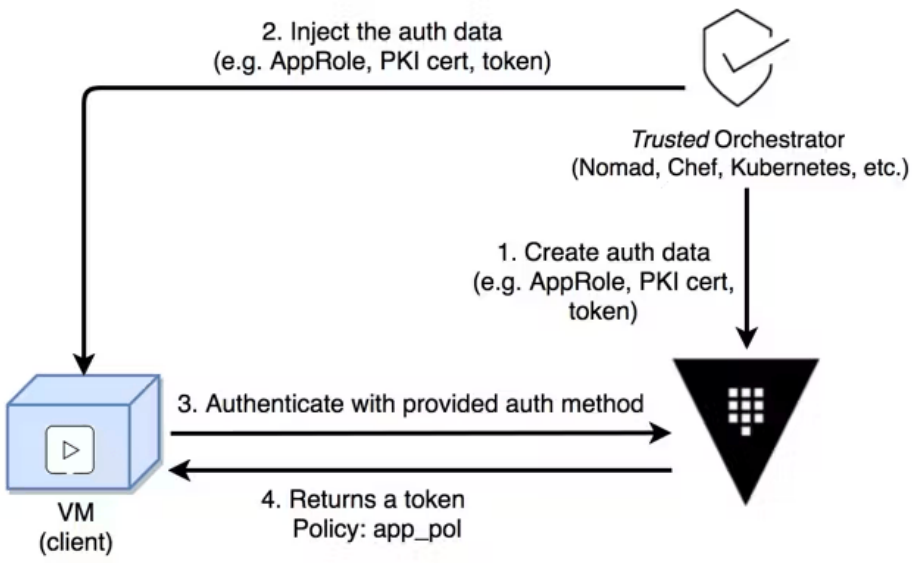


Figure 9: Trusted orchestrator model

For example, suppose Terraform is being used as a trusted orchestrator. This means Terraform already has a Vault token, with enough capabilities to generate new tokens or create new credentials. Terraform can interact with platforms such as VMware to provision new virtual machines.

VMware does not provide a cryptographic identity, so a platform integration is not possible. Instead, Terraform can create and inject an AppRole credential into a new machine, acting as a trusted orchestrator that extends trust to the machine, allowing it or its application to authenticate against Vault using the injected credentials.

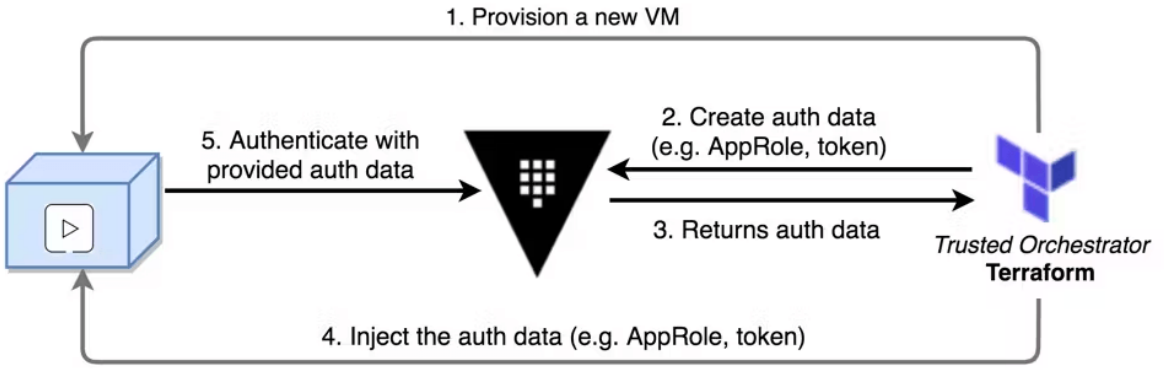


Figure 10: Terraform trusted orchestrator

Use case

When you are using an orchestrator tool such as Chef to launch applications, this model can be applied in an environment where platform integration isn't possible (e.g. VMware host). See the section on AppRole auth method for more details.

Reference materials

- AppRole Auth Method
- TLS Certificates Auth Method
- Token Auth Method

5.1.3 Vault agent

Vault Agent is a client daemon which automates the workflow of client login and token refresh. It can be used with either platform integration or trusted orchestrator approaches. Vault Agent and its usage is covered in more detail in the Consumption of Secrets section later in this document.

5.2 Configure machine auth method

We recommend that customers use the most specific auth method for their environment. In this section, we will cover two common machine auth methods that organizations utilize: cloud provider auth method and the AppRole auth method.

5.2.1 Cloud provider auth method

The basic steps to configure a cloud provider auth method are as follows.

1. Enable the auth method
2. Configure credentials to allow Vault access to the cloud provider API
3. Create a Vault policy specifying the resources that the application can access
4. Create a Vault role to associate a cloud entity to a set of Vault policies

In this section, you will learn how to configure the AWS auth method.

AWS auth method overview

The `aws` auth method provides two ways to authenticate to Vault using either `iam` principals or `ec2` instances.

> Note
>
> The `ec2` method was implemented before the primitives to implement the `iam` method were supported by AWS. The `iam` method is the recommended approach as it is more flexible and aligns with best practices to perform access control and authentication. Therefore, we will focus on the `iam` method in this document. See the documentation comparing the two auth methods for more information.

With the `iam` method, an authenticating client creates a signed GetCallerIdentity AWS request and uses it to authenticate with Vault. The GetCallerIdentity request is signed using the IAM credentials that are automatically supplied to AWS instances in IAM instance profiles, Lambda functions, and others. This signed AWS request is included in the login request sent to Vault. When Vault receives the login request, it can execute the STS query without needing to know its contents. Depending on the response from the STS service, Vault then authenticates the client by comparing the IAM principal's ARN mapped to the Vault role used for authentication. Clients authenticating to Vault

must have an ARN that matches one of the ARNs bound to the role they are attempting to login to.

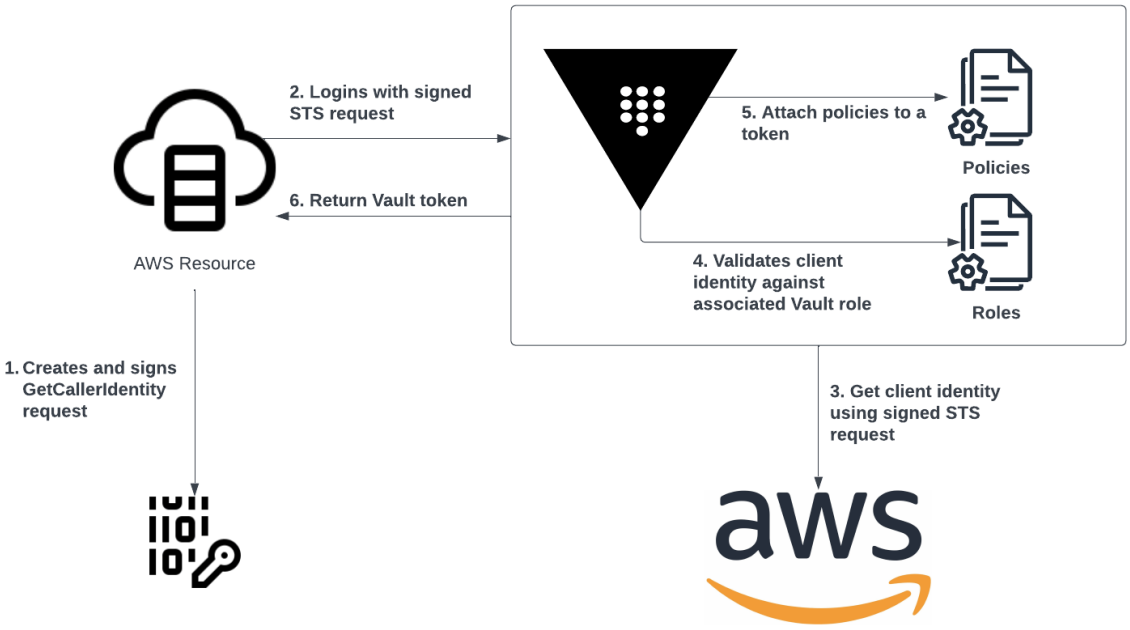


Figure 11: AWS auth method

The steps below are examples of how to configure the AWS auth method using the `iam` method. For details on specific settings, please refer to the AWS Auth Method documentation and the AWS Auth Method API.

Configuring AWS auth method

Step 1: Enable the auth method

Similar to configuring an auth method for human access, you must enable the AWS auth method before you can use it. You can enable and configure the auth method using the UI, CLI, or the API. We will focus on CLI commands in this document.

Using the Vault CLI, log in to Vault as an admin user using the auth method you configured as part

of the initial cluster configuration. Next, enable the AWS auth method under the tenant namespace using the commands below.

```
$ export VAULT_ADDR=https://vault:8200
$ export VAULT_NAMESPACE=<tenant namespace>
$ vault auth enable aws
```

Step 2: Configure auth method with AWS credentials

Next, you need to configure the auth method with a credential that would allow Vault to make AWS API calls to validate the identity of a requesting client. If Vault is deployed on AWS EC2 instances, then Vault will attempt to use standard environment variables (`AWS_ACCESS_KEY_ID` and `AWS_SECRET_ACCESS_KEY`) or IAM EC2 instance role credentials without having to embed credentials into the auth method configuration.

The IAM account or role to which the credentials map must allow `iam:GetUser` and `iam:GetRole`. These permissions are used when using the `iam` auth method and binding to an IAM user or role principal to determine the AWS IAM Unique Identifiers or when using a wildcard on the bound ARN to resolve the full ARN of the user or role. An example policy can be found in our Recommended Vault IAM Policy documentation.

For Vault servers deployed outside of AWS, use the commands below to configure the client credential, substitute the `secret_key` and `acces_key` with your own values that have the permissions described previously.

```
$ vault write auth/aws/config/client \
    secret_key=vCtSM8ZUEQ3mOFVlYPBQkf2sO6F/W7a5TVzrl3Oj \
    access_key=VKIAJBRHKH6EVTTNXDHA
```

Step 3: Configure the X-Vault-AWS-IAM-Server-ID header

We recommend requiring an additional header, `X-Vault-AWS-IAM-Server-ID`, for the AWS auth method to mitigate against different types of replay attacks. Clients must include this header in the headers of the login request, and further this header must be among the signed headers validated by AWS. This is to protect against different types of replay attacks, for example a signed request sent to a dev server being resent to a production server. This header should be set to the Vault server's DNS name.

```
$ vault write auth/aws/config/client \
    iam_server_id_header_value=vault.example.com
```

Step 4: Configure the policies on the role

Next, create a role to map the IAM principal to Vault policies.

```
$ vault write auth/aws/role/demo auth_type=iam \
    bound_iam_principal_arn=arn:aws:iam::123456789012:role/MyRole \
    policies=default \
    token_ttl=8h \
    max_ttl=8h
```

The command above creates a new role called *demo* and associates it to the *default* policy. The *default* policy is used here as an example since you have not enabled any secrets engines at this point. You will create additional roles and map them to policies granting access to the KV secrets engine later in this document. The `bound_iam_principal_arn` defines the list of IAM principals that are permitted to login to the role using the `iam` auth method. Wildcards are supported at the end of the ARN, e.g. "arn:aws:iam::123456789012:*" will match any IAM principal in the AWS account 123456789012. The `token_ttl` specifies the incremental lifetime for generated tokens and the `max_ttl` specifies the maximum lifetime for generated tokens.

Step 5: Test the login

The easiest way to test the login is to create an EC2 instance and attach it to an instance profile with the `bound_iam_principal_arn` IAM roles specified in the previous step. Once you deploy the test EC2 instance, install the Vault binary.

Test the login using the Vault CLI. The CLI will generate the signed STS request for you.

```
$ export VAULT_ADDR=https://vault:8200
$ export VAULT_NAMESPACE=<tenant namespace>
$ vault login -method=aws header_value=vault.example.com role=demo

Success! You are now authenticated. The token information displayed below
is already stored in the token helper. You do NOT need to run "vault login"
again. Future Vault requests will automatically use this token.

Key                      Value
---                      -----
token                    hvs.root.token
token_accessor           FgsZpKvvD0NYbbGGETfn08Ig.Ojft7
token_duration           8h
token_renewable          true
token_policies           ["default"]
identity_policies        []
policies                 ["default"]
token_meta_account_id    258559098798
token_meta_auth_type     iam
token_meta_role_id       eec3784b-e51c-7c14-e263-8c3a64b6be07
```

Another way to test the login without an EC2 instance is to directly pass the IAM credentials on the command line (not recommended for production usage), omitting `aws_security_token` if not applicable.

```
$ vault login -method=aws header_value=vault.example.com role=demo \
    aws_access_key_id=<access_key> \
    aws_secret_access_key=<secret_key> \
    aws_security_token=<security_token>
```

Reference material

- AWS Auth Method

5.2.2 AppRole auth method

The AppRole auth method allows machines or applications to authenticate with Vault using custom roles defined in Vault. It is typically used in an environment where platform integration is not possible (e.g. VMware host). The AppRole auth method is designed to enable automated workflows for a large number of applications, where a unique role is created for each application that wants to

access Vault. Each role is attached to a set of Vault policies, which restricts the access needed for the application.

Each role configured under the AppRole auth method is associated with a RoleID and SecretID, which in essence is a username and password for machines. During application authentication with Vault, both the RoleID and SecretID are included in the authentication request. Upon successful validation of the request, Vault returns a token. The application uses the token to make subsequent requests to Vault, such as reading secrets, generating dynamic credentials, or encrypting data.

> **Note**
>
> While AppRole is a valid use case, use this auth method as a last resort. Relying on alternative platform authentication methods is inherently more secure because AppRole can be misconfigured, leading to a less secure posture than is possible.

> **Note**
>
> Multiple SecretIDs can be created for the same RoleID. The value of each SecretID for the same RoleID is unique and they can all be used to authenticate to Vault. For this reason, it is important to set a TTL for each SecretID generated to prevent the risk of a leaked SecretID. See the AppRole Auth Best Practices for more details.

Below is a basic workflow for the AppRole auth method.

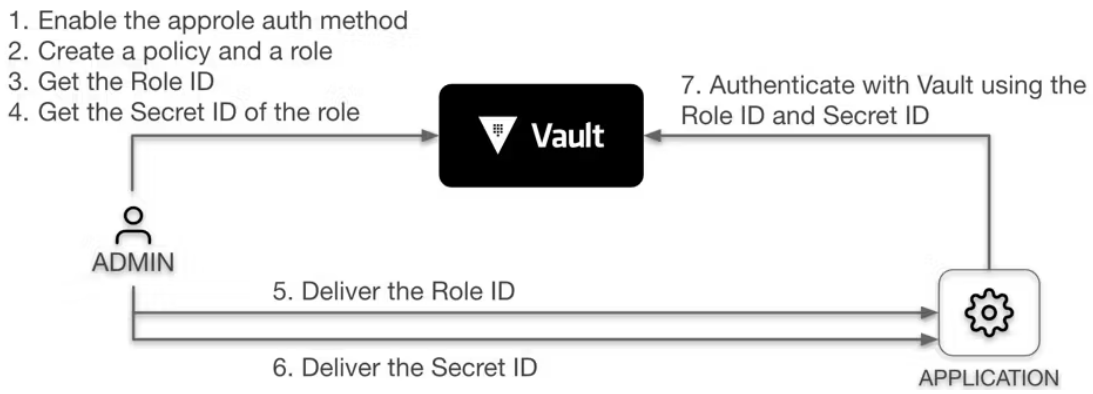


Figure 12: AppRole Auth Method

Configuring AppRole auth method

Step 1: Enable the auth method

You must enable the AppRole auth method before you can use it.

Using the Vault CLI, login to Vault as an administrator using the auth method you configured as part of the initial cluster configuration. Next, enable the AppRole auth method under the tenant namespace using the commands below.

```
$ export VAULT_ADDR=https://vault:8200
$ export VAULT_NAMESPACE=<tenant namespace>
$ vault auth enable approle
```

> Note
>
> Use the `path` parameter to customize the path where you want to mount the auth method. For example, `vault auth enable -path=apps approle`.

Step 2: Create a policy and a role

Next, create a role and map it to Vault policies.

```
$ vault write auth/approle/role/demo \
secret_id_num_uses=1 \
secret_id_ttl=15m \
token_policies="default" \
token_ttl=1h \
token_max_ttl=1h
```

The command above creates a new role called _demo _and associates it to the *default* policy. The *default* policy is used here as an example since we have not enabled any secrets engines yet. You will create additional roles and map them to policies granting access to the KV secrets engine later in this document.

The `secret_id_num_uses` parameter determines how many times a particular SecretID generated for this role can be used to authenticate with Vault. Setting this value to 1 effectively makes any generated SecretID a one time password. This helps to prevent the SecretID from being reused elsewhere. The default value is 0, which will allow unlimited uses of the SecretID. The `secret_id_ttl` parameter sets the TTL of any SecretID generated for this role. Setting this value ensures that any generated SecretIDs that did not get used expire automatically. We recommend that you always set these two settings to reduce the attack surface in case the SecretID was accidentally leaked.

> **Note**
>
> Limiting the SecretID TTL and its number of uses means that a new SecretID will need to be delivered to the application once the application's Vault token reaches its maximum TTL and can no longer be renewed.

> **Tip**
>
> Also consider setting secret_id_bound_cidrs to restrict the source IP range of the connecting devices.

You can use the command below to view the details of the role you just configured.

```
$ vault read auth/approle/role/demo

Key                        Value
---                        -----
bind_secret_id             true
local_secret_ids           false
secret_id_bound_cidrs      <nil>
secret_id_num_uses         1
secret_id_ttl              15m
token_bound_cidrs          []
token_explicit_max_ttl     0s
token_max_ttl              1h
token_no_default_policy    false
token_num_uses             0
token_period               0s
token_policies             [default]
```

Step 3: Get the RoleID

The next step is to generate the RoleID. The RoleID is static and is not considered sensitive information, similar to a username.

Read the RoleID using the command below.

```
$ vault read auth/approle/role/demo/role-id
Key        Value
---        -----
role_id    04be0443-b30b-f5ab-8803-9afff043919b
```

Step 4: Get the SecretID

Next, generate a SecretID for the role.

```
$ vault write -force -field=secret_id auth/approle/role/demo/secret-id
```

Step 5: Authenticate with Vault

Login to Vault using your RoleID and SecretID.

```
$ vault write auth/approle/login \
    role_id=04be0443-b30b-f5ab-8803-9afff043919b \
    secret_id=a4173afc-6a04-0acc-7adc-6ab78445e752

Key                    Value
---                    -----
token                  hvs.root.token
token_accessor         ugevnwKij1v9xE4XRWO9w5Lx.Ojft7
token_duration         1h
token_renewable        true
token_policies         ["default"]
identity_policies      []
policies               ["default"]
token_meta_role_name   demo
```

Note that in step 2 we configured the TTL of the SecretID to be 15 minutes. If you see the error below, generate a new SecretID. Note that the namespace in the example below, `admin`, is specific to HCP Vault but the equivalent (`/`) is just as relevant in self-managed instances of Vault Enterprise.

```
Error writing data to auth/approle/login: Error making API request.

Namespace: admin/
URL: PUT https://dev.vault.org:8200/v1/auth/approle/login
Code: 400. Errors:

* invalid secret id
```

AppRole auth best practice

As we alluded to earlier, the AppRole auth method is typically used with a trusted orchestrator when a platform identity is not available. In that scenario, the trusted orchestrator is responsible for the delivery of the AppRole credential to the application.

A potential risk with the trusted orchestrator pattern is that if the trusted orchestrator is responsible for delivering the AppRole credentials, it would also have access to the applications' secrets. If the

trusted orchestrator is compromised, then application secrets would be at risk as well.

In order to prevent any one system, other than the target client, from obtaining the complete set of credentials (RoleID and SecretID), we recommend implementing AppRole auth method to deliver those values separately through two different channels. This enables you to provide narrowly-scoped tokens to each trusted orchestrator to access either RoleID or SecretID, but never both.

The diagram below illustrates the recommended workflow to securely deliver the RoleID and SecretID to the target application.

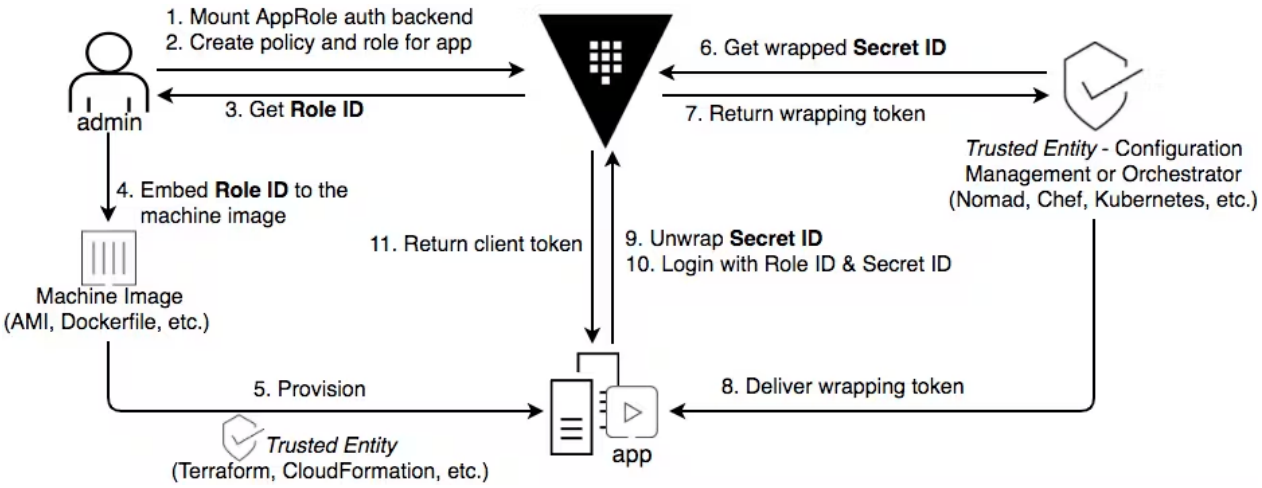


Figure 13: AppRole Auth Method Organization

Step 1

Take the time to consider role management before enabling the auth method. If you intend to delegate role management to others, assess whether they require full access to all roles. If not, it might be beneficial to enable AppRole at multiple paths, enabling targeted policies for managing specific sets of roles.

Step 2

Create roles and policies for your applications. Each role is associated with a set of Vault policies that define the secrets an application can access. Follow the principle of least privilege for each application and map each application to a role that grants access to the secrets deemed necessary - if two applications use the same role, they will have access to the same secrets.

Remember, roles can have multiple attached policies. To handle shared and unique secrets among apps, craft one policy for common secrets and app-specific policies for unique needs.

When creating roles, consider using these features when possible:

1. TTL, usage limits, and source CIDR limits for the SecretID to control authentication.
2. The same limits should be applied separately to the token generated upon authentication.

For example, to limit the number of uses of the SecretID, set the secret_id_num_uses on the role definition.

```
vault write auth/approle/role/jenkins token_policies="jenkins" \
     token_ttl=1h token_max_ttl=4h \
     secret_id_num_uses=10
```

Step 3 and 4

Request and obtain the RoleID for the role. The RoleID is not a secret value. It's a static UUID that identifies a specific role configuration. Because it is not a secret, it can be embedded into an application-related artifact, such as the VM image or container as a text file or environment variable.

For example:

- Build an image with Packer with RoleID stored as an environment variable.
- Use Terraform to provision a machine embedded with RoleID.

There are a number of different patterns through which this value can be delivered.

The application running on the machine or container will read the RoleID from the file or environment variable to authenticate with Vault.

An appropriate policy is required to read RoleID from Vault. For example, to get the RoleID for a role named, "jenkins", the policy should look as below.

```
# Grant 'read' permission on the 'auth/approle/role/<role_name>/role-id' path
path "auth/approle/role/jenkins/role-id" {
capabilities = [ "read" ]
}
```

Step 5

At this point, the running application has half the necessary authentication information, with the remaining portion provided in the following step.

> Note
>
> Steps 6–8, which generates and delivers the SecretID, are sometimes done before step 5. However, the order of steps laid out here are the most secure way to deliver the SecretID to ensure that the application is fully started before the SecretID is delivered for immediate use. This minimizes the risk of exposure of an unconsumed SecretID. Ideally, application instances should wait for the SecretID and consume it immediately, or they should terminate and restart (e.g. by a supervising orchestrator) until the SecretID is available. In some cases, pre-delivery of the SecretID may be necessary if the application cannot gracefully handle startup before the SecretID is present.

Step 6 and 7

Your runtime platform now will need to request and obtain a SecretID to be provided to the application. Use a Vault identity for this that is associated with policies that allow only this operation. For example, to get the SecretID for a role named, "jenkins", the policy should look as below.

```
# Grant 'update' permission on the 'auth/approle/role/<role_name>/secret-id' path
path "auth/approle/role/jenkins/secret-id" {
   capabilities = [ "update" ]
}
```

The SecretID is equivalent to a password for its corresponding RoleID. Since it is a secret and should only be read by the intended recipient, additional considerations need to be accounted for when distributing the SecretID.

1. Binding CIDRs
2. AppRole response wrapping

Binding CIDRs

When defining an AppRole, you can use the secretid_bound_cidrs parameter to specify blocks of IP addresses which can perform the login operation for this role. You can further limit the IP range per token using token_bound_cidrs.

```
vault write auth/approle/role/jenkins \
      secret_id_bound_cidrs="0.0.0.0/0","127.0.0.1/32" \
      secret_id_ttl=60m \
      secret_id_num_uses=5 \
      enable_local_secret_ids=false \
      token_bound_cidrs="0.0.0.0/0","127.0.0.1/32" \
      token_num_uses=10 \
      token_ttl=1h \
      token_max_ttl=3h \
      token_type=default \
      period="" \
      policies="default","test"
```

AppRole Response Wrapping

To guarantee confidentiality, integrity, and non-repudiation of SecretID, you can use the `-wrap-ttl` flag when generating the SecretID. Instead of providing the SecretID in plaintext, Vault response-wraps the SecretID and returns a single-use token that can be used for an "unwrap" operation in the Vault API. When unwrapping, Vault returns the underlying secret - in this case an AppRole SecretID.

Example: The following CLI command retrieves the SecretID for a role named, "jenkins". The generated SecretID is wrapped in a token which is valid for 60 seconds to unwrap.

```
vault write -wrap-ttl=60s -force auth/approle/role/jenkins/secret-id

Key Value
--- -----
wrapping_token: s.yzbznr9NlZNzsgEtz3SI56pX
wrapping_accessor: Smi4CO0Sdhn8FJvL8XvOT30y
wrapping_token_ttl: 1m
wrapping_token_creation_time: 2021-06-07 20:02:01.019838 -0700 PDT
wrapping_token_creation_path: auth/approle/role/jenkins/secret-id
```

Finally, you can monitor your audit logs for attempted read access of your SecretID. If Vault throws a use-limit error when an application tries to read the SecretID, you know that someone else has read the SecretID and alert on that. For example, if you are using this pattern for CI/CD pipelines and a SecretID is generated for a pipeline role but no pipelines are running. The audit logs will indicate where the SecretID read attempt originated.

You can ensure response wrapping by applying a policy enforcing the application of TTLs on SecretID creation.

```
# This effectively makes response wrapping mandatory for this path by setting
    min_wrapping_ttl to 1 second.
# This also sets this path's wrapped response maximum allowed TTL to 90 seconds.
path "auth/approle/role/my-role/secret-id" {
    capabilities = ["create", "update"]
    min_wrapping_ttl = "1s"
    max_wrapping_ttl = "90s"
}
```

Step 8

Deliver the SecretID wrapping token to the authorized application using your preferred delivery method. For example, you can push it via an API call to your application or have the application pull it by reading from a file. Whichever delivery method you use, ensure that it is not the same channel used for delivering the RoleID. It is also recommended to promptly remove the wrapping token once it is used for authentication (if you are using the Vault agent, it will attempt to delete the SecretID file by default after it has been read).

> **Note**
>
> We recommend configuring your CI/CD pipeline to raise a security alert to the Platform Team should this method result in a 500 error being received by an application attempting to use the wrapping token. Wrapping tokens are single use only, and a 500 API response code from Vault indicates that this token is now invalid suggesting it has already been used. We recommend testing this alerting mechanism to ensure it works as expected.

Step 9-11

The application unwraps the SecretID, authenticates with Vault using the RoleID and SecretID, and obtains a Vault token. If you are using the Vault Agent, it automatically unwraps the SecretID when a wrapping token is detected.

Remember, it is crucial for the application to promptly unwrap and use the SecretID. Delaying its use increases the risk of an exposed credential, expiration of the wrapping token or SecretID, and the potential for a compromised token to go undetected until authentication is attempted.

Reference material

- AppRole auth method documentation

- AppRole auth method API
- How (and Why) to Use AppRole Correctly in HashiCorp Vault
- Recommended Vault AppRole usage pattern (Jenkins example)
- AppRole usage best practices

6 MFA administration

Multi-Factor Authentication (MFA) is a system that requires more than one form of authentication to verify the identity of a user for a login or transaction. Typically implemented for governing human access to a system, MFA works by requiring any two or more of the following verification methods:

- Something the user knows (their password)
- Something the user owns (security token, one-time code, push notification)
- Something the user is (biometric information such as their fingerprint or retinal pattern)

6.0.1 Vault MFA capabilities

Vault incorporates two types of MFA:

1. Login MFA - Additional layers of security to login workflows
2. Vault Enterprise "step up" MFA - An enhanced security layer to protect sensitive API endpoints.

6.0.2 1. Login MFA via OpenID Connect (OIDC) or external identity provider

Many external identity providers (IDPs) like Okta or Ping Identity enable login multi-factor authentication (MFA) on their platforms. While MFA methods vary across providers, they typically mandate MFA verification during the authentication process, before granting access to applications. If an identity provider supports the OpenID Connect (OIDC) protocol, you can set up Vault's OIDC authentication method as outlined below. This setup ensures that users must complete the MFA process established at the IDP level for Vault access.

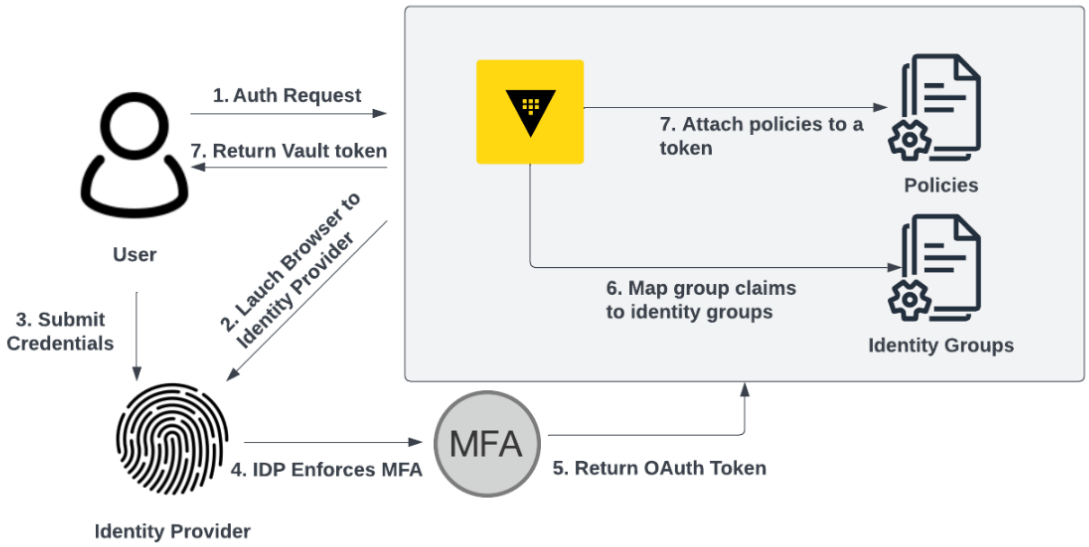


Figure 14: Login MFA

Figure 1: OIDC Auth with MFA Workflow

This approach is preferred when the IDP supports MFA because it allows you to keep all human identity management, access control, and MFA configurations in one place. It also provides scalability as your organization grows. MFA for login will only need to be configured in one place (the IDP) as opposed to separately setting up MFA in a consuming application such as Vault.

Vault Login MFA with LDAP and TOTP

The Vault HVD specifies two different human authentication methods that can be configured for organization-wide human access to Vault: OIDC and LDAP (Lightweight Directory Access Protocol). While many OIDC-compliant IDPs support implementing MFA at the identity provider level, it is less common for LDAP providers to offer native MFA support. In cases where an LDAP provider doesn't support MFA, Vault allows for MFA configurations through the LDAP method with Time-Based One-Time Password (TOTP). TOTP requires configuring a pre-shared key between Vault and a client mobile application or hardware device. The client uses a combination of the pre-shared key and the current time to generate a unique code that is only valid for a specific amount of time. This code is used as the additional factor to complete login.

Although TOTP is usually the best method when an external provider is not available, Vault also integrates with other platforms like Okta or Ping. However, it's worth noting that if Okta or Ping is accessible, the recommended approach is to configure MFA at the IDP level instead of separately within Vault to maintain unified access control and ease of management.

Due to the enrollment process involving sharing a key between Vault and an end-user's mobile device, the TOTP method is not suitable for enrolling large numbers of users. It is therefore recommended to implement TOTP with LDAP authentication on the relatively smaller number of highly privileged users (Super administrators and regular administrators) to protect their access to Vault.

To utilize this method, a TOTP MFA method needs to be configured in Vault and associated with the organization LDAP auth method. Each user will need to enroll their device with a pre-shared key generated by Vault.

Configuration

The steps for configuration of TOTP MFA enforcement on an LDAP auth mount are listed below. Note that steps 3 and 4 will need to be repeated for each user that needs TOTP MFA setup.

Step 1: Enable the LDAP auth method

Enable LDAP prior to TOTP configuration. Refer to the Vault Administration Guide to configure LDAP authentication for your organization.

Step 2: Enable login MFA method

Enable a Login MFA method.

Note the `method_id` that is returned by Vault. This ID will be required to generate a QR code that will configure an end-user TOTP device.

**NOTE: **Consider the algorithms supported by your end-user authenticator application. For example, Google Authenticator for Android supports only SHA1 as the value of the algorithm. The Vault MFA API defaults to the SHA1 algorithm, if no value is supplied during configuration.

Step 3: Login with the user for which MFA should be configured

The user who needs to have MFA configured should log in to Vault.

Look up the user's token properties to capture the user's `entity_id` generated by Vault.

Step 4: Generate a TOTP setup QR code

Generate a PNG image of the TOTP QR code for the user. This QR Code should be scanned by the appropriate user with a TOTP generation app such as Google Authenticator. Use the TOTP `method_id` and user `entity_id` noted earlier.

Delete the QR Code image after the scanning process is complete.

Step 5: Enforce MFA on LDAP login

In order to enable MFA enforcement, you will need the accessor value of the LDAP auth mount as well as the TOTP `method_id` noted earlier. Capture the accessor value for the LDAP auth method mount.

Create the enforcement by tying the MFA `method_id` to the LDAP auth mount accessor.

Step 6: Test MFA enforcement on LDAP auth method

Test the enforcement of TOTP MFA on the LDAP auth method by logging in with the Vault CLI.

6.0.3 2.Vault Enterprise Step Up MFA

Vault Enterprise and HCP Vault feature a "step up" MFA workflow that requires a user to present an additional identity factor when issuing specific API requests, even if they have already successfully authenticated to Vault.

Guidance: In a situation where highly critical secrets are stored in Vault for human users to access, it is recommended to implement a step up MFA policy that provides additional security and identity assurance on highly critical secrets in Vault.

****Note:** **Enterprise "step up" MFA cannot be configured on HCP Vault, the sys/mfa/ path is only accessible from the root namespace, which is not available to the end user in HCP Vault.

For example, if a static secret were stored at the key value path `secret/topsecret/data/foo`, a "step-up" MFA policy could be configured to require that any authenticated user accessing the path complete an MFA workflow, prior to being granted access to the secret. This provides additional assurance that the user accessing the path is the same user that originally authenticated, and that their Vault token has not been compromised.

For "step-up" MFA, Vault supports TOTP, Okta push notification, Duo push notification, and PIngID push notification methods. For ease of enrollment, it can be useful to leverage any pre-existing IDP MFA capabilities that may exist in your environment. However, if there are no IDP provided MFA methods available, it is recommended to implement TOTP MFA in Vault.

Configuration

This section will cover configuration of TOTP for "step-up" MFA in Vault. For other methods, steps 1, 2, and 3 will differ. However, steps 4 and 5, which link the MFA method to a specific Vault ACL policy, will remain the same for other MFA methods.

The configuration steps below assume that you already have a human auth method such as OIDC or LDAP configured on your Vault instance. Note that step 2 and 3 will need to be repeated for each user that needs to enroll in TOTP.

Step 1: Configure the TOTP MFA method

Create a TOTP MFA method that will be referenced in a Vault ACL policy to limit access to sensitive secrets.

**NOTE: **Consider the algorithms supported by your end-user authenticator application. For example, Google Authenticator for Android supports only SHA1 as the value of the algorithm. The Vault MFA API defaults to the SHA1 algorithm, if no value is supplied during configuration.

Make a note of the `id` that is returned by Vault from the `read` command. This ID will be required to generate a QR code that will configure an end-user TOTP device.

Step 2: Login with the user for which MFA should be configured

The user who needs to have MFA configured should log in to Vault.

Lookup the user's token properties to capture the user's entity_id generated by Vault.

Step 3: Generate a TOTP setup QR code

Generate a PNG image of the TOTP setup barcode for the user. This QR code should be scanned by the appropriate user with a TOTP generation app such as Google Authenticator. Use the TOTP `method_id` and user `entity_id` noted earlier.

Delete the QR image after the scanning process is complete.

Step 4: Create policy enforcing MFA method on sensitive secret

Create a policy that gates access to a sensitive secret through the previously created MFA method.

Note that this policy will need to be applied to the appropriate users who should have access to this secret. Configure the appropriate auth method and/or role to assign this policy to the appropriate users.

Step 5: Test accessing the secret

When reading the secret, the `mfa` flag needs to be supplied containing the appropriate TOTP code, otherwise Vault will not grant access to the secret.

7 Static secrets management

> **Note**
>
> This page covers the operator setup and management of static secrets. For how users consume static secrets, see the User Guide: Static secrets.

7.1 Overview

With the initial cluster configuration completed and auth methods enabled, you are now ready to enable your first use case. While Vault has many capabilities around cryptography and secrets management, the initial use-case is often the manual creation of static secrets and secure storage and retrieval.

Vault's KV secrets engine stores arbitrary static secrets. Secrets can be anything from passwords to database connection strings to API keys. Vault stores these securely, and the nature of the static secret is unimportant. What is important to consider is the workflow around the lifecycle of the secrets, namely who or what has permission to create, read, update, and delete any particular secret.

Vault handles this through the use of authentication and policy.

- Authentication: Any interaction with Vault must be first authenticated. Authentication verifies the identity of a Vault client (e.g. user, machine, or application) to interact with Vault. It does not define any permissions on what that entity can look at inside Vault.
- Policy - Policies are associated with Vault tokens and define both what secrets the authenticated client can interact with and in what ways. For example:

- userA can read secret1 and not read secret2.
- userB can update secret2 but not update secret1
- Any user in groupC can read secret3

This section discusses the following.

- Static secrets and what you should consider when laying out your secrets schema.
- Policies and what you should consider when creating them.
- Secret consumption and what you should consider when planning this strategy.

7.2 Static secrets management

At this point, you should have one human auth method (OIDC or LDAP) and one machine auth method (AWS or AppRole) enabled, but no secrets in Vault and no policies set up to enforce access permissions of an authenticated entity.

Before you populate Vault with any static secrets, it is important to understand how Vault stores static secrets and plan how you will organize them.

7.2.1 Key/Value (KV) secrets engine

The KV secrets engine is a generic key-value store used to store arbitrary secrets within the configured physical storage for Vault. Secrets written to Vault are encrypted and then written to backend storage. Therefore, the backend storage mechanism never sees the unencrypted value and doesn't have the means necessary to decrypt it without Vault.

The KV secrets engine comes in two versions: version 1 and version 2. Version 2 provides secret versioning and utilizes a different API compared to version 1. KV v1 is more performant than v2 since there are fewer storage calls due to the lack of additional metadata or history being stored. Migration from v1 to v2 is possible but requires that the secret engine be taken offline during the migration. The process could potentially take a long time depending on the amount of data stored. In general, the additional features provided by KV v2 offset the additional write and storage overhead. Therefore, KV v2 is recommended by default unless you have specific performance requirements. Working with KV v1 is out of scope for this document.

Working with the KV secrets engine is straightforward. You can interact with the secrets engine through the UI, CLI, or API. Vault static secrets are laid out like a virtual filesystem. The path where

you enable the KV secrets engine acts as the root of the file system. Everything after the root is a key-value pair to write to the secrets engine. For example, the commands below first enables the KV v2 secrets engine at the path `kvv2/`, then writes a new key-value secret to the path `myapp`, with a key of `foo` and value of `bar`. Finally, it reads the secret using the `vault kv get` command.

```
$ vault secrets enable -version=2 -path="kvv2" \
-description="Demo K/V v2" kv

Success! Enabled the kv secrets engine at: kvv2/

$ vault kv put -mount=kvv2 myapp foo=bar

= Secret Path =
kvv2/data/myapp

======= Metadata =======
Key                Value
---                -----
created_time       2023-09-18T20:32:54.482700528Z
custom_metadata    <nil>
deletion_time      n/a
destroyed          false
version            1

$ vault kv get -mount=kvv2 myapp

= Secret Path =
kvv2/data/myapp

======= Metadata =======
Key                Value
---                -----
created_time       2023-09-18T20:32:54.482700528Z
custom_metadata    <nil>
deletion_time      n/a
destroyed          false
version            1

=== Data ===
Key    Value
---    -----
foo    bar
```

7.2.2 Mount and path structure

Before writing any data to the KV secrets engine, it is important to consider how to organize the data under the secrets engine mount. The KV structure is important because it helps to simplify and reduce the number of Vault policies by grouping similar data under the same path.

Antipatterns for KV path structure

Some common antipatterns when designing a KV path structure are detailed below.

Creating paths based on environments

For example `secret/my-app/dev/*`, `secret/my-app/uat/*`, `secret/my-app/prod/*`. We do not recommend creating paths based on environments. Customers wanting to have Vault in different environments should run a separate Vault instance in each environment. For instance, if you have a dev, prod, and preprod environment, you should have a Vault instance in dev, prod, and preprod as well. If this is not feasible due to cost and overhead, you should at a minimum separate prod and non-prod. This allows Vault admins to:

1. Have a process for letting devs add secrets in lower environments
2. Maintain the same path structure across all environments, which minimizes need for application code changes.

> Note
>
> You should consider the increased load on your production cluster from non-prod clients, which might require you to provision hardware with better compute and storage performance.

Creating paths based on teams

We also do not recommend creating paths based on teams. This antipattern tightly couples application code to organizational topology, which may prove confusing in the event of team restructuring. We recommend instead to design paths based on the application concern. Generically, structure your paths as such: <app-name>/<service>/<component>/secret. For example, in a webstore app, you might have a billing service and an identity service, and within them you have secrets for specific components:

`webapp/billing/checkout/<secret1>` `webapp/identity/password-reset/<secret1>`

Recommended KV mount and path structure

We recommend that you mount a single KV v2 secrets engine with sub-paths per application concern. For example, in a webstore app with a 3-tier architecture (presentation, application, data), the mount structure would look like:



Figure 15: Mount structure

This mount structure provides several benefits:

- It reduces the potential of hitting mount table limits.
- It reduces operational complexity by centralizing all static secrets under a single mount

> **Note**
>
> There is a limit to the number of secret engines you can mount. For integrated storage, the maximum number of secret engine mount points is approximately 14000. See Vault limits and maximums for more information.

7.2.3 Configure the KV secrets engine

In this section, you will enable your first KV secrets engine and test out the API. Similar to all other auth methods and secrets engines, you must enable the KV secrets engine before you can use it. Login to your Vault cluster as an admin and use the command below to enable the KV v2 secrets engine using the CLI on a custom path under the tenant namespace.

```
$ export VAULT_NAMESPACE=<tenant namespace>
$ vault secrets enable -version=2 -path=secret kv

Success! Enabled the kv secrets engine at: secret/
```

Working with KV v2

Here we will briefly explore the functionalities of the KV v2 secrets engine. For a more thorough understanding, please review the tutorials and documentation below.

- KV secrets engine - version 2
- Versioned Key/value secrets engine

Writing data

```
$ vault kv put -mount=secret my-secret foo=a bar=b

==== Secret Path ====
secret/data/my-secret

======= Metadata =======
Key                Value
---                -----
created_time       2023-09-20T18:57:04.098790466Z
custom_metadata    <nil>
deletion_time      n/a
destroyed          false
version            1
```

Reading data

```
$ vault kv get -mount=secret my-secret

==== Secret Path ====
secret/data/my-secret

======= Metadata =======
Key                Value
---                -----
created_time       2023-09-20T18:57:04.098790466Z
custom_metadata    <nil>
deletion_time      n/a
destroyed          false
version            1

=== Data ===
Key    Value
---    -----
bar    b
foo    a
```

Write another version of secret

```
$ vault kv put -mount=secret my-secret foo=aa bar=bb
==== Secret Path ====
secret/data/my-secret

======= Metadata =======
Key                Value
---                -----
created_time       2023-09-20T18:58:31.636294687Z
custom_metadata    <nil>
deletion_time      n/a
destroyed          false
version            2
```

Reading the same secret now will return the latest version

```
$ vault kv get -mount=secret my-secret

==== Secret Path ====
secret/data/my-secret

======= Metadata =======
Key                Value
---                -----
created_time       2023-09-20T18:58:31.636294687Z
custom_metadata    <nil>
deletion_time      n/a
destroyed          false
version            2

=== Data ===
Key    Value
---    -----
bar    bb
foo    aa
```

Read a previous version by specifying the **-version** flag

```
$ vault kv get -mount=secret -version=1 my-secret

==== Secret Path ====
secret/data/my-secret

======= Metadata =======
Key                Value
---                -----
created_time       2023-09-20T18:57:04.098790466Z
custom_metadata    <nil>
deletion_time      n/a
destroyed          false
version            1

=== Data ===
Key    Value
---    -----
bar    b
foo    a
```

Delete the latest version of a secret

```
$ vault kv delete -mount=secret my-secret

Success! Data deleted (if it existed) at: secret/data/my-secret
```

Undelete a specific version of a secret

```
$ vault kv undelete -mount=secret -versions=2 my-secret

Success! Data written to: secret/undelete/my-secret

$ vault kv get -mount=secret my-secret

==== Secret Path ====
secret/data/my-secret

======= Metadata =======
Key                Value
---                -----
created_time       2023-09-20T18:58:31.636294687Z
custom_metadata    <nil>
deletion_time      n/a
destroyed          false
version            2

=== Data ===
Key    Value
---    -----
bar    bb
foo    aa
```

Permanently delete a specific version of a secret

```
$ vault kv destroy -mount=secret -versions=2 my-secret

Success! Data written to: secret/destroy/my-secret
```

7.3 Vault policies

At this point, you should have one human auth method (OIDC or LDAP) and one machine auth method (AWS or AppRole) enabled. You have also enabled the KV v2 secrets engine with some secrets populated. The next step is to configure policies to manage access to these secrets for authenticated clients.

Policies provide a declarative way to grant or forbid access to certain paths and operations in Vault. When an identity authenticates to Vault it receives a token, and all policies associated with that identity are attached to that token.

Vault static secrets are organized and accessed using paths, and it is the role of ACL policies to describe the permissions on these paths. For example:

1. A secret object is created at `secrets/billing-service`.
2. A policy named `billing-service-read` is created where `read` capabilities are defined against the above path.
3. The identity group `billing-service-dev` is mapped to the `billing-service-read` policy in the OIDC auth method.
4. As a member of the `billing-service-dev` group, Bob authenticates with Vault using OIDC, and the token Bob receives has the `billing-service-read` policy attached.
5. Bob is allowed to read the secret at `secrets/billing-service`.

A thorough exploration of Vault policies can be found in the following documents.

- Policies - concepts
- Getting started guide to policy writing

There are some best practices that you should keep in mind when writing Vault policies.

7.3.1 Principle of least privilege

Err on the side of caution if a consumer is uncertain or lacks clarity regarding their access and policy requirements. If a client realizes the need for access to a secret at a later point, it can be identified and addressed accordingly. In contrast, granting access to an unnecessary secret can be challenging to detect and carries potential risks.

7.3.2 Role-based policies

It is often helpful to create several core policies based on functional roles when onboarding users to Vault, such as the following.

- **Vault cluster administrator**: full access to Vault.

- **Vault operator**: responsible for general Vault operations such as configuring auth methods, secrets engines, and policies.
- **Security team**: consume audit logs, review and approve policies changes.
- **Developer**: read-only access to specific paths required by their applications.
- **Application owners**: write access to specific paths for their applications.

In addition to role-based policies for Vault users, organizations often have numerous policies tailored for application integrations. Each application may have distinct requirements that warrants the creation of individual policies. Moreover, multiple policies may be crafted for various stages in the software development lifecycle (e.g., development, quality assurance, staging, production) for each application. Furthermore, automation workflows, such as CI/CD pipelines or other application build tools, may also require specific policies.

7.3.3 Policy templates

Being as restrictive as possible in your policies can lead to a large number of policies to manage. To simplify this process, we recommend the use of ACL Policy Path Templating.

Policy templating allows for a much smaller set of policies to be used, but still provide the fine grained access controls needed.

7.3.4 Version control

We recommend that you put your policies in a version control system such as GitHub and have a peer-review process for both security and audit purposes. Change management and standardization becomes much easier when policies are centrally managed in a code repository. Vault operators or application teams can submit changes using pull requests and Vault administrators or security teams can approve or deny policy changes.

We recommend using Terraform to codify the configuration of Vault, including policy management. Refer to the Codify Management of Vault Enterprise tutorial which demonstrates policy deployment in multiple namespaces.

7.3.5 Configure policies for KV Secrets

In this section, you will create policies for a hypothetical application to allow the application

- Owners to create and update static secrets.
- Developers to read static secrets for their application.
- To read its required static secrets.

Before you begin, you should have

- Configured either the OIDC or LDAP auth method for human access.
- Configured either the AWS or AppRole auth method for machine access.
- Enabled the KV v2 secrets engine.

Step 1: Write a secret

Login with an admin user and write a secret for the hypothetical application (billing-service).

```
$ vault kv put -mount=secret billing-service foo=bar

== Secret Path ==
secret/data/billing-service

======= Metadata =======
Key                Value
---                -----
created_time       2023-09-21T18:27:29.267845823Z
custom_metadata    <nil>
deletion_time      n/a
destroyed          false
version            1
```

Step 2: Create policy for application owners

```
$ vault policy write billing-service-owner -<<EOF
path "secret/data/billing-service" {
  capabilities = ["create", "read", "update", "delete"]
}
EOF
Success! Uploaded policy: billing-service-owner
```

Step 3: Create policy for the application and its developers

```
$ vault policy write billing-service-dev -<<EOF

path "secret/data/billing-service" {
  capabilities = ["read"]
}
EOF

Success! Uploaded policy: billing-service-dev
```

Step 4: Associate policy for human auth method

For OIDC auth:

Similar to the configuration for Vault admins in setting up the OIDC auth method, you can use the commands below to create an external group called `billing-service-owner` for the application owners and associate it to the `billing-service-owner` policy. This external group is tied to an OIDC group called `billing-service-owner` by the group alias.

```
$ GROUP_ID=$(vault write -format=json identity/group \
  name="billing-service-owner" \
  type="external" \
  policies="billing-service-owner" | jq -r ".data.id")

$ MOUNT_ACCESSOR=$(vault read -field=accessor sys/mounts/auth/oidc)

$ vault write identity/group-alias \
  name="billing-service-owner" \
  mount_accessor=$MOUNT_ACCESSOR \
  canonical_id=$GROUP_ID
```

Create another external group called `billing-service-dev` for the application developers.

```
$ GROUP_ID=$(vault write -format=json identity/group \
  name="billing-service-dev" \
  type="external" \
  policies="billing-service-dev" | jq -r ".data.id")

$ MOUNT_ACCESSOR=$(vault read -field=accessor sys/mounts/auth/oidc)

$ vault write identity/group-alias \
  name="billing-service-dev" \
  mount_accessor=$MOUNT_ACCESSOR \
  canonical_id=$GROUP_ID
```

For LDAP auth:

Map the LDAP group called `billing-service-owner` for the application owners to the `billing-service-owner` policy.

```
$ vault write auth/ldap/groups/billing-service-owner policies=billing-service-owner
```

Map the LDAP group called `billing-service-dev` for the application developers to the `billing-service-dev` policy.

```
$ vault write auth/ldap/groups/billing-service-dev policies=billing-service-dev
```

Step 5: Associate policy for machine auth method

For AWS auth:

Similar to the configuration when you set up a demo role for the AWS auth method, use the command below to create a new role called `billing-service-dev` for the application and associate it to the `billing-service-dev` policy.

```
$ vault write auth/aws/role/billing-service-dev auth_type=iam \
bound_iam_principal_arn=arn:aws:iam::123456789012:role/aws-ec2role-for-vault-
    autthmethod \
policies=billing-service-dev token_ttl=8 max_ttl=8

Success! Data written to: auth/aws/role/billing-service-dev
```

For AppRole auth:

Create a role called `billing-service-dev` for the application and map it to the `billing-service-dev` policy.

```
$ vault write auth/approle/role/billing-service-dev \
secret_id_num_uses=1 \
secret_id_ttl=15m \
token_policies="billing-service-dev" \
token_ttl=1h \
token_max_ttl=1h
```

> **Note**
>
> The application is responsible for renewing the Vault token once it authenticates using the Secret ID. Recall that limiting the SecretID TTL and its number of uses means that a new SecretID will need to be delivered to the application once the application's Vault token reaches its maximum TTL and can no longer be renewed.

Step 6: Validate access

For OIDC auth:

Login with a user belonging to the `billing-service-owner` group. You should see that the token is attached to the `billing-service-owner` identity policy.

```
$ export VAULT_NAMESPACE=<tenant namespace>
$ vault login -method=oidc

Waiting for OIDC authentication to complete...
Success! You are now authenticated. The token information displayed below
is already stored in the token helper. You do NOT need to run "vault login"
again. Future Vault requests will automatically use this token.

Key                    Value
---                    -----
token                  hvs.token
token_accessor         84HTrye0clS28Elc13ZDgiXo.mFAaM
token_duration         1h
token_renewable        true
token_policies         ["default"]
identity_policies      ["billing-service-owner"]
policies               ["default" "billing-service-owner"]
token_meta_email       alice@domain.com
token_meta_role        default
token_meta_username    alice
```

Create a new secret.

```
vault kv put -mount=secret billing-service pizza=pepperoni

== Secret Path ==
secret/data/billing-service

======= Metadata =======
Key                Value
---                -----
created_time       2023-09-21T20:18:44.084663057Z
custom_metadata    <nil>
deletion_time      n/a
destroyed          false
version            2
```

Perform step 1 and 2 to validate access for application developers. You should only be able to read the secret under the path `secret/billing-service`.

For LDAP auth

Login with a user belonging to the `billing-service-owner` group. You should see that the token is attached to the `billing-service-owner` identity policy.

```
vault login -method=ldap username=nwong

Password (will be hidden):
Success! You are now authenticated. The token information displayed below
is already stored in the token helper. You do NOT need to run "vault login"
again. Future Vault requests will automatically use this token.

Key                    Value
---                    -----
token                  hvs.token
token_accessor         EyhczVcVIT8BUuGU1r5GIJ16.JeR4H
token_duration         1h
token_renewable        true
token_policies         ["default" "billing-service-owner"]
identity_policies      []
policies               ["default" "billing-sevice-owner"]
token_meta_username    nwong
```

Create a new secret.

```
vault kv put -mount=secret billing-service pizza=pepperoni

== Secret Path ==
secret/data/billing-service

======= Metadata =======
Key                Value
---                -----
created_time       2023-09-21T20:18:44.084663057Z
custom_metadata    <nil>
deletion_time      n/a
destroyed          false
version            2
```

Perform step 1 and 2 to validate access for application developers. You should only be able to read the secret under the path `secret/billing-service`.

For AWS auth

Login from an EC2 instance attached to an instance profile that is bound to the IAM principal you configured for the billing-service-dev role. You should see that the token is attached to the `billing-service-dev` policy.

```
$ vault login -method=aws role=billing-service-dev

Success! You are now authenticated. The token information displayed below
is already stored in the token helper. You do NOT need to run "vault login"
again. Future Vault requests will automatically use this token.

Key                          Value
---                          -----
token                        hvs.token
token_accessor               cGAOT1f11td0tDCgR4Ff1Xw8.Ojft7
token_duration               8s
token_renewable              true
token_policies               ["default" "billing-service-dev"]
identity_policies            []
policies                     ["default" "billing-service-dev"]
token_meta_client_arn        arn:aws:sts::123456789012:assumed-role/aws-ec2role-for
    -vault-authmethod/i-02ddcf5aed1121703
token_meta_client_user_id    AROATYM2SX6XH5UEKGF7G
token_meta_role_id           53f43687-2f55-c5e9-3570-c51b1cb69763
token_meta_account_id        123456789012
token_meta_auth_type         iam
token_meta_canonical_arn     arn:aws:iam::123456789012:role/aws-ec2role-for-vault-
    authmethod
```

Verify that you can read the secret under the `secret/billing-service`.

```
$ vault kv get -mount=secret billing-service

== Secret Path ==
secret/data/billing-service

======= Metadata =======
Key                Value
---                -----
created_time       2023-09-21T21:38:43.532185535Z
custom_metadata    <nil>
deletion_time      n/a
destroyed          false
version            1

=== Data ===
Key    Value
---    -----
foo    bar
```

For AppRole auth:

Read the RoleID using the command below.

```
$ ROLE_ID=$(vault read -field=role_id auth/approle/role/billing-service-dev/role-
    id)
```

Next, generate a SecretID for the role.

```
$ SECRET_ID=$(vault write -force -field=secret_id auth/approle/role/billing-service
    -dev/secret-id)
```

Login using the `billing-service-dev` role.

```
$ vault write auth/approle/login role_id=$ROLE_ID secret_id=$SECRET_ID

Key                    Value
---                    -----
token                  hvs.token
token_accessor         JhFFIcLlqdjmM6Rq15v0DkGW
token_duration         768h
token_renewable        true
token_policies         ["default" "billing-service-dev"]
identity_policies      []
policies               ["default" "billing-service-dev"]
token_meta_role_name   billing-service-dev
```

Verify that you can read the secret under the `secret/billing-service`.

```
$ vault kv get -mount=secret billing-service

== Secret Path ==
secret/data/billing-service

======= Metadata =======
Key                Value
---                -----
created_time       2023-09-21T21:38:43.532185535Z
custom_metadata    <nil>
deletion_time      n/a
destroyed          false
version            1

=== Data ===
Key    Value
---    -----
foo    bar
```

8 Secrets sync management

> **Note**
>
> This page covers the operator setup and management of Secrets Sync. For how users consume the synchronized-secret workflow, see the User Guide: Secrets Sync.

8.1 Overview

Vault Enterprise helps organizations manage secrets sprawl by centralizing the governance and control of secrets stored within other secret managers. This functionality, called secrets sync, addresses two main challenges:

- First, some applications or services cannot communicate directly with Vault Enterprise. Cloud providers often support their own secrets management solutions. For example, AWS RDS Proxy and AWS Glue can only access database credentials provided by AWS Secrets Manager. This leads to secrets being spread across different cloud environments and CI/CD systems.
- Second, organizations want full control over how their secrets are stored, organized, accessed, and protected, while also benefiting from cloud-native access. Secrets sync allows application owners to use their existing platform knowledge, reducing the effort needed to adopt Vault. By leveraging cloud provider tools, application owners can maintain their current workflows without learning new technologies.

The following sections are intended for the Vault platform team, which uses Vault Enterprise for secrets lifecycle management. However, their organization requires that the secrets be made available through a cloud-provided secrets manager or CI/CD system.

8.2 Secrets sync components

Secrets sync allows organizations to synchronize secrets as needed, continually updating secrets from Vault to external secrets managers so they are always current and accessible by applications or users.



Figure 16: Secrets sync process

The main components of secrets sync are:

- **Sync destinations**: Locations where secrets can be synced to. There are five types of destinations:
  - AWS Secrets Manager
  - Azure Key Vault
  - GCP Secret Manager
  - GitHub organization, repository, or environment secrets
  - Vercel Project
- **Secret associations**: A single Vault secret can be linked to multiple destinations. For example, a Jira token stored as a Vault secret used in CI workflows can be synced to multiple GitHub repositories, each represented by a distinct Vault sync destination. Secret associations link an existing Vault K/V secret to a sync destination. Each secret has its own association, and many associations can share a destination.
- **Name templates:** You can define the name format of synced secrets for each destination. The default name template is `vault/<accessor>/<secret-path>`. Name templates help prevent accidental name collisions at the destination and clearly identify the origin of the secrets.

8.3 Secret management challenges

Organizations face challenges in deploying and managing secrets, but these challenges also present opportunities to close security gaps and improve efficiency.

8.3.1 People

Managing secrets involves handling sensitive material. Vault's user interface, API, and CLI allow users to generate, update, delete, rotate, and revoke secrets efficiently and securely.

Accessing sensitive information in new ways or on new machines can lead to accidental data leaks. A centralized secrets management platform like Vault minimizes the risk of breaches caused by human error during manual transfers. Instead of switching between multiple cloud solutions and machines, all secrets are managed, rolled out, and monitored by Vault.

Different teams manage their secret lifecycles differently, writing policies and defining rotation cadences. Syncing secrets from a single management plane increases productivity by eliminating the need to interact with multiple key managers across different clouds.

8.3.2 Processes

Many organizations still use manual processes to rotate secrets, which often require maintenance windows even for minor changes. Secrets that are not regularly rotated provide more time for bad actors to gain access to sensitive data. While Vault secrets sync cannot directly rotate secrets, it can help to automate the process by synchronizing changes done in Vault to external secrets managers. This automation enhances the security posture of cloud-hosted applications by ensuring secrets are always up-to-date.

Secrets consumption sprawl occurs when a single credential is used in many different places. Unrotated secrets and broad internal distribution increase the attack surface. Use Vault to granular manage secret access via paths and keys, segregating them for individual applications and teams.

Managing secrets at scale is tedious, involving endpoint management, consumption, and auditing processes. Centralizing secrets with Vault allows operators to audit in one place, tracking when, by whom, and where secrets are modified or accessed. This centralized approach enhances productivity for governance teams, implements security best practices, and improves compliance monitoring.

8.4 Example scenario

Consider this situation: You need to provide credentials, such as database credentials or API keys, to multiple applications provisioned across various cloud providers. These applications already consume secrets from the secrets manager within their respective cloud environments, or the credentials are hard-coded into the application code. Each application development team needs to manage their own secrets while also having them stored centrally for organizational auditing. Each development team is assigned a tenant namespace in your organization's Vault Enterprise deployment.

The recommended organizational layout would be as follows:

- Root administrators
    - Enable the secrets sync feature.
    - Set up tenant namespaces.
- Application A
    - Resides in AWS and GCP.

        - Managed by application development team A.
    - Application B
        - Resides in Azure.
        - Managed by application development team B.
    - Application development team A
        - Responsible for the development and integration of application A.
        - Namespace users: Can read and update K/V secrets and associate these secrets with a sync destination.
        - Namespace admins: Have elevated admin rights and can set up and configure sync destinations.
    - Application development team B
        - Responsible for the development and integration of application B.
        - Namespace users: Can read and update K/V secrets and associate these secrets with a sync destination.
        - Namespace admins: Have elevated admin rights and can set up and configure sync destinations.

This structure allows each development team to manage their secrets independently, while ensuring centralized auditing and control through Vault Enterprise.

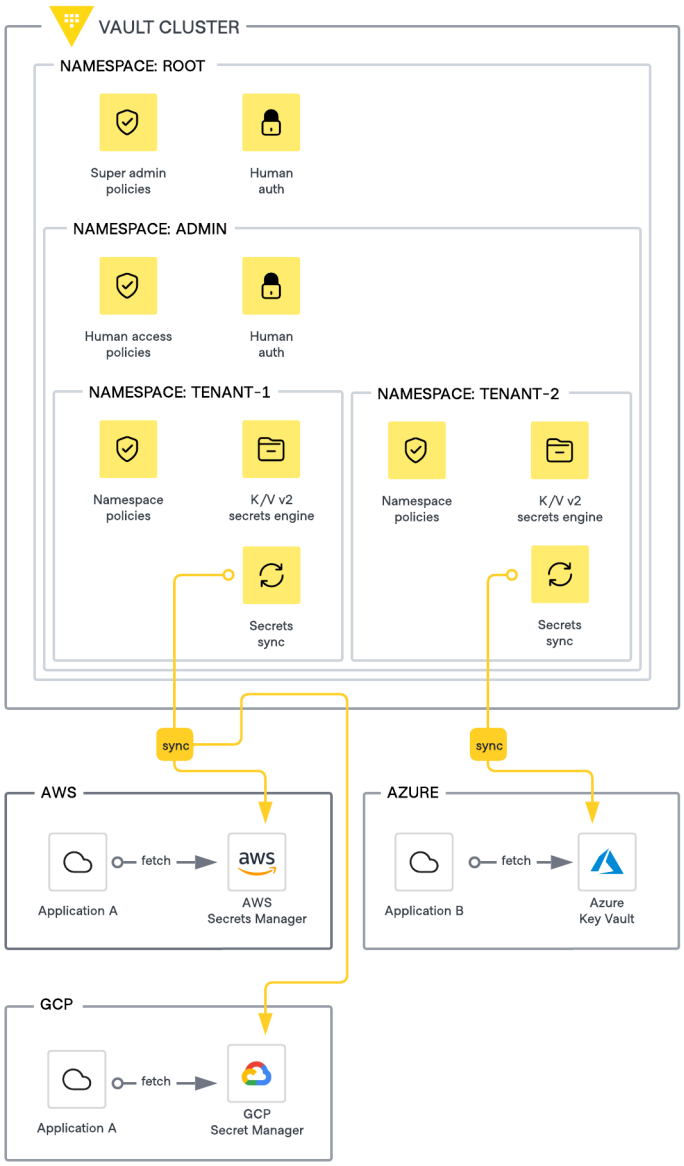


Figure 17: Secrets sync namespace design

8.5 Design considerations

8.5.1 Infrastructure architecture

The key architectural aspects to understand are below.

- **Network path requirements:** Vault requires network access to the secrets sync destinations, and this connectivity must be in place before using the secrets sync feature. The same requirement applies to the disaster recovery (DR) cluster, which also needs outbound network access to the sync destinations. While the DR cluster may not use this connection during normal operations, having it ensures a successful transition in the event of a DR failover.
- **Active node processing**: Sync operations are only processed on the active node of a primary cluster. Write operations, such as writing an association or a K/V v2 secret, can only occur on the active node. This also means that secrets sync operations will not benefit from scaling the Vault cluster with either performance replication or adding performance standby nodes.

8.5.2 Mount and namespace structure

You can configure multiple secrets sync destinations per Vault namespace, allowing you to manage secrets effectively within your organization. Here's how you can structure your mounts and namespaces:

- **Multiple secrets sync destinations:** You can configure several secrets sync destinations within a single namespace. This flexibility enables you to associate secrets from different K/V mounts, as long as they reside within the same namespace as the destination.
- **Name templates:** Each sync destination has its own name template. If you need different templates for secrets, you can define the same destination multiple times with different names. This allows for customized naming conventions for various secrets.
- **Per application configuration:** Configuring a secret sync destination for each application enables isolated management of secrets for different applications. This approach helps in managing policy requirements, templating, and rotation separately for each application. It ensures that the secrets of one application do not interfere with those of another.
- **One destination per application:** It is generally recommended to create one destination per application with multiple secret associations. This practice simplifies the access manage-

ment via Vault policies, allows effective templating, adheres to rotation requirements for every application, while it can also help with secret segregation at the destination.

- **Disaster recovery (DR) considerations:** The destinations might need to be DR-aware and have secret-specific configurations. For example, a primary application in AWS us-west-2 region and a DR application in eu-west-1 might need different synced credentials, even if the same application is deployed.

8.5.3 Policy management

The key considerations and best practices for managing policies related to secrets sync are as follows.

- Administrator roles
  - **Configuration:** Only namespace administrators should configure sync destinations to avoid exposing high-privileged cloud service account credentials.
  - **Write capabilities:** Namespace administrators, who interact with the sync engine, need high-privileges, such as access to write capabilities for paths under `/sys/sync`.
- User roles
  - **Read access:** Users need read access to the Vault secret itself before configuring a sync association. This ensures the Secret Sync feature cannot be used for privilege escalations.
  - **Write access:** Typically, namespace users have access to the K/V secrets mount to update the value of secrets, which may or may not be synced elsewhere.
- Security considerations
  - **Access control:** Vault does not control permissions at the destination. It is the platform team's responsibility to configure and maintain proper access controls on external systems to prevent unintentional access to synced secrets.

Policy configuration examples

Here are some example policies for managing secret sync destinations and associations, within a Vault child namespace, e.g. "tenant-1", while the secrets sync administrator and user will authenticate to the parent namespace "admin".

Secrets sync administrator policy

```
# Allow full access to the secrets sync feature.
path "tenant-1/sys/sync/*" {
  capabilities = ["read","create","update","delete","list"]
}
# List existing secret engines.
path "tenant-1/sys/mounts" {
  capabilities = ["read", "list"]
}
# Manage secret engines.
path "tenant-1/sys/mounts/*" {
  capabilities = ["create", "read", "update", "delete", "list"]
}
# Allow full access to the secret mount called ‘’secret.
path "tenant-1/secret/*" {
  capabilities = ["read","create","update","delete","list","patch"]
}
```

Secrets sync user policy

```
# List existing destinations.
path "tenant-1/sys/sync/destinations*" {
  capabilities = ["read", "list"]
}
# List existing associations.
path "tenant-1/sys/sync/associations*" {
  capabilities = ["read", "list"]
}
# Create an association for destination type 'aws-’sm
# and the destination name 'my-account'
path "tenant-1/sys/sync/destinations/aws-sm/my-account/associations/set" {
  capabilities = ["create", "update"]
}
# Remove an association for destination type 'aws-’sm
# and the destination name 'my-account'
path "tenant-1/sys/sync/destinations/aws-sm/my-account/associations/remove" {
  capabilities = ["create", "update"]
}
# Allow full access to the secret mount called ‘’secret.
path "tenant-1/secret/*" {
  capabilities = ["read", "create", "update", "delete", "list", "patch"]
}
```

Workflow examples

Admin workflow: Admins configure a sync destination.

```
export VAULT_NAMESPACE=admin/tenant-1
vault write sys/sync/destinations/aws-sm/my-account \
  access_key_id="$ACCESS_KEY_ID" \
  secret_access_key="$SECRET_ACCESS_KEY" \
  region="eu-north-1" \
  secret_name_template="vault/{{ .NamespacePath }}{{ .MountPath }}/{{ .SecretPath }}"
```

User workflow: User configures an association between an existing K/V secret and the destination.

```
export VAULT_NAMESPACE=admin/tenant-1
# User creates a K/V secret.
vault kv put secret/database/dev \
  api_key="abc" \
  key_id="123"
# User associates the secret with a destination.
vault write sys/sync/destinations/aws-sm/my-account/associations/set \
   mount="secret" \
   secret_name="database/dev"
```

If the user wishes to unsync a previously synced secret, they can delete the association:

```
export VAULT_NAMESPACE=admin/tenant-1
vault write sys/sync/destinations/aws-sm/my-account/associations/remove \
   mount="secret" \
   secret_name="database/dev"
```

8.5.4 Lifecycle management

- **Update and removal**: If a secret is updated or removed in Vault, the same changes will be reflected in the external systems. Applications should be rotation-aware to handle these updates seamlessly.
- **Last-write-wins**: Vault operates with a last-write-wins strategy. If a secret with the same name exists at the destination, Vault will overwrite it during synchronization.
- **Use templates and tags**: Employ name templates and tags to manage secret access at the destination effectively and to prevent accidental overwrites or exposure.

8.5.5 Automation

Scaling the secrets sync feature via automation can be effectively achieved using Terraform.

Resource types

To configure the secrets sync feature via Terraform, you can to utilize several resource types, each corresponding to different destination types:

- `vault_secrets_sync_association`
- `vault_secrets_sync_aws_destination`
- `vault_secrets_sync_azure_destination`
- `vault_secrets_sync_config`
- `vault_secrets_sync_gcp_destination`
- `vault_secrets_sync_gh_destination`
- `vault_secrets_sync_github_apps`
- `vault_secrets_sync_vercel_destination`

Example configuration

Below is an example Terraform configuration to set up the secrets sync feature, including the resources:

- **Mount resource:** Creates a K/V v2 mount at the path "secret".
- **K/V secret resource:** Defines a key-value secret named "database/dev". The resource also includes a lifecycle meta-argument that ignores changes that are done outside of Terraform, that means you can update this secret via Vault API, CLI or UI.
- **AWS destination resource:** Configures an AWS secrets sync destination with the name "my-account".
- **Sync association resource:** Associates the K/V secret with the AWS sync destination.

```
resource "vault_mount" "tenant_mount" {
  path      = "secret"
  type      = "kv"
  options = {
    version = "2"
  }
}

resource "vault_kv_secret_v2" "tenant_secret" {
  mount     = vault_mount.tenant_mount.path
  name      = "database/dev"
  data_json = jsonencode(
    {}
  )

  lifecycle {
    ignore_changes = [
      data_json
    ]
  }
}

resource "vault_secrets_sync_aws_destination" "aws" {
  name                = "my-account"
  access_key_id       = var.access_key_id
  secret_access_key   = var.secret_access_key
  region              = "eu-north-1"
  secret_name_template = "vault/{{ .NamespacePath }}{{ .MountPath }}/{{ .SecretPath
     }}"
  custom_tags = {
    "ochestrator" = "terraform"
  }
}

resource "vault_secrets_sync_association" "aws-test" {
  name        = vault_secrets_sync_aws_destination.aws.name
  type        = vault_secrets_sync_aws_destination.aws.type
  mount       = vault_mount.tenant_mount.path
  secret_name = vault_kv_secret_v2.tenant_secret.name
}
```

8.6 Best practices for sync destinations

Select your sync destination below for further guidance.

AWS

We recommend the following best practices for AWS Secrets Manager:* **Define AWS IAM resource-based policies**: Ensure synced secrets are read-only in AWS Secrets Manager and are only readable by applications and services that need access. This prevents developers from reading or accidentally overwriting the secret values. - Use AWS resource tags to allow or restrict access to secrets. For example, attach an AWS IAM policy that prevents Vault from modifying secrets not created by a sync operation. - Add a negative condition on all write-access policies not used by Vault to prevent out-of-band overwrites. Detailed examples are documented here.

- **Use name templates:** Define a hierarchical naming convention for secrets synced using secrets sync's name templates. This helps you establish fine-grained access controls to secrets based on their names in AWS Secrets Manager.
- **Use resource tags:** To categorize secrets synced to AWS Secrets Manager, tag them e.g. by purpose, team, environment, etc. Define this tagging when configuring the AWS sync destination. For more details, see AWS Tagging Best Practices.
- **Use IAM assume role**: Use IAM assume role instead of tying the secrets management policies directly to an IAM user for Vault. This setup enhances security and flexibility. Instructions for setting up assume role are available here. Assume role also allows granting access to AWS Secrets Manager across AWS accounts. For more details, see AWS Cross-account setup of Vault Secret sync using Roles.Be aware of these limitations:* **AWS Managed Streaming for Kafka (MSK)**: MSK can be integrated with AWS Secrets Manager when using SASL/SCRAM for secrets sync. However, MSK requires a KMS encryption key different from the default KMS encryption key for storing secrets in AWS Secrets Manager. Vault secrets sync currently only supports the default KMS encryption key, so this integration will not work. Refer to the MSK password tutorial and update encryption keys for more information.
- **AWS Relational Database Service (RDS) Proxy**: RDS Proxy allows applications to pool and share database connections, simplifying connection management and improving resilience to database failures. While RDS Proxy can be integrated with AWS Secrets Manager, the current iteration of secrets sync does not support dynamic secrets (database secrets engine). Therefore, you would need to create database credentials manually in RDS and store them in Vault, which is not a best practice because AWS Secrets Manager has native support for rotating RDS credentials. Refer to the RDS Proxy user guide for more information.

GCP

We recommend the following best practices for GCP Secret Manager:* **Grant minimal permissions:** Assign only the necessary permissions to users and applications. Use IAM roles like roles/secretmanager.viewer and roles/secretmanager.secretAccessor appropriately to

9 Dynamic secrets management

> **Note**
>
> This page covers the operator setup and management of dynamic secrets. For application consumption considerations, see the User Guide: Dynamic secrets.

9.1 Dynamic secrets overview

The process of generating and consuming dynamic secrets involves the following steps:

- **Configure Vault:** Vault needs to be configured with the appropriate secrets engine (e.g., AWS, Azure, database, etc.) and the necessary authentication and authorization mechanisms.
- **Request dynamic credentials:** An application or service requests dynamic credentials from Vault, specifying the desired service and any additional parameters (e.g., database name, user permissions).
- **Generate and lease credentials:** Vault generates temporary credentials for the requested service and associates them with a lease. The lease determines the lifetime of the credentials.
- **Use credentials:** The application or service can use the dynamic credentials to authenticate and access the requested service or system for as long as the lease is active
- **Renew or revoke:** Before the lease expires, the application can renew the credentials to extend their lifetime. Alternatively, the credentials can be revoked manually or will be automatically revoked when the lease expires.

HashiCorp Vault includes a large ecosystem of plugins to support identity and access integrations across a wide variety of external services:

- Cloud credentials - AWS, Azure, GCP
- Database credentials - Postgres, MySQL, Oracle, MongoDB, RabbitMQ etc
- Certificates - PKI
- API credentials - Kubernetes, Nomad, Consul, Terraform Cloud, etc
- Identity management** **- SSH, LDAP, Kerberos, Okta, GitHub, etc
- Ecosystem credentials - Artifactory, Venafi, 3rd party plugins

A full list of Vault integrations is available in the Vault integrations directory.

9.2 Architecture of dynamic secrets

We recommend prioritizing dynamic secrets adoption in a phased approach::

- **Cloud credentials in HCP Terraform**: If you use HCP Terraform to provision infrastructure, leverage Vault's dynamic provider credentials.
- **Cloud credentials in jobs, CI/CD pipelines:** If you use cloud credentials embedded in jobs or CI/CD pipelines, transition to using dynamic cloud credentials.
- **DB credentials in jobs, CI/CD pipelines:** If you use static DB credentials in jobs or CI/CD pipelines, we recommend converting them to dynamic DB credentials.
- **DB credentials in applications: **Convert applications accessing sensitive databases to leverage dynamic credentials. Leverage the Vault Agent for an application that is not Vault-aware.
- **PKI**: Drive certificate signing and renewals.
- Third-party integrations: Where needed

9.2.1 Recommendation on TTL

Dangling leases should be avoided, due to memory consumption involved in lease management and cleanup. Tune your TTLs. Start with a generous TTL value (24-48 hrs) and fine-tune based on the application requirements and lifecycle.

Ensure batch and pipeline jobs' TTL threshold exceeds the maximum potential of long-running jobs.

Short-lived applications should have short-lived TTLs. Long-lived leases for short-lived applications can have negative memory consequences for Vault.

9.2.2 Network considerations

For dynamic secrets to be implemented, Vault needs a network path to the resource being managed.

In the example below, the Vault cluster is in a shared cloud account managed by the platform team. In this scenario each business unit leveraging Dynamic Credentials for Databases operates a separate account with databases in their own isolated DB subnets. For adoption of Vault Dynamic DB Credentials, the following networking topology where vault has access to each of the db subnets where will be required.

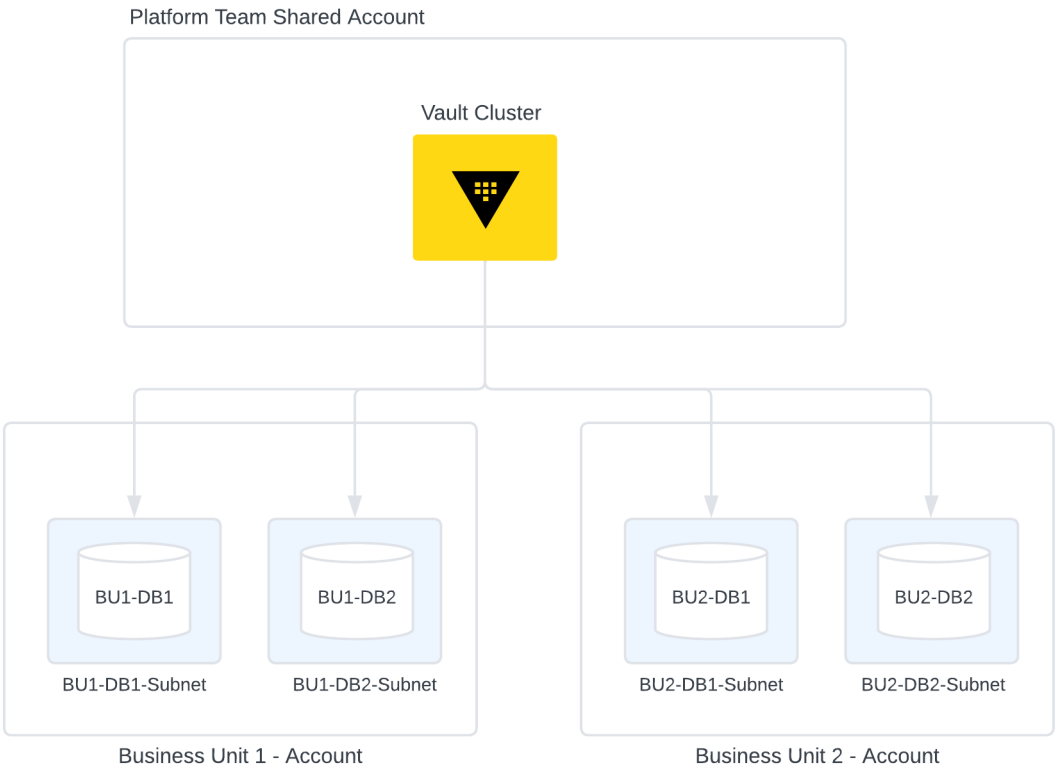


Figure 18: Dynamic Secret Network Considerations

When a Vault DR/PR cluster is available, ensure that the secondary clusters have access to the tenant databases as well. This will ensure that in case of a failover, the DR/PR cluster is able to manage dynamic secrets database accounts.

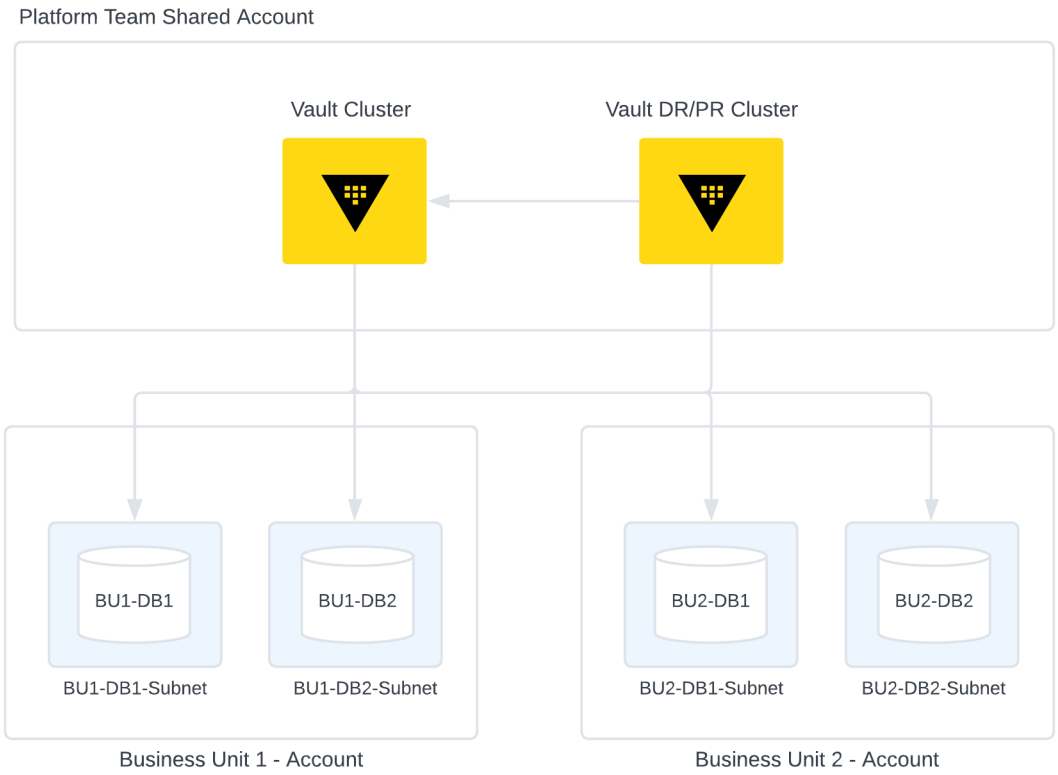


Figure 19: Dynamic Secret Network Considerations DR

In case of region failure and the DR Vault cluster and the DR DB is promoted - please note that due to the way the DB mount is configured, the Vault DR cluster may not have a way to access the DR DB. We recommend that a separate DB mount is created for DR purposes. This also assumes that the application is configured to access the DR DB mount and has the ability to request new leases.

The connection parameter in the database secrets engine config is the key to reaching the database from Vault. By using the same client-facing FQDN or IP address for all failover replicas of your database, you can avoid configuring multiple mounts for database failover. Configure your database replication with Virtual IP addresses (VIPs), global load balancer, or DNS with failover in order to ensure access across multiple regions of Vault clusters.

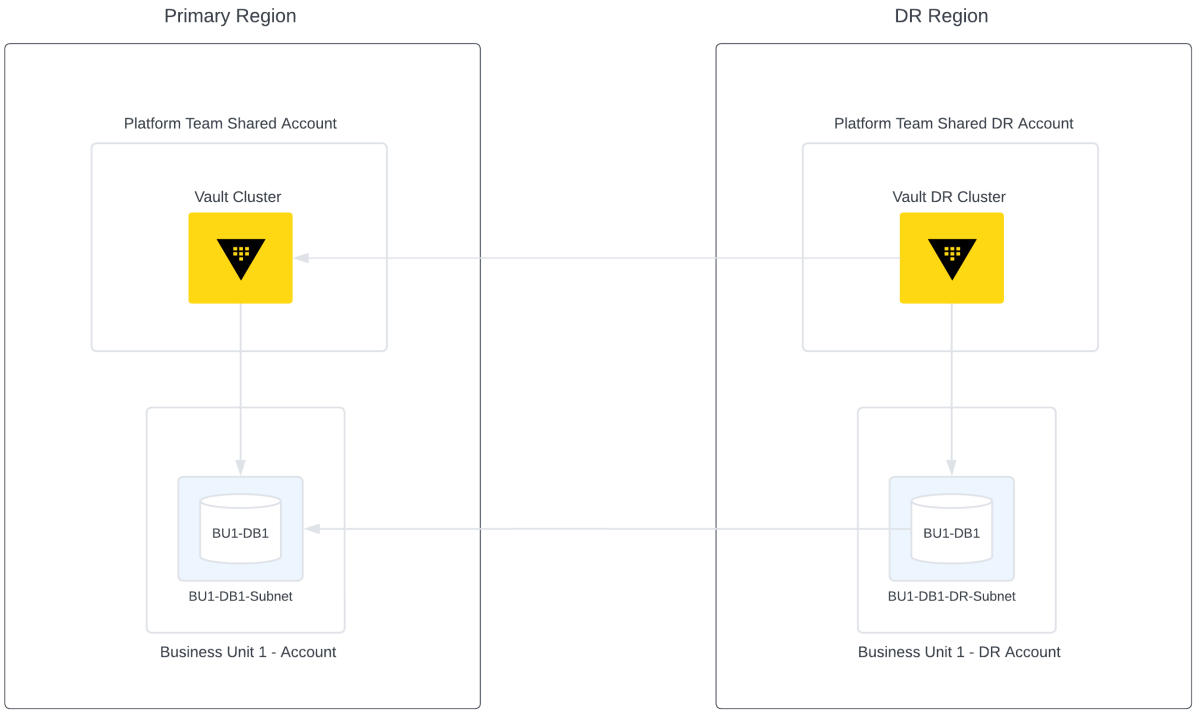


Figure 20: Dynamic Secret Network Considerations DR

9.2.3 Dynamic secrets engine path

The location or mount point at which a dynamic secrets engine is activated determines specific capabilities and restrictions that impact the organization of your dynamic credentials. This includes root credential management, connection parameters, and credential roles for Vault to manage.

"Root" user credential and rotation

The "root" user in this context is a superuser service account created in an external service exclusively for Vault to manage roles. This account is then configured as the "root" user in one dynamic secrets engine in Vault.

For an AWS secrets engine, this would be an IAM user and its access key. For a PostgreSQL database secrets engine, this would be a database superuser and password. The endpoint `rotate`

`-root` is used to manually rotate the credential of this root user. After Vault rotates the credential, there is no way to retrieve it.

There must be a 1:1 relationship between "root" users and dynamic secrets configurations in Vault. Once added to Vault, this "root" user must not be used or managed outside of Vault, and must not be used for more than one dynamic secrets engine within Vault. For the same reason, do not use the same service account for a "root" user in a secrets engine and the "root" user in an auth method.

The "root" user should have only the permissions it needs to create service accounts with their further-reduced scope. These "root" user permissions are managed in the external service. For the AWS secrets engine, the access would be defined by an AWS policy and role applied to the IAM "root" user. For the PostgreSQL database secrets engine the access would be defined by the SQL statements used to create a PostgreSQL role in the PostgreSQL database server.

Define the appropriate RBAC and account creation statements for your root and dynamic users with the help of your identity and access team and a cloud platform SME or database SME in your organization.

Dynamic secrets engine roles

Role configurations are nested under a dynamic secrets engine mount path. The parameters for a role depend on the type of dynamic credential. There are two types of roles within most dynamic secrets engines. **Dynamic** roles are used to manage the full lifecycle of credentials in external systems. **Static roles** are used to manage and rotate only the secret portion of a persistent credential.

Dynamic roles are created and destroyed by Vault. A dynamic secret role configuration defines the parameters for the role, how credentials under that role are created, and the length of time a credential will be valid.

A static role configuration specifies an external credential under management, including the timing and method of rotation of its secret. Static credentials are created in the external system first and then added to Vault using an initial credential which is rotated under management. \

All built-in database secrets engine plugins support both dynamic and static roles. LDAP, AWS, and GCP secrets plugins also provide static roles. The Azure secrets engine has the option to use existing service principals, although that plugin calls them `application_object_id`.

For more information about database dynamic roles, see section on "Database dynamic roles" in this document.

9.2.4 Database secrets engine

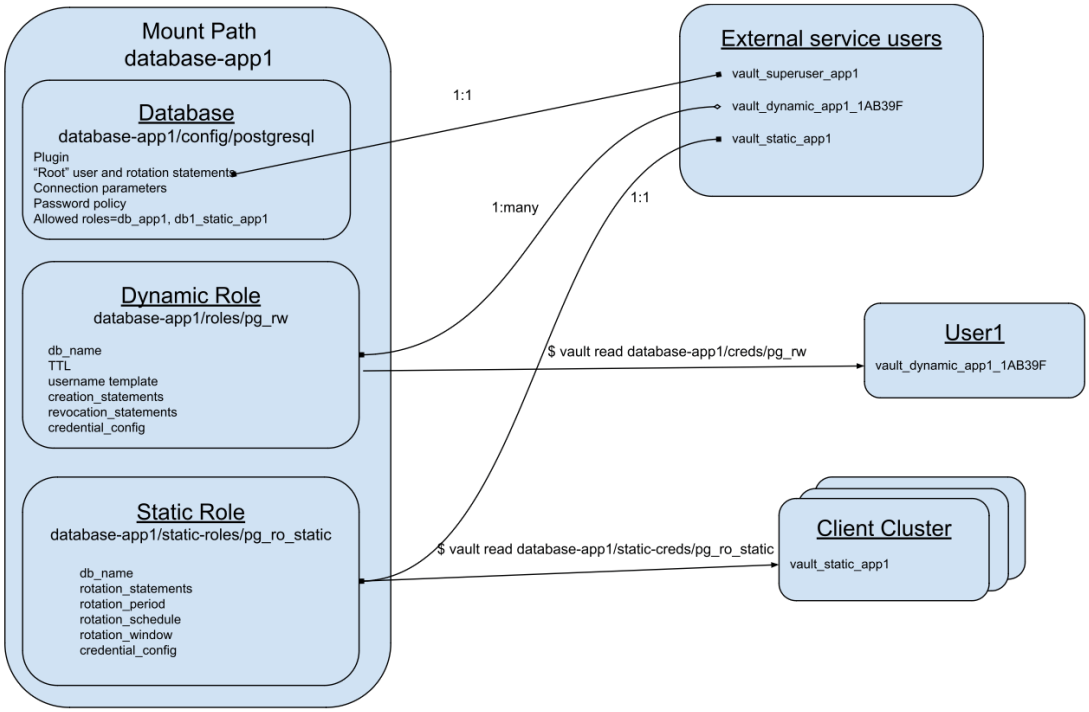


Figure 21: Database Secrets Engine

9.2.5 Config

The `config/:name` endpoint of a database secrets engine includes settings to configure each database connection and type. These parameters are defined by the database plugin being used, so they may differ between various databases. The `name` used here must be re-used by the roles that leverage this connection.

For more information, see the database secrets engine documentation and the configure connection API reference.

Example configuration

```
$ export POSTGRES_URL=127.0.0.1:5432

$ vault write database-app1/config/postgresql \
plugin_name=postgresql-database-plugin \
allowed_roles=pg-ro,pg-rw,pg-static \
connection_url="postgresql://{{username}}:{{password}}@$POSTGRES_URL/demoapp?sslmode=
    disable" \
username=db_admin_vault \
password=insecure_password

Success! Data written to: database-app1/config/postgresql
```

Root credential rotation

To secure Vault's "root" credential after it is configured, Vault provides an endpoint to rotate the credential. This will replace the manually-entered credential with a credential that only this Vault configuration knows. For this reason, do not use the same "root" credential in more than one location, even within the same Vault cluster. Immediate root credential rotation reduces risk of someone else accessing and using that credential.

```
$ vault write -force database-app1/rotate-root/postgresql
```

Considerations for database secrets engine organization

You can configure multiple database plugins per mount point, by writing different configs for different database plugins, with distinct connection parameters. If you have multiple database types configured under one mount path, use the `allowed_roles` parameter to restrict which plugin is used by each role.

Configuring multiple database types and connections under one mount point has some pros and cons. Doing so can consolidate configuration of Vault policies for an app that happens to use multiple database connections. However, providing multiple database connections under one mount point could lead to inadvertent mismanagement, such as revoking credentials across multiple databases accidentally, or requesting the wrong credential altogether. It also could add forensic complexity during an audit.

Configuring a secrets engine path per application enables management of one application's secrets separately from other applications' secrets, isolating policy management, root credentials, and credential lifecycle management.

For these reasons, it is generally a best practice to create one secrets engine mount point per database, per application.

9.2.6 Database Roles

Roles define the **credential generation** of the database secrets engine. You can have multiple roles per database configuration. For example, you may want both read-only and read-write roles for a PostgreSQL server. The same Vault "root" user will require appropriate capabilities in the database to manage all roles that use the same database configuration.

Database roles have **creation_statements** which define what SQL statements will be executed by Vault when it connects to the database to create a role.

Username **customization** is a feature to improve visibility within the database. For example, you can prepend roles with 'vault_managed' to make it more clear to database engineers which roles are managed exclusively via Vault. This can help prevent accidental deletion of Vault's managed accounts.

Configurable **password policies** enable you to meet your organization's complexity requirements for compliance. Password policies can be defined at the database configuration level, and the role level, enabling the ability to inherit or override the password policy for each database role.

Create a dynamic database role

First, you need to define the creation statements for your role. The creation statements should be determined in conjunction with your database administration, security, and IAM teams as appropriate for your organization. The database team may provide the statements to you in a file. This example writes statements to a local file, defining broad read-only access:

```
$ tee readonly.sql <<EOF
CREATE ROLE "{{name}}" WITH LOGIN PASSWORD '{{password}}' VALID UNTIL '{{expiration}}
    ';
COMMENT on ROLE "{{name}}" IS 'Role managed by Vault';
GRANT CONNECT ON DATABASE demoapp TO "{{name}}";
GRANT SELECT ON ALL TABLES IN SCHEMA public TO "{{name}}";
EOF
```

The next command creates the role in Vault, using the statements from a file. The `db_name` value *must match* the name of the database secrets engine object created earlier (not the database

hostname). In this example, it is `postgresql`.

```
$ vault write database-app1/roles/pg-ro \
db_name=postgresql \
creation_statements=@readonly.sql \
default_ttl=1h max_ttl=24h

Success! Data written to: database-app1/roles/pg-ro
```

View the dynamic database role

You can read the same role endpoint to view the role configuration and SQL statements.

```
$ vault read database-app1/roles/pg-ro

Key                     Value
---                     -----
creation_statements     [CREATE ROLE "{{name}}" WITH LOGIN PASSWORD '{{password}}'
    VALID UNTIL '{{expiration}}';
COMMENT on ROLE "{{name}}" IS 'Role managed by Vault';
GRANT CONNECT ON DATABASE demoapp TO "{{name}}";
GRANT SELECT ON ALL TABLES IN SCHEMA public TO "{{name}}";]
credential_type         password
db_name                 postgresql
default_ttl             1h
max_ttl                 24h
renew_statements        []
revocation_statements   []
rollback_statements     []
```

Generate a dynamic database credential

To create a new dynamic credential, you need to perform a read against the credential endpoint for the role.

```
$ vault read database-app1/creds/pg-ro

Key             Value
---             -----
lease_id        database-app1/creds/pg-ro/6XNimuvRkRgQqMoKHUdXigII
lease_duration  1h
lease_renewable true
password        -upnMCdrAyBdl0MwRjcS
username        v-root-pg-ro-55zd1HKtF4qzsqQiEklO-1712362373
```

List leases and investigate a lease generated by a database role

Note that the lease_id from the above command matches the output in this command. In this example, there is only one active lease:

```
$ vault list sys/leases/lookup/database-app1/creds/pg-ro

Keys
----
6XNimuvRkRgQqMoKHUdXigII

$ vault lease lookup database-app1/creds/pg-ro/6XNimuvRkRgQqMoKHUdXigII

Key             Value
---             -----
expire_time     2024-04-05T20:12:53.168521-05:00
id              database-app1/creds/pg-ro/6XNimuvRkRgQqMoKHUdXigII
issue_time      2024-04-05T19:12:53.168518-05:00
last_renewal    <nil>
renewable       true
ttl             50m47s
```

Revoke leases

The following command will revoke one lease, deleting its related database credential:

```
$ vault lease revoke database-app1/creds/pg-ro/6XNimuvRkRgQqMoKHUdXigII
```

By comparison, this command will revoke **all leases under the `pg-ro` role** (and delete all related database credentials):

```
$ vault lease revoke -prefix database-app1/creds/pg-ro
```

Even more destructively, the following command deletes **all credentials for all dynamic roles** under the database-app1 mount path:

```
$ vault lease revoke -prefix database-app1/creds/
```

9.2.7 Static Roles

Static roles are used to rotate the secret password component of a database login credential, while maintaining the same username portion. Using static roles enables Vault operators to rotate pass-

words without needing to create and delete users from the database. This can be important for performance at scale, since creating and deleting users can cause table locks impacting database performance. Static roles can be rotated based on a cron-like schedule, and limited to a window of time.

Configure a database static role

The static database role must already exist in the remote database. Vault will immediately rotate the password when the role is configured.

The Vault "root" user must have RBAC that allows it to ALTER the static role to change the password. Then, set up a file with rotation statements:

```
$ tee rotation.sql <<EOF
ALTER USER "{{name}}" WITH PASSWORD '{{password}}';
EOF
```

Then the static role can be connected to Vault:

```
$ vault write database-app1/static-roles/pg-static \
db_name=postgresql \
rotation_statements=@rotation.sql \
username="vault_static_app1" \
rotation_period=86400
```

Read the static role's password, which will stay the same until it is rotated:

```
$ vault read database-app1/static-creds/pg-static

Key                 Value
---                 -----
last_vault_rotation 2024-04-05T20:07:45.462742-05:00
password            B8uk8rmn6cFiIvOZ-trN
rotation_period     24h
ttl                 23h59m47s
username            vault_static_app1
```

9.3 Other database types

9.3.1 AWS DynamoDB - AWS Secrets Engine

While DynamoDB is not supported by the database secrets engine, you can use the AWS secrets engine to provision dynamic credentials capable of accessing DynamoDB.

9.3.2 IBM DB2

Access to Db2 is managed by facilities that reside outside the Db2 database system. By default, user authentication is completed by a security facility that relies on operating system based authentication of users and passwords. This means that the lifecycle of user identities in Db2 aren't capable of being managed using SQL statements, which creates an operational challenge for credential lifecycle management.

To provide flexibility in accommodating authentication needs, Db2 ships with authentication plugin modules for LDAP.

For more information, see the IBM Db2 and OpenLDAP tutorial.

10 PKI certificate management

> Note
>
> This page covers the operator setup and management of Vault PKI. For certificate consumption workflows, see the User Guide: PKI certificates.

10.1 Overview

The public key infrastructure (PKI) secrets engine dynamically issues X.509 certificates, allowing clients to obtain certificates without manually generating key pairs, submitting certificate signing requests (CSRs), or waiting for approval workflows, which enhances agility. Vault's authentication and authorization mechanisms act as an identity-driven PKI registration authority, enforcing role-based access control (RBAC) and policy-based request validation to ensure tightly controlled certificate issuance.

Each application instance receives a cryptographically unique certificate (and optionally, private key), eliminating shared credentials and simplifying both certificate and key lifecycle management. By adopting short-lived certificates, organizations not only bolster their security posture but can also minimize the need for revocation, reducing, or eliminating reliance on certificate revocation lists (CRLs) and improving scalability.

While the Vault PKI secrets engine provides the foundation for secure, automated certificate issuance (the producer), a complete solution also requires a client-side mechanism (the consumer) to handle retrieval, renewal, and integration with applications. A Vault Agent solution, a client library, or a custom service via a spectrum of supported protocols can fulfill this role. Together, the producer and consumer enable workloads to automatically obtain, renew, and apply certificates with minimal effort.

10.2 A pattern for automatic certificate renewal

This page includes guidance on the configuration of HashiCorp Vault's PKI secrets engine, CA hierarchy design, and access control best practices. We focus on using Vault Agent for seamless certificate retrieval and renewal, enabling secure and efficient management of ephemeral X.509 certificates. See the related resources section at the end of the page for other consumption workflows.

The primary benefits of this approach include:

- **Enhanced compliance**: Enforce strong, consistent, identity-based security controls and streamline certificate issuance to meet regulatory and organizational policies.
- **Downtime prevention**: Automate certificate renewal and integrate post-rotation actions to maintain secure, uninterrupted service operations.
- **Cost optimization**: Minimize manual effort and administrative overhead by automating routine certificate lifecycle management tasks.
- **Reduced risk**: Mitigate the possibility of certificate or key compromise by enforcing short lifetimes and automating rotation, limiting the impact of leaked secrets.

10.3 Producers and consumers of certificates

We recommend a producer/consumer model for Vault PKI, applying the following roles to improve the security posture of certificate management.

- **Platform operator**: Maintains and scales the Vault platform from an infrastructure perspective. This role may reside within security-focused teams or broader infrastructure platform groups.
- **Secrets producer**: Configures and manages PKI secrets and operational patterns in Vault. Responsibilities include configuration of certificate authorities, setting up certificate issuance policies, defining access controls, and ensuring compliance with applicable regulations or controls. This role may include Vault administrators, PKI specialists, or security engineers.
- **Secrets consumer**: Integrates Vault-issued certificates with various applications. This category includes DevOps engineers, application developers, and service owners who rely on certificates for secure communications.

10.4 People and process considerations

Implementing automatic certificate renewal with a producer and consumer model requires coordination across multiple roles and a shared understanding of responsibilities. Successful adoption depends not only on technical integration but also on effective collaboration, change management, and knowledge sharing.

10.4.1 Cross-team collaboration

- Establish shared workflows between platform and application teams and define ownership boundaries clearly, for example, who is responsible for debugging failed certificate renewals or policy misconfigurations.
- Regularly review access controls, PKI roles, and machine identity configurations as part of security posture assessments.

10.4.2 Change management

- Integrate automated testing, such as through CI/CD pipelines, to validate consumption behaviors and ensure reliable certificate issuance.
- Use a version-controlled repository to manage PKI configurations and policies. Where possible, define these through Terraform to support auditability, repeatability, and safe change rollouts.

10.4.3 Enablement and documentation

- Provide onboarding guides and training for application teams integrating with Vault for the first time.
- Maintain a centralized knowledge base that includes instructions for debugging common issues, configuring PKI roles, and validating certificates.

10.4.4 Monitoring and alerting

- Monitor certificate issuance frequency and details via Vault's audit logs.
- Set up alerts for failures in the renewal process, and on certificate expiration especially those used in critical systems.
- Use dashboards to visualize certificate usage trends, identify outliers, and inform capacity planning.

10.5 Validated architecture

10.5.1 PKI hierarchy

Vault namespaces provide administrative isolation, making them well-suited for multi-tenancy scenarios. While namespaces are often used to separate resources and configurations at the organizational level, such as lines of business or large deployments with distinct administrative practices, namespaces can also serve as strong boundaries for CA isolation. Each namespace can host one or more PKI secrets engines, enabling organizations to define separate certificate authorities and issuance policies tailored to specific use cases and business practices.

In the diagram below, we propose the following namespace structure for an enterprise CA hierarchy:

- Admin namespace: Hosts a centralized, Vault-managed subordinate CA (signed by an external root CA), used to sign issuing CA certificates
- Tenant namespaces: Hosts intermediate CAs used to issue leaf certificates for application workloads

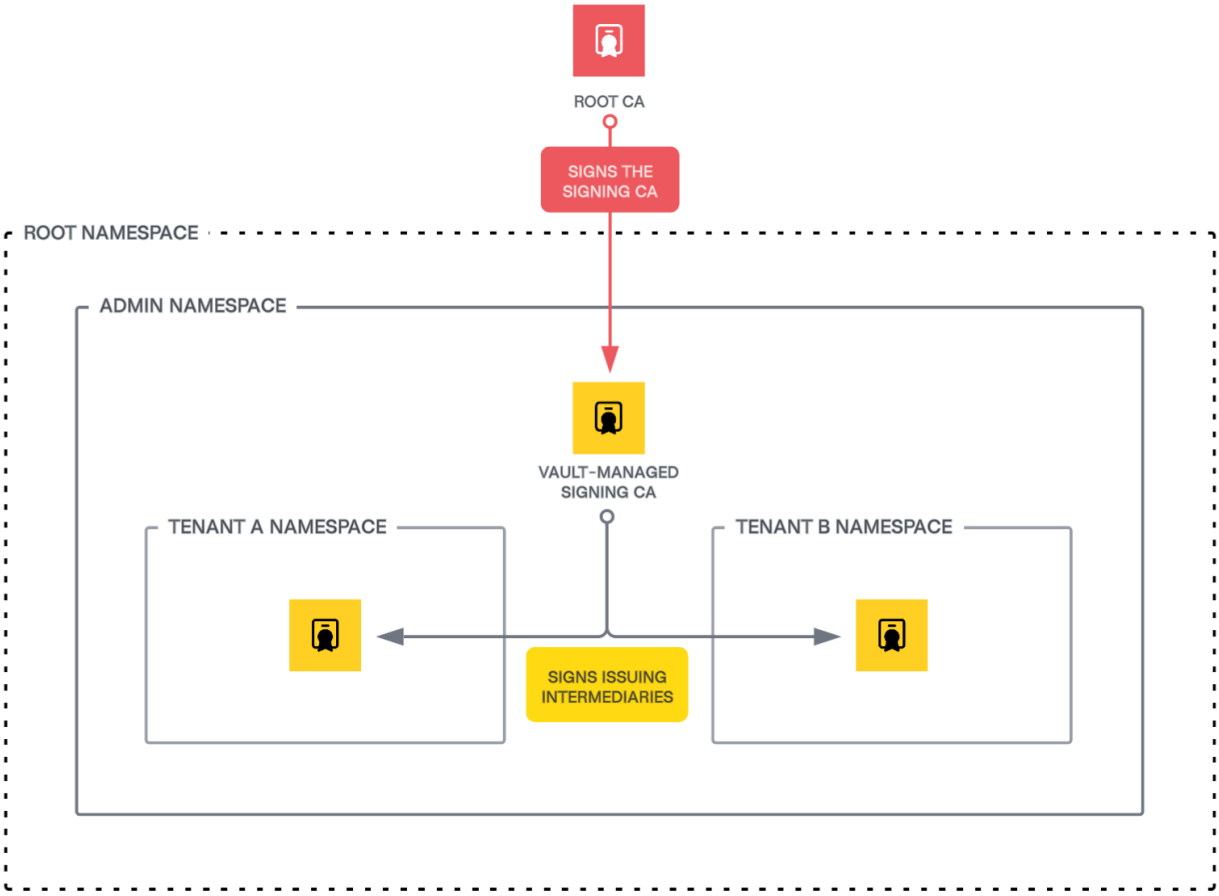


Figure 22: PKI hierarchy and Vault namespace design

Root CA

While Vault is perfectly capable of acting as a root, keep the root CA and corresponding key material outside of Vault while managing any intermediate and issuing CAs within Vault. This approach aligns with industry best practices and reduces the risk of compromise of the root CA, which should be strongly protected by a secure key management system or hardware security module (HSM). HSM systems may optionally be air-gapped (offline), depending on your specific environment and compliance requirements.

By isolating the root CA while leveraging Vault for certificate issuance, organizations enhance security, improve resiliency through automation, and enable high performance for consuming appli-

cations.

Regardless of the specific approach to root CA management, the root CA should never directly issue leaf certificates; instead, it should delegate this responsibility to intermediate CAs managed by Vault.

Vault-managed signing CA

Within Vault, a central signing CA (an intermediate CA signed by the root) resides in the administrative namespace and is responsible for signing subordinate issuing CAs. Since its primary role is to sign intermediate CAs rather than leaf certificates directly, you configure it with a relatively long lifespan, or time-to-live (TTL) value. This design enhances flexibility and simplifies the maintenance of the certificate hierarchy, avoiding the complexity and process that often accompany root CA interactions.

Consider the following factors for the Vault-managed signing CA:

- Single issuer: The signing CA should only sign Intermediate CA requests from tenant-specific PKI secrets engines.
- Key type: Elliptic Curve (EC) private keys (for example, P-384 for CAs and P-256 for leaf certificates) provide a good balance of security, performance, and compatibility across use cases (for example, IoT, web servers). Ultimately, the choice of key type may depend on your organization's preferences by your organization's preferences and standard practices. See the Vault documentation for all available options.
- Long TTL: The Vault-managed signing CA should have a relatively long TTL, as it is only used for signing tenant issuing CAs (intermediate CAs). However, its TTL should not be so long that its certificate expires simultaneously with the root CA. For example, if your Root CA has a TTL of 10 years, set the maximum TTL for the Vault-managed signing CA to 5-7 years to ensure staggered certificate renewals. Your prevailing organizational practices and security policies should inform this decision.
- Security restrictions: Keep the managed central signing CA under strict security controls; similar to the root CA, you should never use it to issue leaf certificates directly, and any interactions should be closely monitored.

Once the Vault-managed central signing CA generates a Certificate Signing Request (CSR), your external root CA must sign it. Use your organization's established methods for signing CSRs, which may involve collaboration with other teams or individuals who manage PKI infrastructure, approvals, and processes.

Issuing CA per tenant namespace

You can delegate the administration of the issuing CAs to namespace administrators (tenants) while maintaining the centralized management and integrity of the central signing CA.

When generating the tenant issuing CAs, the signing CA can impose strong restrictions on the resulting capabilities of the issuers. For example, various URI and DNS domain constraints, exclusions, and path length controls you can and should enforce according to the desired governance model.

Tenant administrators should have the ability to configure and manage their PKI engines according to their needs, within the constraints set by the signing CA. Clients of a given namespace should only be able to interact with the PKI engine in their namespace, subject to the localized governance model and access controls appropriate for that environment.

You can use Sentinel policies to enforce universal governance of PKI engine configurations at the platform level for tenant administrators and consumers. Examples of these controls include:

- Setting a global maximum TTL for certificates
- Enforcing or restricting the storage of certificates within Vault
- Restricting hostnames, subdomains, and wildcards to specific values

Outside global controls, each tenant namespace should host its own issuing CA to prevent certificate issuance across trust boundaries. Apply the following best practices for PKI administrators configuring issuing CAs:

- Certificate lifetimes: Keep application leaf certificate TTLs short (<30 days) to minimize reliance on CRLs and enhance security posture.
- Automatic tidying: If storing certificates, enable automatic cleanup of expired or revoked certificates in the storage backend.
- Automatic certificate revocation list (CRL) rebuilding: If using CRLs, automatically manage CRL rotation on a schedule to avoid expiration.
- Storage settings:
    - Use store mode where required, such as for Automatic Certificate Management Environment (ACME) challenge validation.
    - Use no-store mode when you do not need to persist certificates, as is typical in cases with ephemeral application instances or other scenarios where rapid issuance and expiration are desirable.

10.5.2 Securing CA key material

For enhanced security, use Vault's *managed key* feature to store the CA key material in an HSM (using Vault's PKCS#11 integration) or cloud-based KMS solution. This ensures key material never leaves tamper-resistant hardware.

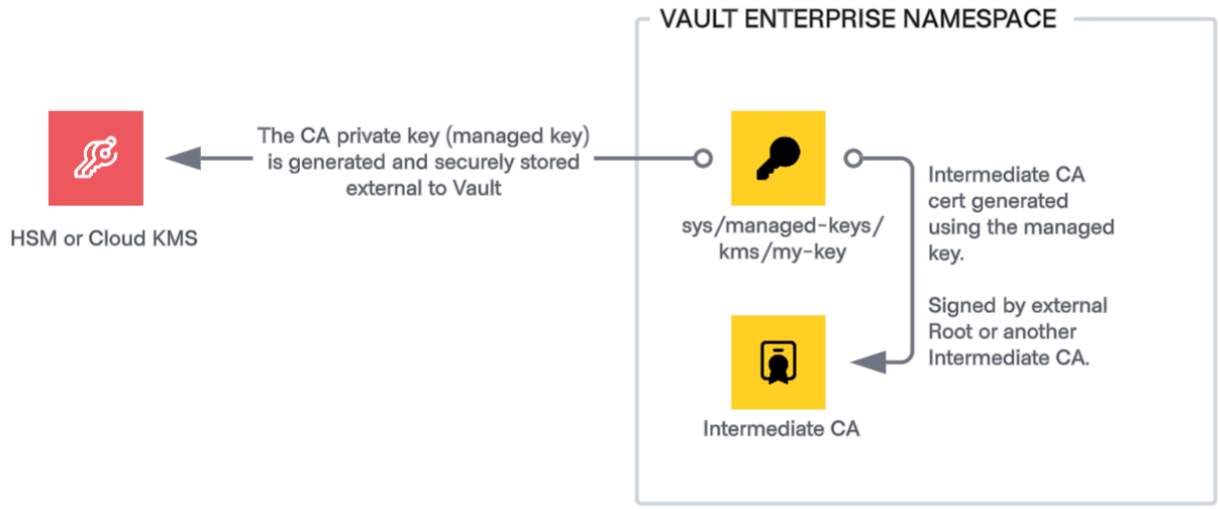


Figure 23: PKI managed keys with HSM or Cloud KMS

While offloading key material storage and signing operations to a trusted external system may enhance security and help you achieve certain policy requirements, doing so introduces a performance tradeoff in the form of increased latency. This latency is outside of Vault's control and is a byproduct of both the required network communication and the chosen HSM or KMS solution's performance capabilities. Selecting elliptic curve key types, which are considerably more efficient, and avoiding certificate storage (covered in the following section) can help offset this latency, but you should test in your environment to determine optimal configurations.

In some cases, customers may decide to optimize performance by using managed keys only for the centralized signing CA, while allowing Vault to store and manage the individual issuing CA key material. For cases where managed keys are not used, you can configure the PKI secrets engine with seal wrapping to enhance key protection. In addition to standard keyring-based encryption, seal wrapping also encrypts private keys using the seal device before storage, adding an extra layer of data security. With seal wrap enabled, Vault caches the private key in memory while in use to optimize the performance of signing operations.

10.5.3 Certificate storage

While Vault never retains private keys even when configured to generate them for clients, it can optionally store issued leaf certificates in its storage backend on a per-role basis. This behavior is configurable and often depends on the use case. A later section of this document covers details about PKI role configuration.

Several factors should inform the decision to store certificates, including the certificate's validity period, revocation workflow requirements, and any applicable internal policies or prevailing governance requirements.

In general, the shorter the certificate's TTL, the less critical revocation becomes. For example, if a certificate is valid for only 10 minutes, the process of revoking it, updating the CRL, and distributing that new CRL before the certificate naturally expires is often impractical and resource-intensive. In such cases, the cost of revocation may outweigh the benefits. It is up to you and your organization to determine what TTL threshold may justify the elimination of certificate storage in Vault.

The standard method for revoking certificates in Vault is to use the certificate's serial number, which requires the certificate to be present in Vault's storage. When certificates are not stored (that is, `no_store=true`), this administrative revocation path is unavailable.

Vault supports revocation with CRLs or Online Certificate Status Protocol (OCSP), even when certificates are not stored. Vault's Bring Your Own Certificate (BYOC) feature enables this, which allows you to supply the full certificate material (PEM-formatted data) at revocation time. This capability may influence whether you choose to persist certificates, depending on your preferred approach to revocation and audit requirements.

Disabling certificate storage (`no_store=true`) also improves PKI issuance performance, as Vault can issue certificates without writing to storage. This allows performance standby nodes to service issuance requests directly without proxying to the cluster leader, resulting in significant performance gains in high-throughput environments. Internal testing demonstrates a 3-5x improvement in leaf issuance rates when certificates are not stored. For more information about how engine configuration influences performance, see the PKI secrets engine considerations documentation.

10.5.4 Revocation considerations

It is important to consider that verification of certificate validity and revoked status is a client responsibility that is not inherent or automatic. You should investigate whether certificate status is actively

and consistently verified by clients that operate in your environment, and if so, what protocol they use (CRL or OCSP). Often, organizations believe that they must support revocation and certificate storage for compliance purposes, only to find out that no clients respect the CRL or OCSP information contained in leaf certificates. If clients do not check for revocation, there is arguably no need to maintain a CRL or run OCSP responders. We still recommend that you enable and configure CRL and OCSP so that revocation remains an option, regardless of whether you store certificates.

10.5.5 Unified CRL and OCSP with cross-cluster revocation

In performance replication topologies, where you use multiple Vault clusters for horizontal scaling, each cluster maintains its own certificate and lease storage, as well as CRL. Even though they share CA configurations and key material, clusters execute signing and issuance operations independently. Configure Authority Information Access (AIA) and CRL Distribution Point URLs for each issuer to ensure consistent revocation data across all clusters. These configurations allow unified CRLs and Online Certificate Status Protocol (OCSP) responses to be uniformly accessible across different clusters.

Consider the following key factors:

- AIA templates: Configure AIA URLs to support unified CRLs across replicated topologies and ACME services. This ensures that clients can accurately validate certificates regardless of which cluster issued the certificate.
- Cross-cluster revocation: Enable cross-cluster revocation to synchronize revocation data across all clusters when using performance replication.
- OCSP support: Configure OCSP endpoints to provide real-time certificate status information, enhancing the efficiency of the revocation process.
- If the CRL becomes excessively large, there are potential performance penalties to consider. In a high-volume environment where revocation support is necessary, forgoing CRLs in favor of a unified (cross-cluster) OCSP view may prove to be the best option. See the PKI considerations reference for additional details.

10.5.6 Automatic CRL rebuilding and rotation

Enable automatic CRL rebuilding (`auto_rebuild=true`) to ensure consistent, timely rotation and to prevent CRLs from expiring unexpectedly. The latter can occur if no revocations occur for a span

of time that extends beyond the CRL expiration date. However, enabling this setting disables the default behavior of immediately rebuilding the CRL upon each certificate revocation. In this case, if you revoke a certificate and must reflect it in the CRL right away, you must trigger a manual CRL rebuild.

For the most accurate, real-time revocation status, consider using OCSP in your environment. Unlike CRLs, OCSP responders provide immediate, up-to-date revocation information for individual certificates.

10.5.7 Tidying operations

Regular tidying operations are essential to maintain the performance and reliability of PKI deployments when you store certificates or utilize CRLs. Tidying is the process of automatically cleaning up expired certificates, issuers, revocation data, and associated metadata, ensuring that the certificate store remains efficient and uncluttered.

Regular tidying provides the following benefits:

- Performance improvement: Removing outdated certificates and metadata reduces storage burden and improves query performance.
- Scalability: Continuous tidying ensures that Vault can handle large-scale deployments without degradation over time.
- Operational efficiency: Automating tidying minimizes manual intervention, enhancing overall system reliability.

Apply the following best practices:

- Set appropriate tidying intervals based on deployment size and certificate issuance rate (for example, 24h.) Use daily tidying as a starting point which you can adjust according to your need.
- Regularly monitor the performance impact of tidying operations and adjust intervals as needed.
- Enable tidying for all tenant PKI mounts to maintain consistency across your environment.

10.6 Access controls

This section aligns with the namespace-based PKI architecture outlined earlier, ensuring that each tenant namespace has a dedicated PKI secrets engine that hosts an issuer. A Vault-wide intermediate CA residing in the `admin` namespace signs these issuing CAs, which are responsible for issuing application leaf certificates.

Vault relies on machine identity to authenticate application workloads and authorize access to secrets. See the Vault documentation to learn more about available auth methods.

1. Machine authentication per tenant
    - Each tenant namespace (for example, `tenant-1`) hosts one or more machine authentication mounts that are specific to the designated consuming application ecosystem (for example, AWS auth method).
    - Applications authenticate using this method to obtain access to the PKI secrets engine and request certificates.
2. Policy-based access to PKI roles
    - Upon successful authentication, Vault issues clients a time-bound token with an associated policy that grants access to specific API paths and methods. In this case, the policy permits access to one or more PKI engine roles.
    - Policies are dynamically mapped based on the business segment/team (for example, `team-a`), and every business segment has a designated role.
3. Simplified policy management with path templating
    - Use ACL policy path templating to dynamically assign access based on entity metadata.
    - This approach reduces manual policy configuration overhead while maintaining strict role-based access controls.

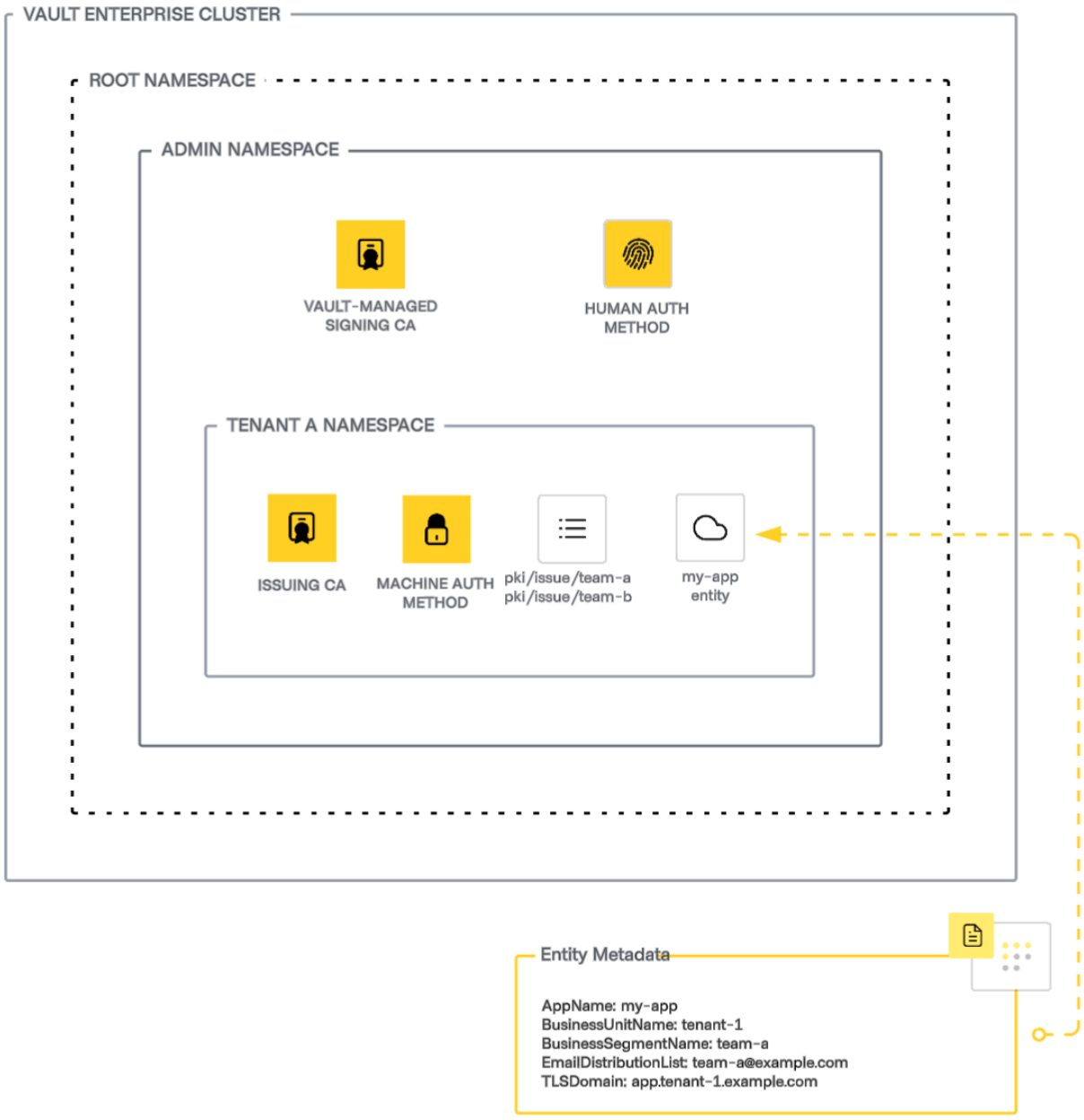


Figure 24: Access control with Vault ACL polices and entity metadata

10.6.1 PKI roles

A PKI role granularly defines the rules and constraints for certificate issuance, similar to a certificate *template* or *policy* in alternative PKI solutions. Use roles to configure certificate attributes such as key type, bit length, and usages, as well as allowed domains, SANs, and other standard X.509 metadata. A single PKI engine and corresponding CA can support many roles with differing configurations based on your requirements. Create multiple roles, each scoped to a specific application or use case, rather than create overly permissive roles that allow unnecessary latitude in request criteria. Access to each role is tightly controlled through ACL policies.

Among many other controls, you configure certificate lifetimes (X.509 `NotAfter` constraints) on a per-role basis via TTL parameters. Adopt a short-lived leaf certificate pattern to minimize reliance on CRLs (if in use) and enhance security. Use Vault Agent or a similar solution to ensure certificate lifecycles are reliably automated.

Depending on the engine and role configurations, certificate issuance can be a resource-intensive operation, requiring writes to Vault's storage backend that Vault must forward to the leader node. However, you can avoid these limitations by disabling certificate storage (and lease generation) where it is not required. When certificates are not stored, performance standby nodes can issue certificates independently, without consulting the cluster leader.

- Performance standby node behavior:
  - By default, only the active node handles PKI certificate issuance requests.
  - To distribute the workload, performance standby nodes can handle requests if certificates are not stored in Vault.
  - This setup requires a Vault Enterprise license and an appropriate load balancer configuration that distributes traffic across all cluster nodes.
- Disabling certificate storage for performance:
  - By setting `no_store=true`, Vault does not store certificates and does not generate leases, reducing storage and performance overhead.
  - Performance standby nodes can process requests, significantly improving issuance rates.
  - Note: It is still possible to revoke a certificate even if not stored in Vault via the BYOC method. You can optionally configure your audit device to write issued certificate PEM data in plaintext to better enable this approach.

If you enable certificate storage, you can configure PKI roles to associate Vault leases with issued certificates. This allows revocation through Vault's idiomatic lease management system. However, turn off lease generation for certificates and rely instead on PKI-native revocation semantics. Leases present non-trivial resource utilization and management overhead that you should avoid for PKI use cases.

The following example defines a comprehensive PKI role optimized for high-performance and large-scale certificate issuance.

CLI

```
vault write pki/roles/team-a \
  allowed_domains="tenant-1.example.com" \
  use_csr_common_name=true \
  require_cn=true \
  use_csr_sans=true \
  allow_subdomains=true \
  allow_bare_domains=false \
  allow_glob_domains=false \
  allow_localhost=false \
  allow_wildcard_certificates=false \
  allow_ip_sans=false \
  enforce_hostnames=true \
  server_flag=true \
  client_flag=false \
  code_signing_flag=false \
  organization="HashiCorp" \
  policy_identifiers="1.3.6.1.4.1.99999.999.1" \
  key_usage="DigitalSignature,KeyAgreement,KeyEncipherment" \
  key_type="ec" \
  key_bits="256" \
  max_ttl="720h" \
  no_store=true \
  generate_lease=false \
  no_store_metadata=false
```

Terraform

```
resource "vault_pki_secret_backend_role" "team-a" {
  backend                     = vault_mount.tenant_issuing_ca.path
  name                        = "team-a"
  allowed_domains             = ["tenant-1.example.com"]
  allow_subdomains            = true
  allow_bare_domains          = false
  allow_glob_domains          = false
  allow_localhost             = false
  allow_wildcard_certificates = false
  allow_ip_sans               = false
  enforce_hostnames           = true
  server_flag                 = true
  client_flag                 = false
  code_signing_flag           = false
  use_csr_common_name         = true
  require_cn                  = true
  use_csr_sans                = true
  organization                = ["HashiCorp"]
  policy_identifiers          = ["1.3.6.1.4.1.99999.999.1"]
  key_usage                   = ["DigitalSignature", "KeyAgreement", "
      KeyEncipherment"]
  key_type                    = "ec"
  key_bits                    = 256
  max_ttl                     = 2592000 # 720h in seconds
  no_store                    = true
  generate_lease              = false
```

Configuration options illustrated in this role include the following:

- Domain and SAN restrictions: Limits certificate issuance to *subdomains* of tenant-1.example.com, and restricts the use of IP SANs and wildcard certificates.
  - Valid: app1.tenant-1.example.com, app2.tenant-1.example.com
  - Invalid: tenant-1xyz.example.com, *.tenant-1.example.com
- Key type and size: Elliptic Curve (EC) P-256, balancing security and performance.
- TTL configuration: Certificates have a maximum lifetime of 30 days (720 hours) to reduce revocation needs. Requests for certificates with longer TTLs are overridden and issued with the maximum allowed value.
- Storage optimization:
  - `no_store=true` prevents Vault from storing issued certificates.
  - `generate_lease=false` disables lease tracking, reducing resource consumption.
  - `no_store_metadata=false` enables metadata storage for auditing and tracking purposes, even if not storing certificates. Note that if a certificate request contains metadata and you set this flag to false, the system forwards the request to the active leader node. This carries a performance penalty.

This configuration ensures efficient, scalable certificate issuance while adhering to Vault's best practices for performance and security.

10.6.2 ACL policies and entity metadata

Define a Vault policy that grants access to the PKI role for authorized client applications. Like any other Vault policy, it should be appropriately scoped to grant the minimum necessary permissions to the client.

For PKI engine workloads, Vault supports two methods of consumption at two separate API endpoints:

- `sign` requires a CSR in the request payload
- `issue` Vault-generated private key delivered to the client with the certificate, no CSR required in the request

Exposing both endpoints to clients allows developers to choose the workflow that works best for their application.

The example below shows a policy with the necessary permissions to generate PKI certificates using the `issue` endpoint of the role `team-a` within the PKI engine mounted at the path `pki`.

```
path "pki/issue/team-a" {
   capabilities = ["update"]
}
```

While this is a suitable approach for small environments, Vault's policy engine is capable of supporting dynamic templates to better facilitate large-scale application consumption use cases. Policy templates allow Vault administrators to define a single policy that may apply broadly across consuming clients for a particular access pattern, such as issuing certificates.

The below policy example leverages templating that references entity metadata, ensuring that only applications associated with the intended team (`TeamName`) can request certificates from the role.

CLI

```
tee /tmp/pki-policy.hcl <<EOF
path "pki/issue/{{identity.entity.metadata.TeamName}}" {
  capabilities = ["update"]
}
EOF

vault policy write pki /tmp/pki-policy.hcl
```

Terraform

```
resource "vault_policy" "pki" {
  name      = "pki"
  policy    = <<EOT
path "pki/issue/{{identity.entity.metadata.TeamName}}" {
  capabilities = ["create", "update"]
}
EOT
}
```

For general information about Vault ACL policies, see the documentation.

10.6.3 Pre-creating entities for machine authentication

To enable identity-based access control, Vault allows pre-defining entities with metadata that ACL policy templates can reference dynamically, such as in the previous example. This approach dynamically associates a machine identity (for example, AWS IAM role) with a policy via Vault's entity construct, based on available metadata.

The following example demonstrates how to complete these tasks:

1. Create an entity for an application (`my-app`).
2. Attach metadata to define business-specific attributes.
3. Link the entity to an AWS authentication identity.

The following command creates an entity with business-specific metadata that Vault ACL policies can reference dynamically.

CLI

```
vault write -format=json identity/entity \
  name="my-app" \
  metadata=AppName="my-app" \
  metadata=BusinessUnitName="tenant-1" \
  metadata=TeamName="team-a" \
  metadata=EmailDistributionList="team-a@example.com" \
  metadata=TLSDomain="app.tenant-1.example.com" \
  | jq -r ".data.id" > /tmp/entity_id.txt
```

```
The command captures the entity ID and stores it in `/tmp/entity_id.txt`
   for later reference.
:::

::: {data-environment=tcolorbox data-environment-keyword="colframe=
   HexColor656A76,coltitle=HexColor656A76,standard jigsaw,title=Terraform,
   opacityback=0.0,opacitybacktitle=0.0"}
```

```hcl
resource "vault_identity_entity" "my-app" {
  name      = "my-app"
  metadata = {
    AppName               = "my-app"
    BusinessUnitName      = "tenant-1"
    TeamName   = "team-a"
    EmailDistributionList = "team-a@example.com"
    TLSDomain             = "foo.tenant-1.example.com"
  }
}
```

To associate an AWS IAM role with the entity, retrieve the auth mount *accessor* for the desired auth method. This example mounts the AWS auth method at the path `aws`.

CLI

```
vault auth list -format=json \
  | jq -r '.["aws/"].accessor' \
  > /tmp/accessor_aws.txt
```Use this accessor to bind an AWS IAM role to the Vault entity. The
    following command associates the AWS IAM role with the pre-created entity
    :```sh
vault write identity/entity-alias \
  name="arn:aws:iam::123456789012:role/this-iam-role" \
  canonical_id=$(cat /tmp/entity_id.txt) \
  mount_accessor=$(cat /tmp/accessor_aws.txt)
```

Terraform

```
resource "vault_auth_backend" "aws" {
  type      = "aws"
}
```Use this AWS auth mount accessor to bind an AWS IAM role to the Vault
    entity. The following Terraform resource associates the AWS IAM role with
    the pre-created entity:```hcl
resource "vault_identity_entity_alias" "aws_role" {
  name           = var.aws-auth-unique-id
  mount_accessor = vault_auth_backend.aws.accessor
  canonical_id   = vault_identity_entity.my-app.id
}
```

10.6.4 Enforcing certificate constraints with Sentinel

To prevent unauthorized manipulation of certificate templates, you can use Sentinel endpoint governing policies (EGPs) to enforce additional security constraints beyond those that standard Vault ACL policies control. For example, you can use Sentinel to ensure the following:

- Applications and developers can only request certificates for subdomains present in the requestor's entity metadata.
- Only a trusted orchestrator (for example, Terraform) or an entity with predefined metadata can request certificates.

```
## This policy is to restrict the common name while issuing a pki cert
import "strings"

# Only care about write and update operations against pki/issue
precond = rule {
  request.operation in ["write", "update"] and
  strings.has_prefix(request.path, "pki/issue")
}

# Check if the trusted orchestrator makes the request
trusted_orchestrator_check = func() {

  print ("trace:identity.entity.name", identity.entity.name)

  # Check identity
  if identity.entity.name matches "terraform" {
    return true
  }
  return false
}

# Check common_name matches the entity metadata
common_name_check = func() {
  print ("trace:Request.data:", request.data)
  print ("trace:TLSDomain", identity.entity.metadata.TLSDomain)

  # Make sure there is request data
  if length(request.data else 0) is 0 {
    return false
  }

  # Make sure request data includes common_name
  if length(request.data.common_name else 0) is 0 {
    return false
  }

  # Check common_name matches app name
  if request.data.common_name matches identity.entity.metadata.TLSDomain {
    return true
  }

  return false
}

# Check the precondition before executing all above functions
main = rule when precond {
  trusted_orchestrator_check() or common_name_check()
}
```

For additional information on Vault and Sentinel, see the Vault Enterprise documentation.

10.7 Vault Agent for VM-based workloads

Using Vault Agent for certificate management of VM-based workloads.

10.7.1 Introduction to Vault Agent

Applications must interact with the Vault API to automate certificate renewal. In some cases, teams might build and maintain custom automation for authentication, certificate retrieval, renewal timing, and persistence. However, this approach may not scale well in large enterprises with diverse platforms and runtimes, or may be wholly inappropriate for the target application.

Vault Agent simplifies client-side integration with Vault by abstracting authentication, token management, and secrets consumption. For PKI workloads, it automates the certificate lifecycle and provides flexible templating features to render certificate files in formats tailored to application requirements. In addition, Vault Agent can trigger scripted actions after it issues a new certificate, such as reloading a service or calling a webhook.

10.7.2 PKI consumption workflow

This consumption pattern demonstrates automating certificate management for applications with Vault Agent. The diagram below illustrates the architecture of an example application and how Vault Agent interacts with Vault to obtain certificates. In this example, an NGINX server running on an AWS EC2 instance requires a certificate for an HTTPS listener. NGINX, like many other software applications, references certificates and private keys from files on the local machine. Other examples of software solutions that implement this pattern include Apache HTTPD, HAProxy, PostgreSQL, MySQL, MongoDB, Kafka, RabbitMQ, Jetty, Tomcat, and Elasticsearch.

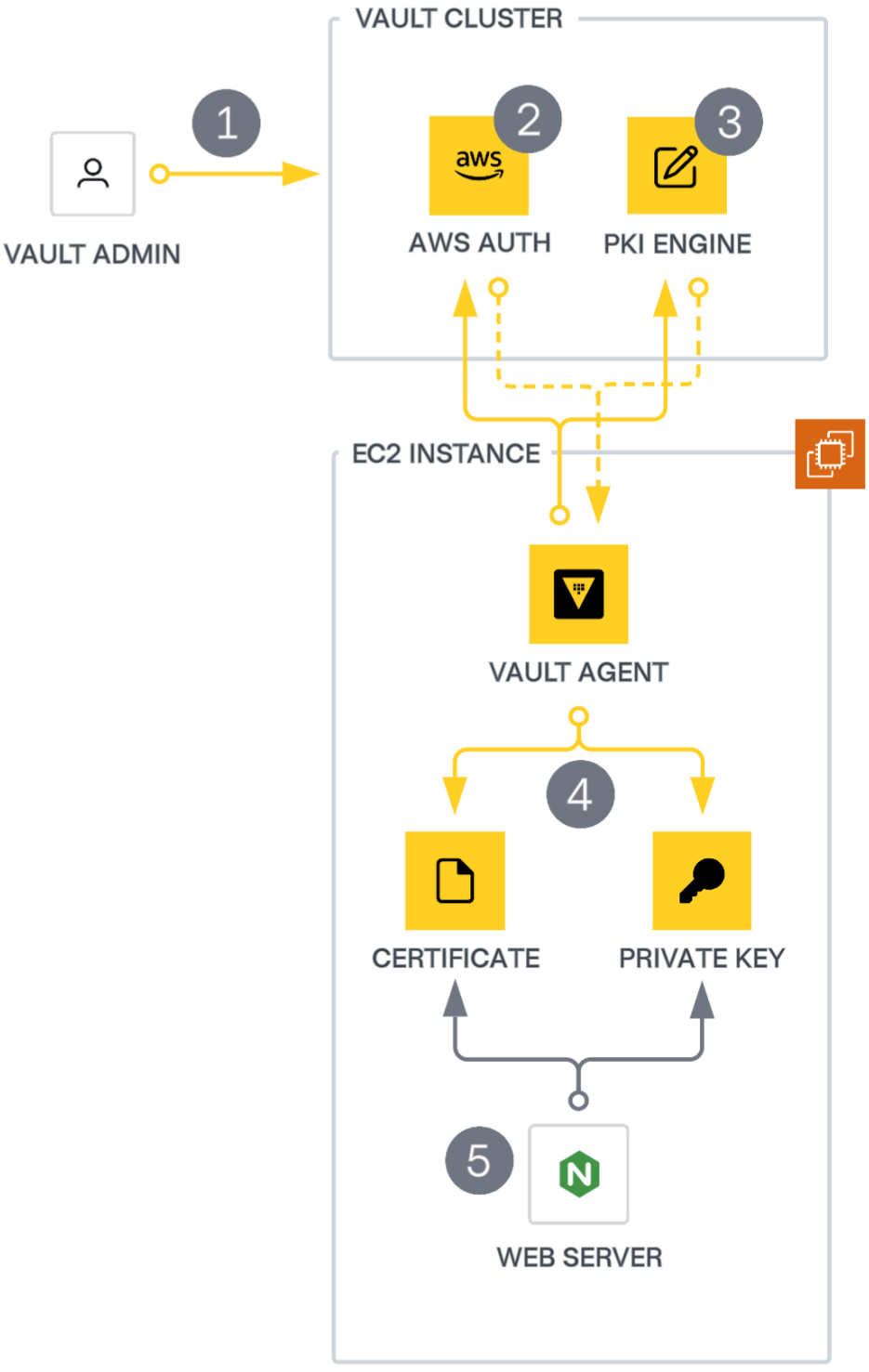


Figure 25: PKI consumption workflow

The workflow for obtaining a certificate consists of five main steps, explained in the diagram:

1. Initial configuration: A Vault administrator must configure an auth method, role, policy, and PKI engine role to allow Vault to authenticate the client and issue a valid certificate. The administrator can use tools such as the Vault CLI, API, or Terraform to perform this configuration. Use Terraform as a best practice.
2. Authentication: Upon startup, the agent automatically authenticates with the Vault cluster and obtains a token for subsequent requests. This example uses the AWS auth method.
3. Certificate retrieval: The agent retrieves a certificate by sending a request to the issuing PKI secrets engine. The secrets engine generates a certificate based on the request and the PKI role configuration and returns the certificate to the Agent.
4. Template rendering: The agent converts the certificate data into the required format for the application and saves it to the filesystem. In this scenario, you have configured the agent to store the certificate and private key in separate files, as is common for most TLS-enabled application servers.
5. Application usage: NGINX can now utilize the rendered certificate and private key by referencing the paths to the agent-rendered files.

Vault Agent continuously monitors and manages the lifecycle of the certificate and key, automating the renewal process and preventing downtime due to an expired certificate. As the expiration date approaches, the agent retrieves a new certificate from the PKI secrets engine, re-renders the certificate and private key to the file system, and performs any necessary steps to notify the application about the new certificate (steps 3-5).

10.7.3 Implementation guide

The guide assumes that you have set up a Vault Enterprise cluster for PKI issuance by following the best practices defined in the Vault HVD. It also presumes that you have implemented the guidance in Validated PKI Architecture, including engine configurations, tenant isolation, CA hierarchy, authentication methods, ACL policies, and application-specific PKI role configurations.

Certificate generation APIs (sign versus issue)

Vault supports two primary methods for obtaining certificates from the PKI secrets engine: `issue` and `sign`. Both endpoints are available to any PKI role, provided the client has appropriate access via ACL policy.

The `issue` endpoint is for fully automated workflows. On request, Vault generates a private key and issues a certificate, returning both to the client. The private key is ephemeral, is not retained by Vault, and must be securely handled by the requesting application. Vault may store the certificate itself if you configure the role to do so.

The `sign` endpoint supports a more traditional CA workflow, where the client generates its own key pair and submits a Certificate Signing Request (CSR) to Vault for evaluation. In this case, Vault signs the CSR and returns only the certificate. The private key remains under the exclusive control of the client and is never seen by Vault.

While both methods are valid, Vault Agent templating requires the issue endpoint. This capability forms the foundation of the agent-based patterns and enables hands-free certificate lifecycle management for a variety of workloads.

Authentication

Vault Agent must authenticate with Vault like any other client. Therefore, the next step is to identify the auth method that the Vault Agent will use to log in and obtain a valid token. In many cases, the Vault Platform Team makes the decision of which auth method to use.

Vault Agent supports the majority of application-oriented auth methods. As with any other use case, use a platform identity source (AWS IAM, Kubernetes service accounts, Azure MSI, and so on) instead of a static credential for Vault authentication. For workloads without a built-in source of identity, we recommend a trusted orchestrator pattern to provide credentials, such as an AppRole secret ID.

Agent installation and configuration

The first step is to install the Vault Agent binary on your application host. In our example, this corresponds to the EC2 instance running NGINX. Ideally, automation would install and configure the agent during workload provisioning. For a detailed tutorial covering agent installation and basic operation, see the Vault Agent and Vault Proxy Quick Start guide.

After installing the Vault Agent, the next step is to construct a valid configuration file that the agent will use when communicating with Vault. The configuration file specifies the Vault cluster address, auth method, and also defines templates for the secrets your application requires.

Auto-authentication

Vault Agent supports automatic authentication. The `auto_auth` block of the configuration file specifies the auth method and any relevant options. You should configure it to utilize the auth method and role defined for your application. While the exact configuration options will differ depending on your chosen auth method, the example below illustrates how you would set it up for AWS IAM authentication.

```
auto_auth {
   method "aws" {
       mount_path = "auth/aws"
       config = {
           type = "iam"
           role = "my-app"
       }
   }
}
```

Note: Auto-auth can also specify token *sinks* that store the Vault token in a local file. Sinks are optional and you should not include them unless your use case requires direct access to a Vault token. In our example, a sink is not required.

PKI template

Vault Agent has powerful templating capabilities, enabling you to render secret data as files or environment variables for applications to use. If a secret changes due to an update or scheduled rotation, the agent will ensure that any templates are re-rendered. This section will discuss best practices for using these templates to manage certificates.

Template configuration

Use `template` blocks to specify individual templates in the agent configuration. The template documentation lists all available configuration options.

```
template {
    source = "/vault-agent/pkiCerts.tmpl"
    destination = "/vault-agent/template-output/pki.data"
}
```

You can specify the template contents either inline through the `contents` field or store them in a file and provide them via the `source` field. Our example assumes you store the source template in a

file at `/vault-agent/pkiCerts.tmpl`. The template renders output to `/vault-agent/template-output/pki.data`. This `destination` file acts as a persistent cache for the `pkiCert` function output and consuming applications do not use it directly.

The `template_config` block specifies global templating options. The most important configuration option for certificates is the `lease_renewal_threshold`. This threshold determines how long the template engine waits to attempt a renewal of the underlying certificate. It is defined as a fraction of the certificate's total lifetime and defaults to 90 percent. Therefore, if your certificate has a TTL of 10 hours, the agent begins its renewal attempts approximately 1 hour before the expiration date. Depending on your configured certificate TTL and preferred practices, you may choose to adjust this buffer. The example below demonstrates changing this to 75 percent of the certificate's lifetime.

```
template_config {
   lease_renewal_threshold = 0.75
}
```

Templating functions

The templating language offers several helper *functions*. The two relevant functions for the PKI use case are `secret` and `pkiCert`. Both can retrieve certificates from Vault, but they differ in how they handle renewals.

The `secret` function is suitable for generic consumption of a variety of Vault secret types. When rendering a certificate using `secret`, the Vault Agent will always fetch a new certificate at startup or during re-authentication, even if the current certificate is valid. This approach may or may not be appropriate for your use case.

The `pkiCert` function manages rendering and renewals by checking the file system for an existing certificate on the target file system. If no certificate exists at the destination path, the agent retrieves and renders a new certificate. If a certificate is already present, the agent examines its expiration date. If the certificate has expired or is past the renewal threshold, a new one replaces it. However, if the certificate is still valid, the existing certificate remains in place and the agent continues to monitor it. Due to the enhanced behavior, we recommend the `pkiCert` function for all certificate management use cases.

Rendering certificates

In the case of our example NGINX application, the rendered certificate file should include the leaf certificate generated by Vault, concatenated with the chain of intermediate certificates used for signing. Note that our example does not include the root certificate, as we expect to distribute anchor certificates out-of-band and install them in device trust stores outside this workflow.

To construct the content of the template source file, we first specify a function (`pkiCert`), an API path (`pki/issue/team-a`), and any necessary or desired parameters that you should include in the request payload, supplied as key-value pairs (for example, `ttl`):

```
{{- with pkiCert "pki/issue/team-a" "common_name=app.tenant-1.example.com" "ttl=14d"
    "remove_roots_from_chain=true" -}}
```

This line of the template, on its own, only defines the request to Vault. It does not generate any output that will be written to the destination file.

Next, we define how the API response from Vault should be interpreted, transformed, and written to the local file system where the agent is running. The `pkiCert` function supports several helper *keys* that simplify this parsing:

- `.Cert`: the certificate body
- `.Key`: the private key
- `.CAChain`: the CA chain defined in the PKI engine configuration

> **Note**
>
> The `.CAChain` field is only available in versions 1.18 or above of Vault Agent. If you are running an older version, see the consul-template documentation for alternative options.

Adding these keys to the template source file writes the corresponding API response values to the configured default template destination. This example template would produce a single file containing the PEM-formatted private key, certificate, and CA chain:

```
{{- with pkiCert "pki/issue/team-a" "common_name=app.tenant-1.example.com" "ttl=14d"
    "remove_roots_from_chain=true" -}}
{{- .Key -}}
{{- .Cert -}}
{{- .CAChain -}}
{{- end -}}
```

The output produced by this template has limited practical utility and likely cannot be used by an application such as NGINX. However, it is important to configure a default template output contain-

ing unique secret data so that the template engine (and `pkiCert` function in particular) establishes a source of comparison when deciding whether to request a new certificate or other secret. This output corresponds to the *cache* file mentioned in the template configuration section.

To generate separate files containing the private key and certificate data as required by NGINX, additional outputs are defined using the `writeToFile` function. This function can write individual response values to specific files, concatenate secret data into existing files, and also set filesystem permissions on these outputs.

Usage of the `writeToFile` function:

```
writeToFile "[output-path]" "owner" "group" "permission-bits"
```

In this example, the private key and certificate are written to distinct files at `/etc/nginx/certs/`, in addition to the default (cache) output of the template. Appropriate filesystem permissions are also enforced.

```
{{- with pkiCert "pki/issue/team-a" "common_name=app.tenant-1.example.com" "ttl=14d"
    "remove_roots_from_chain=true" -}}
{{- .Key -}}
{{- .Cert -}}
{{- .CAChain -}}
{{- .Key | writeToFile "/etc/nginx/certs/private.key" "" "" "0600" -}}
{{- .Cert | writeToFile "/etc/nginx/certs/server.crt" "" "" "0644" -}}
{{- end -}}
```

Since a standard TLS server handshake should include any intermediate certificates needed to construct a valid trust chain, we must also render the CA chain from the API response, concatenating it onto the leaf certificate file at `/etc/nginx/certs/server.crt`. Because the chain is represented as a list object in the API response, we must iterate through the chain using the `range` function and then append those certificates to the output.

This example demonstrates a complete template, suitable for managing certificates for our NGINX application:

```
{{- with pkiCert "pki/issue/team-a" "common_name=app.tenant-1.example.com" "ttl=14d"
    "remove_roots_from_chain=true" -}}
{{- .Key -}}
{{- .Cert -}}
{{- .CAChain -}}
{{- .Key | writeToFile "/etc/nginx/certs/private.key" "" "" "0600" -}}
{{- .Cert | writeToFile "/etc/nginx/certs/server.crt" "" "" "0644" -}}
{{- range .CAChain -}}
{{- . -}}
{{- . | writeToFile "/etc/nginx/certs/server.crt" "" "" "0644" "append" -}}
{{- end -}}
{{- end -}}
```

Certificate metadata

Vault 1.17 introduced the ability to add custom metadata to your certificates, allowing you to associate any valuable context or information with them. Examples of custom metadata include application, certificate owner, contact information, business unit, risk profile, host, and more. The system stores metadata separately from the certificates themselves, enabling you to utilize this feature even if you choose not to store issued certificates.

To configure a PKI role to store metadata, regardless of whether you store certificates, set `no_store_metadata=false`. Note that including metadata in a certificate request forces the request to forward to the leader node since this action constitutes a storage write operation. This carries a performance penalty in the form of latency, similar to that of certificate storage.

You can add certificate metadata at the time of certificate creation by setting the `cert_metadata` input field. The metadata can be in any format you choose, but it must be base64-encoded before sending it to the API. Typically, you will want the metadata in a standard format, such as JSON or YAML, for easier processing. Vault Agent offers several helper methods that you can use to add this formatted metadata.

```
{{- scratch.MapSet "certMetadata" "teamName" "team-a" -}}
{{- scratch.MapSet "certMetadata" "application" "my-app" -}}
{{- scratch.MapSet "certMetadata" "contact/email" "team-a@example.com" -}}
{{- scratch.MapSet "certMetadata" "contact/slack" "#team-a" -}}
{{- $certMetadata := scratch.Get "certMetadata" | explodeMap | toJSON | base64Encode
    -}}
{{- $certMetadataArg := printf "cert_metadata=%s" $certMetadata -}}


{{- with pkiCert "pki/issue/team-a" "common_name=app.tenant-1.example.com" "ttl=14d"
    "remove_roots_from_chain=true" $certMetadataArg -}}
...
```

The preceding template uses the `scratch` helper to construct a map of the custom certificate metadata we want to assign to the certificate. The `/` separator can be used when defining the keys to create nested objects within the metadata. The map is then processed through `explodeMap`, `toJson`, and base64Encode to generate a base64-encoded JSON object.

The template passes the encoded string in as an argument to the `pkiCert` call and the system stores it by certificate serial number. You can see the resulting certificate metadata for our sample template below:

```
$ vault read -field cert_metadata pki/cert-metadata/<serial> | base64 -d | jq
{
  {
  "application": "my-app",
  "contact": {
    "email": "team-a@example.com",
    "slack": "#team-a"
  },
  "teamName": "team-a"
}
```

Integrating with applications

Often, applications need to be notified when new certificate data is available. For example, NGINX does not automatically reload the TLS certificate when it changes on disk. Instead, it continues to use the previously loaded certificate and key until the configuration is reloaded. In this section, we will discuss controlling application behavior based on changes in certificate (or other secret) data.

Post-render commands

Vault Agent enables you to execute arbitrary commands after a template is rendered, including when secrets are modified. To specify a post-render command, use an `exec` block within the template configuration stanza. The `exec` block contains a `command` field where you specify the exact command and its arguments.

The example below demonstrates how to trigger a reload of the Nginx TLS configuration with `nginx -s reload` whenever the certificate is updated. This ensures that the service is always running with a valid certificate, avoiding outages related to expired certificates.

```
template {
    source = "/vault-agent/pkiCerts.tmpl"
    destination = "/vault-agent/template-output/pki.pem"
    exec {
       command = ["nginx", "-s", "reload"]
    }
}
```

Agent deployment and operations considerations

Given that certificates are a crucial component of most web applications, it is essential that Vault Agent be deployed in a resilient manner. If your application is long-running, run the agent in daemon mode (with `exit_after_auth` set to **`false`**) and managing its execution with a service manager, such as `systemd` or Windows Service Control Manager, to ensure it starts automatically on boot and restarts in the event of any unexpected errors.

If the application requiring a certificate runs on a host with other applications that also need secrets or certificates via Vault Agent, you should deploy a single instance of Vault Agent for each application. This enables each application to authenticate individually, helping you maintain least-privilege access principles. This separation further reduces the blast radius of any potential misconfiguration.

Lastly, Vault Agent supports telemetry and logging, both of which should be used. The metrics allow you to monitor its performance, authentication status, and more. Consider ingesting your metrics and logs into your enterprise monitoring solutions to proactively identify issues with agents and prevent certificates from not being rotated due to problems like authentication failures.

10.8 Conclusion

Automating certificate renewal with Vault's PKI Secrets Engine offers a scalable and secure approach to managing certificates across diverse environments. By combining a structured PKI hierarchy with Vault Agent automation, organizations can reduce manual effort, improve operational efficiency, and strengthen their overall security posture.

This validated pattern highlights the benefits of using short-lived certificates, enforcing strict access controls, and isolating workloads through Vault namespaces and sound PKI architecture.

Successful implementation depends on understanding the limitations of this approach and addressing operational considerations. Compatibility with external systems and adherence to organizational compliance requirements must be evaluated. Close collaboration between platform operators, application developers, and security teams is essential to ensure reliable, automated certificate lifecycle management.

11 Key lifecycle management administration

> **Note**
>
> This page covers the operator setup and administration of key lifecycle management. For consumer usage workflows, see the User Guide: Key lifecycle management.

11.1 Overview

Organizations generate and store vast quantities of sensitive data. Protecting this data from unauthorized access, corruption, or breaches is crucial. Encryption serves as a critical line of defense, safeguarding information by rendering it unreadable without a decryption key. However, the security of encrypted data hinges entirely on the proper management of those keys. This is where encryption key lifecycle management comes into play.

Key lifecycle management encompasses the entire lifecycle of cryptographic keys, from their secure generation and distribution to their controlled usage, storage, archiving, and eventual destruction. Its primary objective is to ensure the confidentiality and integrity of encrypted data by safeguarding keys from cybercriminals, unauthorized individuals, or accidental exposure.

HashiCorp Vault as a KMS solution allows organizations to maintain centralized control of their keys while also providing robust key lifecycle management. We'll explore each Vault key management solution as well as best practices for leveraging Vault's capabilities to generate, store, access, rotate, and audit cryptographic keys. By adopting these best practices, organizations can ensure the security of their encryption keys, safeguarding sensitive data.

11.1.1 Bring Your Own Key (BYOK)

Bring Your Own Key (BYOK) is a key management strategy that empowers organizations to retain control over their encryption keys across various platforms and services. By generating and managing their own keys, organizations can significantly enhance data security and compliance.

The key benefits of a BYOK approach include:

- **Independent Key Generation**: BYOK allows organizations to verify the integrity and security of their encryption keys by independently generating them using trusted entropy sources. This ensures compliance with stringent security and regulatory requirements.
- **Exclusive Key Ownership**: Organizations maintain exclusive ownership of their encryption keys, ensuring complete control over data access and usage. This eliminates reliance on a third party for key management, mitigating potential risks.
- **Disaster Recovery**: By retaining the original copy of the key material outside of the cloud environment, organizations can implement robust disaster recovery plans to protect against data loss due to cloud service disruptions or failures.
- **Seamless Integration**: BYOK facilitates smooth integration between on-premises and cloud environments, enabling organizations to use the same encryption keys across different platforms. This flexibility enhances operational efficiency and reduces management overhead.

Vault offers several key lifecycle management solutions:

1. **Key Management Secrets Engine**: Enables centralized lifecycle management of cryptographic keys while allowing copies of the key material to be securely distributed to cloud key management service (KMS) providers for cryptographic operations.
2. **KMIP Secrets Engine**: Vault acts as a KMIP server, allowing services and applications to delegate cryptographic key management to Vault using a standardized protocol.

11.2 Key Management Secrets Engine

Organizations adopt a KMS to satisfy relevant compliance and privacy requirements, and to address specific security needs. Cloud providers typically offer a KMS solution with native integrations for their services. For example, Microsoft Azure provides encryption at rest for various storage services using keys managed by the Azure Key Vault KMS platform.

As organizations adopt a cloud operating model, key management becomes a significant security and operational challenge. In particular, it is challenging for organizations to maintain centralized control over the generation and lifecycle of key material while still being able to take advantage of native cloud integrations for securing data. Operating in multiple clouds can further amplify these challenges and HashiCorp Vault aims to solve these challenges by utilizing the Key management secrets engine.

The Key Management secrets engine supports lifecycle management of keys in the following KMS providers:

- Azure Key Vault
- AWS KMS
- GCP Cloud KMS

Requirements:

- Vault Enterprise license with the Advanced Data Protection (ADP) module
- Any of the cloud KMS providers (Azure Key Vault, AWS KMS or GCP KMS)
- Review the compatible key types for each KMS Provider (link)

Generally, the cloud KMS providers support key lifecycle operations such as:

- **Importing** of key material using secure key wrapping methods
- **Rotation** of keys/versions
- **Enabling and disabling** of keys/versions
- **Deletion and archival** of keys/versions

The Cloud KMS providers allow you to create a KMS key without key material and then import your own key material into that KMS key. The ImportKey implementation will carry out the secure key import procedure for the Cloud KMS provider as specified in their BYOK specification.

Key material will always be securely transferred in accordance with the key import specification of the supported KMS providers.

A typical implementation follows the below diagram

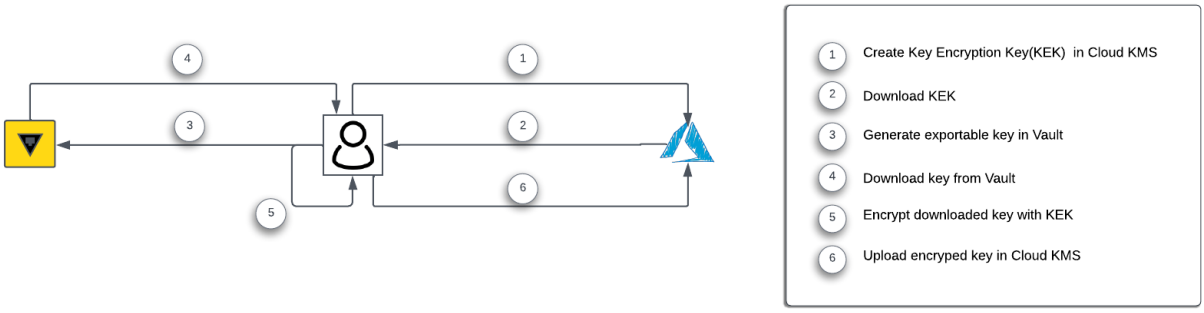


Figure 26: Implementation Diagram

11.2.1 Setup

Configure the secret engines in advance as below. These steps are usually completed by an operator or configuration management tool.

```
vault secrets enable keymgmt
```

11.2.2 Usage

After the secrets engine is mounted and a user/machine has a Vault token with the proper permission, it can use this secrets engine to generate, distribute, and manage the lifecycle of cryptographic keys in supported KMS providers.

Key Distribution

The general process for distributing encryption keys from HashiCorp Vault to the cloud KMS Provider is as below,

Create a named cryptographic key of a specific type (e.g., 2048 bit RSA key)

```
vault write -f keymgmt/key/example-key type="rsa-2048"
```

Configure the KMS provider in Vault with the required credentials

```
vault write keymgmt/kms/example-kms \
    provider="azurekeyvault" \
    key_collection="keyvault-name" \
credentials=client_id="${CLIENT_ID}" \
    credentials=client_secret="${CLIENT_SECRET}" \
    credentials=tenant_id="${TENANT_ID}"
```

The values supplied to the generic set of parameters will differ depending on the specified provider. This example represents the configuration of Azure KMS as a KMS provider.

Distribute the key from steps above to the KMS provider key vault.

```
vault write keymgmt/kms/example-kms/key/example-key \
    purpose="encrypt,decrypt" \
    protection="hsm"
```

This operation distributes a copy of the named key to the KMS provider with a specific `purpose` and `protection`. The `purpose` defines the set of cryptographic capabilities that the key will have in the KMS provider. The `protection` defines where cryptographic operations are performed with the key in the KMS provider.

Key Rotation

Rotating a key creates a new key version that contains new key material. The key will be rotated in both Vault and the KMS provider that the key has been distributed to. The new key version will be enabled and set as the current version for cryptographic operations in the KMS provider.

```
vault write -f keymgmt/key/example-key/rotate
```

Enable or Disable Key versions

The `min_enabled_version` of a key can be updated in order to enable or disable sequences of key versions. All versions of the key less than the `min_enabled_version` will be disabled for cryptographic operations in the KMS provider that the key has been distributed to. Setting a `min_enabled_version` of `0` means that all key versions will be enabled.

```
vault write keymgmt/key/example-key min_enabled_version=2
```

Remove a key from a KMS provider

This operation results in the key being deleted from the KMS provider. The key will still exist in the secrets engine and can be redistributed to a KMS provider at a later time.

To permanently delete the key from the secrets engine, the delete key API may be invoked.

```
vault delete keymgmt/kms/example-kms/key/example-key
```

11.3 General Best Practices

HashiCorp recommends the following best practices for effective key lifecycle management.

Define a process: Establish a key lifecycle management strategy encompassing key generation, distribution, utilization, rotation, and decommissioning to guarantee consistent and secure key handling throughout their existence

Key Generation: Key has to be selected based on the purpose that defines a set of cryptographic capabilities that the key will have in the KMS provider. Supported values include “encrypt”, “decrypt”, “sign”, “verify”, “wrap”, and “unwrap”.

Key Rotation: Establish a defined key rotation schedule based on risk assessment and regulatory requirements. Keep at least two versions of encryption keys to facilitate smooth transitions during rotation, considering that the time required to distribute a key to a KMS provider is proportional to the number of key versions..

> Note
>
> If a timeout occurs when distributing a key to a KMS provider, you may need to increase the value of VAULT_CLIENT_TIMEOUT. Note that rotating a key in the secrets engine will result in a new key version in Vault and in the related KMS provider.

Key Revocation: Define a revocation process to promptly deactivate and replace compromised keys. Effective key revocation is critical to preventing data exposure and limiting the damage caused by security incidents.

Monitoring and Auditing: Continuous monitoring and auditing capabilities to track key usage patterns and identify anomalous behavior will have to be established at the respective KMS provider. Regularly analyze logs and audit trails to maintain compliance and proactively detect potential security threats.

11.4 KMIP Secrets Engine

The OASIS Key Management Interoperability Protocol (KMIP) standard is a widely adopted protocol for handling cryptographic workloads and secrets management for enterprise infrastructure such as databases, network storage, and virtual/physical servers.

While Vault's Transit secrets engine provides transparent data encryption through its native APIs, many organizations have existing systems and applications that require cryptographic operations which rely on external key management services. To address this, Vault provides a KMIP secrets engine, enabling it to function as a KMIP server.

KMIP is a standardized protocol designed to abstract and offload the complexities of cryptographic operations. By delegating key storage, lifecycle management, and cryptographic operations to a KMIP server, applications can focus on their core concerns without the overhead of managing cryptographic material. Vault's KMIP secrets engine serves as a secure and efficient bridge that integrates organizations' legacy systems with modern key management practices, offering a standardized interface while leveraging Vault's inherent security and flexibility.

11.4.1 Supported Profiles

Vault implements version 1.4 of the following Key Management Interoperability Protocol Profiles:

- Baseline Server
- Symmetric Key Lifecycle Server
- Basic Cryptographic Server
- Asymmetric Key Lifecycle Server
- Advanced Cryptographic Server

11.4.2 Components

The KMIP secrets engine is composed of two components:

- First is the KMIP server component, which exposes a listener to accept KMIP client requests. Client-server communication will occur over TCP with TLS using the KMIP protocol.
- Second, the secrets engine exposes regular Vault HTTP API paths for configuring the listener, managing roles, and generating client certificates.

Vault's KMIP secrets engine listens on a separate port from the standard Vault listener. Each Vault server in a Vault cluster configured with a KMIP secrets engine uses the same listener configuration, defaulting to port 5696. The KMIP listener port is configurable, which is necessary if there are multiple KMIP secrets engine mounts configured on a cluster. KMIP clients connect and authenticate to this KMIP secrets engine listener port using Vault-generated TLS certificates. KMIP Vault clients may connect to any server in the cluster using the configured KMIP port.

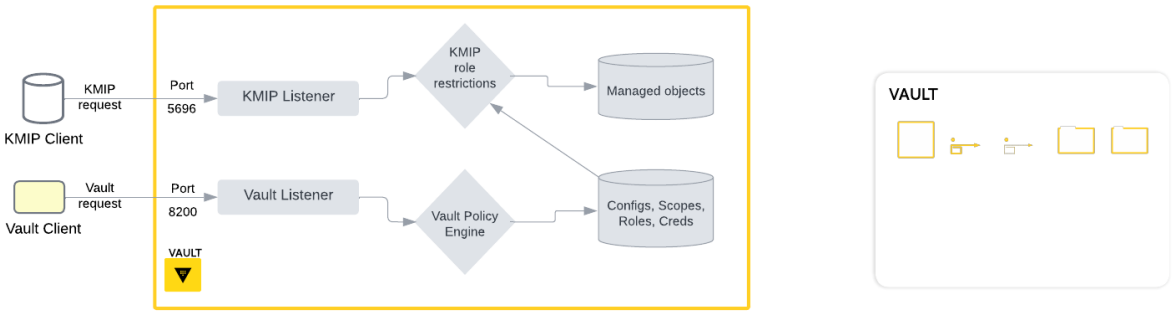


Figure 27: KMIP Client Diagram

> **Note**
>
> The KMIP listener will only be enabled for performance standby servers. This means that if performance standby is not used, the load balancer must ensure that the KMIP route is directed only to the leader of the cluster. This can be achieved through health checks via the cluster port (8200) to determine the current leader node.

11.4.3 KMIP Certificate Authority

When the KMIP secrets engine is initially configured, Vault generates a KMIP Certificate Authority (CA) to authenticate KMIP client certificates.

Vault uses the internal KMIP CA to generate unique certificates for each client authenticating to Vault over the KMIP protocol. You cannot import an externally generated KMIP certificate authority: all KMIP authentication must use the internal KMIP CA.

11.4.4 Scopes and Roles

Vault policies do not come into play during these KMIP requests. The KMIP secrets engine determines the set of KMIP operations the clients are allowed to perform based on the roles that are applied to the TLS client certificate.

Scopes are created to partition KMIP managed object storage into multiple named buckets. Within a scope, roles can be created which dictate the set of allowed operations that a particular role can perform. TLS client certificates can be generated for a role, which services and applications can then use when sending KMIP requests to Vault's KMIP secret engine.

11.4.5 KMIP Supported Operations

The below table outlines the KMIP operations supported by Vault. When you grant KMIP operations, the argument convention is `operation_<KMIP_operation>` like *operation_activate or operation_revoke*

activate	discover_versions	mac	revoke
add_attribute	encrypt	mac_verify	sign
create	get	modify_attribute	signature_verify
create_keypair	get_attribute_list	query	rng_seed
decrypt	get_attributes	register	rng_retrieve
delete_attribute	import	rekey	
destroy	locate	rekey_keypair	

Pseudo-operations:

Additionally, there are two pseudo-operations that can be used to allow or deny all operation capabilities for a role.

operation_all

operation_none

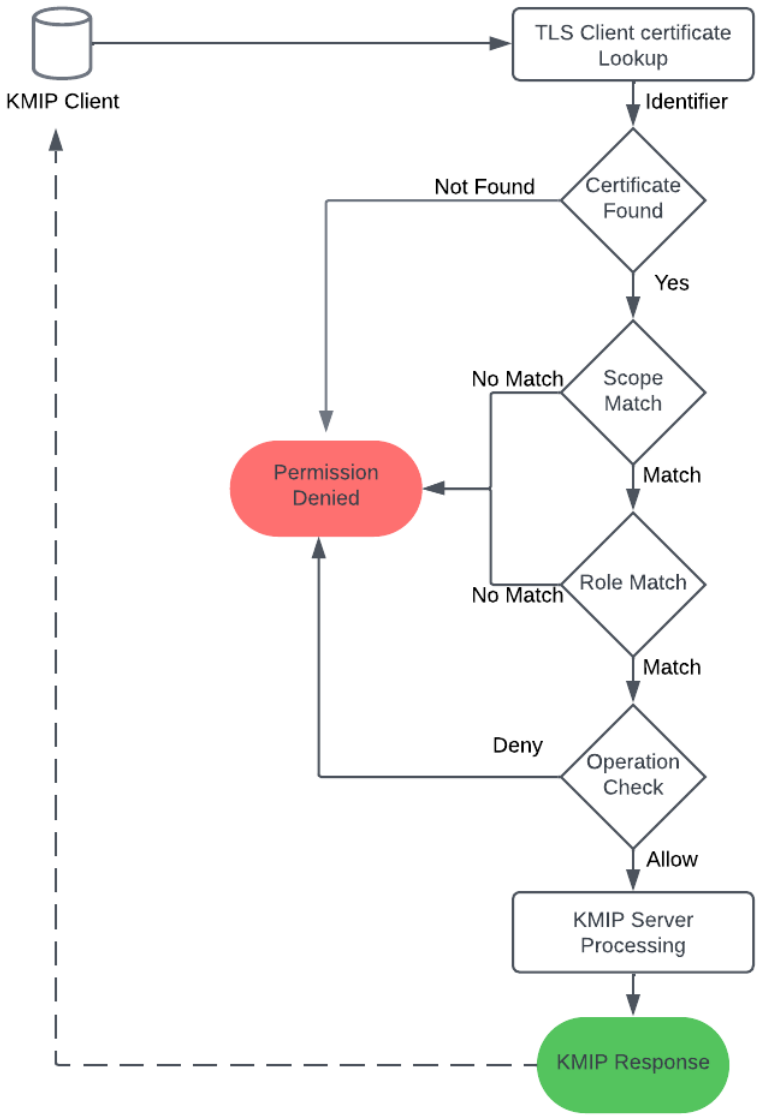


Figure 28: KMIP Client Flow Chart

11.4.6 KMIP Debugging & Troubleshoot

To facilitate debugging and troubleshooting for KMIP-specific operations, you must enable additional environment variables before starting Vault. This allows Vault to provide more detailed KMIP logs.

```
export KMIP_CONNECTION_DEBUGGING=1
export KMIP_REQUEST_DEBUGGING=1
export KMIP_RESPONSE_DEBUGGING=1
export KMIP_DECODING_DEBUGGING=1
export KMIP_INDEX_DEBUGGING=1
```

11.4.7 Implementation of Vault KMIP

Vault KMIP can be integrated with any platform that functions as a KMIP client to communicate with Vault. Since the integration heavily depends on how the KMIP client is constructed and designed, the implementation of Vault KMIP may vary across different platforms.

Below are just some examples that illustrate the differences in configuration across platforms.

Example of KMIP Implementation & Reference

Platform	Reference
MongoDB	MongoDB Field Level Encryption with HashiCorp Vault KMIP Secrets Engine MongoDB Encryption
IBM Storage Scale	Setup Encryption using HashiCorp Vault KMIP Secrets Engine as the RKM key server
AWS KMS External Key Store (XKS)	AWS XKS Integration through PKCS#11 Provider

While the examples above illustrate some Vault KMIP implementations, they do not provide a comprehensive list of compatible platforms. HashiCorp maintains a list of compatible platforms here, which includes all the platforms that have been tested to work with Vault as an external key management system.

11.4.8 General Considerations

- Vault does not require or recommend the use of a separate KMIP cluster, but a separate KMIP cluster for high-traffic environments or demanding KMIP operations should be considered.
- To ensure successful KMIP client authentication, Vault clusters operating behind a load balancer require the load balancer to be configured in pass-through mode. This configuration is essential for proper client certificate validation by Vault, as the KMIP protocol mandates direct client-server communication for authentication. The load balancer should support long-lived connections and it may use a round robin routing algorithm since Vault servers will forward to the primary Vault server if necessary.
- KMIP credentials are used for authentication to Vault. However, these keys are typically stored according to the implementation approach of various KMIP clients. It is important to ensure that appropriate security measures are in place to protect these KMIP credentials.
- KMIP client requests to performance standbys and secondary clusters will be forwarded to the leader node on the primary cluster.
- Vault audits all KMIP requests and responses. Establish monitoring and auditing to track key usage and identify suspicious behavior. Regularly analyze logs and audit trails to maintain compliance and proactively detect potential security threats.

12 Transit and Transform management

> **Note**
>
> This page covers the operator setup and management of Transit and Transform engines. For how users consume these encryption capabilities, see the User Guide: Transit and Transform secrets engines.

12.1 Prerequisites

Before deploying the Transform engine, ensure that the following prerequisites are met:

- **Vault Enterprise Deployment**: The Transform engine is a feature of Vault Enterprise and is available with the ADP license.

- **Authentication and Authorization Configuration**: Proper authentication methods and role-based access controls (RBAC) should be configured to secure access to the Transit and Transform engines.
- **Data Classification and Compliance Assessment**: Before implementing the Transform engine, conduct a thorough assessment of the data types that require protection and ensure that the configuration aligns with the relevant regulatory compliance requirements, such as GDPR, PCI-DSS, or HIPAA.

12.2 Scaling Vault for Transit and Transform operations

As your use of Vault's Transit and Transform engines grows, you may need to scale your deployment to maintain performance and reliability. Vault's horizontal scalability is particularly advantageous for the read-intensive operations common to these engines. This section provides guidance on how to effectively scale your Vault deployment to support growing Transit and Transform workloads, while also benefiting other Vault operations.

12.2.1 Key scaling considerations

- **Read-oriented Operations**: The Transit engine and most Transform engine operations (excluding tokenization) are primarily read-oriented. This allows these operations to benefit significantly from horizontal scaling. Adding non-voter (performance standby) nodes to your Vault cluster increases capacity to handle Transit and Transform operations, as these nodes can serve read requests without participating in the consensus protocol.
- **Tokenization Considerations**: Tokenization involved high-frequency write operations to store plaintext-token mappings. This can place substantial load on Vault's storage backend. To optimize performance, use external SQL stores. Configure an external SQL database (such as MySQL, PostgreSQL, or Microsoft SQL server) to store tokenization mappings. This helps offload any write-intensive tokenization tasks from Vault's internal storage to an external database, leveraging the database's ability to handle high write loads efficiently, while allowing Vault to scale its other operations effectively.

12.2.2 Scaling strategy

1. **Monitor Performance**: Regularly track key performance indicators such as CPU utilization, throughput, and latency in your Vault cluster. Establish alerts based on predefined thresholds to proactively scale your deployments before they impact performance.
2. **Optimize Load Balancing**: Configure your load balancers to distribute read traffic efficiently across non-voter nodes. This approach reserves voter nodes for write operations and cluster management tasks, improving overall performance.
3. **Scale CPU and Memory Resources**: Since Transit and Transform operations are CPU-intensive due to cryptographic processing, scaling up CPU resources on Vault nodes can improve throughput and reduce latency. Ensure sufficient memory is available to handle concurrent operations effectively.
4. **Add Performance Replicas**: For additional scaling, consider adding performance replicas. This reduces the load on the primary cluster and improves response times.

13 Monitoring and observability

> **Note**
>
> This page covers recurring monitoring, alert response, audit-log review, operational-log review, and incident response. For install-time telemetry and health endpoint setup, see the Installation Guide: Observability setup.

In this section, we focus on Vault monitoring and observability. Vault provides metrics, operational logs, audit logs, and a health API. Combined with host monitoring, use all of these elements to gather an accurate picture of the security, compliance, and performance of your Vault environment.

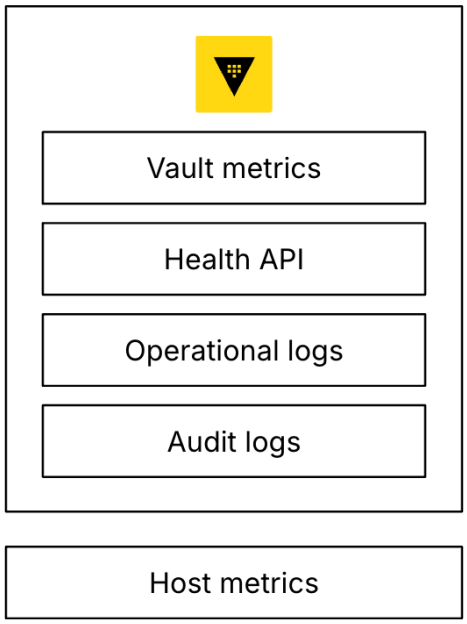


Figure 29: Vault monitoring and observability focus areas

13.1 General recommendations

To get a full picture and observe any anomalies in resource usage or metrics it is important to profile operational monitoring of Vault over time. Monitor all metrics continuously and implement tested alerting thresholds. We recommend establishing baselines and trends, with any deviation from them an indication of a potential problem.

HashiCorp cannot provide definitive metric values in every case, so detecting deviations from a moving average is the most reliable indicator. In order to define a mean value, isolate the initial sample value for each metric and then monitor this over a day and a normal business week to establish a reasonable starting point. Keep in mind that those mean values can change over time as your usage changes. Sampling and reporting are not only useful for immediate alerting but also aggregation weekly, monthly, and annually for observability reporting purposes.

Vault does not offer a solution for aggregating, visualizing, or alerting on telemetry or audit data. We recommend monitoring the following metrics with solutions such as Prometheus and Grafana or Splunk.

13.2 Telemetry

Vault provides many metrics you can export via telemetry. The Vault reference documentation provides a full list of all metrics.

The Vault Installation Guide explains how to configure the telemetry stanza in your Vault configuration.

In this section we focus on the key monitoring metrics to maintain system health and performance.

13.2.1 Key metrics

Vault's key metrics can be divided into host metrics, seal status and leadership metrics, core metrics, request handling metrics, usage metrics, replication and availability metrics, and storage metrics. The following tables show the respective metric, its unit, type, and a description of what the metric monitors. We include guidance on alert thresholds for each metric.

Host metrics

These metrics cover system level monitoring of the Vault hosts. Individual metrics for infrastructure components (CPU, memory, disk, I/O) are always good indicators of issues if they are outside of expected norms.

Metric/Unit/Type	What	Alert
`swap.used_percent` / `percent` / `gauge`	The percentage of swap space currently in use. When using Integrated Storage, swap generally should be disabled to avoid Vault's encryption keys being written to disk.	> 0
`diskio.read_bytes` / **`byte`** / `gauge`	Integrated Storage: IO is crucial for cluster stability.	Average bytes rises by 50% or more than 3 standard deviations from the mean.

Metric/Unit/Type	What	Alert
`diskio.write_bytes` / **`byte`** / `gauge`	Integrated Storage: IO is crucial for cluster stability. When using storage backends other than Integrated Storage, Vault generally does not require too much disk I/O, so a sudden change in disk activity could mean that debug or trace logging has accidentally been enabled in production, which can impact performance.	Average bytes rises by 50% or more than 3 standard deviations from the mean.
`disk.used_percent` / **`byte`** / `gauge`	Percentage of total disk space used by user processes.	Average bytes rises by 50% or more than 3 standard deviations from the mean.
`cpu.user_cpu` / `percent` / `gauge`	Percentage of CPU used by user processes (such as Vault)	Standard host threshold
`cpu.iowait_cpu` / `percent` / `gauge`	Percentage of CPU time spent waiting for I/O tasks to complete	> 10%
`net.bytes_recv`	Bytes received on each network interface	>50% deviation from baseline
`net.bytes_sent`	Bytes transmitted on each network interface	>50% deviation from baseline
`linux_sysctl_fs.file-nr`	Number of file handles used across all processes on the host	>80%
`linux_sysctl_fs.file-max`	Total number of available file handles	N/A see following information

Memory utilization

Memory usage is constant when your system is in a steady state, as there is a balance of lease issue and revocation. If you see memory usage starting to grow, then begin investigating.

Memory usage does not spike. It grows in line with an increase in the workload. If it grows more quickly, then this is an indication that something is wrong.

CPU utilization

Encryption can place a heavy demand on the CPU. If the CPU is too busy, Vault may have trouble keeping up with the incoming request load. You may also want to monitor each CPU individually to make sure requests are evenly balanced across all CPUs. Monitor and warn on `cpu.iowait_cpu` greater than 10%.

CPU monitoring is particularly important if the Transit or the Transform secrets engines are in use.

Network utilization

A sudden spike in network traffic to Vault might be the result of a misconfigured client causing too many requests or an additional load you did not plan for. Look for sudden large changes (greater than 50% deviation from baseline) to the net metrics.

File descriptor usage

Heavy utilization of the network due to lots of connections can often cause high utilization of file descriptors. Monitor `file-nr` and `file-max`. If `file-nr` reaches 80% of `file-max`, alert and investigate.

Disk usage

Implement disk space monitoring and alerting on audit endpoints. Alert when `/var/log` reaches 65% full so that there is plenty of time to review and arrange for additional disk space for the `File` audit endpoint.

Seal status and leadership metrics

These metrics cover Vault's seal and leadership election.

Metric/Unit/Type	What	Alert
`vault.core.unsealed` / `bool` / `gauge`	Vault-level detection of Vault being sealed.	Any host reports `0`
`vault.core.post_unseal` / `ms` / `gauge`	Time taken by post-unseal operations handled by Vault core.	INFO every time
`vault.core.leadership_setup_failed` / `ms` / `summary`	Time taken by cluster leadership setup failures which have occurred in a highly available Vault cluster. Monitor and alert on this for overall cluster leadership status.	> 0
`vault.core.leadership_lost` / `ms` / `summary`	Time taken by cluster leadership losses which have occurred in a highly available Vault cluster. Monitor and alert on this for overall cluster leadership status.	> 0
`vault.core.step_down` / `ms` / `summary`	Duration of time taken by cluster leadership step downs. Monitor this, and set alerts for overall cluster leadership status.	If count rises by 50% or more than 3 standard deviations from the mean.
`vault.core.license.expiration_time_epoch` / `seconds` / `gauge`	Time as epoch at which license will expire.	< 30 days left

Seal status

- Any Vault node that is sealed during normal operation needs to be attended to immediately as it could impact cluster availability. Set up alerting systems so that staff are notified of changes immediately.

Post-unseal

- The `vault.core.post_unseal` metric is good for support or debugging problems with Vault startup after unsealing.
- Unexpected spikes in the `vault.core.post_unseal` metric can alert you to problems during the post-unseal process that affect the availability of your server. Check your Vault server logs to gather more details about potential causes, such as issues mounting a custom plugin backend or setting up auditing.

Leadership metrics

- Cluster leadership changes, particularly those in unhealthy clusters, are identified via these metrics. The main target is to understand how long leaders were leaders for, when a leader loses leadership, and how long it takes to get the system fully operational following a leadership election. You want to know every time a new leader is elected as frequent leadership changes can indicate cluster instability.
- Generally, leadership transitions in a Vault cluster are infrequent. Vault provides two metrics that are helpful for tracking leadership changes in your cluster. `vault.core.leadership_lost`, which measures the amount of time a server was the leader before it lost its leadership, and `vault.core.leadership_setup_failed`, which measures the time taken by cluster leadership setup failures.
- A consistently low `vault.core.leadership_lost` value is indicative of high leadership turnover and overall cluster instability. Spikes in `vault.core.leadership_setup_failed` indicate that your standby servers are failing to take over as the leader, which is something you will want to look into right away (e.g., was there an issue acquiring the leader election lock?). Monitor and alert on both of these metrics, as they can indicate a security risk. For instance, there might be a communication issue between Vault and its storage backend or a larger outage that is causing all of your Vault servers to fail.

License metric

- The metric `vault.core.license.expiration_time_epoch` is given as Unix time. This means the time at which the Vault license expires is given as the number of seconds since January 1, 1970 (Unix epoch).

```
$ date -r 1641049200
Sat Jan  1 16:00:00 CET 2022
```

Core metrics

These metrics cover the Vault binary.

Metric/Unit/Type	What	Alert
`vault.runtime.alloc_bytes` / **`byte`** / `summary`	Number of bytes allocated by the Vault process.	3 x 20% deviations from the mean
`vault.runtime.sys_bytes` / **`byte`** / `summary`	Number of bytes allocated to the Vault process.	3 x 20% deviations from the mean
`vault.runtime.num_goroutines` / `goroutines` / `summary`	Number of goroutines associated with the vault process. This metric can serve as a general system load indicator and is worth establishing a baseline and thresholds for alerting. Blocked goroutines can increase memory usage and slow garbage collection.	3 x 20% deviations from the mean

Metric/Unit/Type	What	Alert
`vault.runtime.gc_pause_ns` / `nanosecond` / `sample`	GC pause is a stop-the-world event, meaning that all runtime threads are blocked until GC completes. Normally these pauses last only a few nanoseconds. But if memory usage is high, the Go runtime may GC so frequently that it starts to slow down Vault. Sample every second	>33,330,000 nanoseconds per second = WARN (2 s/min); >83,330,000 nanoseconds per second = CRITICAL (5 s/min)
`vault.barrier.get` / `ms` / `summary`	Duration of time taken by GET operations at the barrier.	If count rises by 50% or more than 3 standard deviations from the mean.
`vault.barrier.put` / `ms` / `summary`	Duration of time taken by PUT operations at the barrier.	If count rises by 50% or more than 3 standard deviations from the mean.
`vault.core.mount_table.size` / `bytes` / `gauge`	Size of a particular mount table. This metric is labeled by table type (auth or logical) and whether or not the table is replicated (local or not).	If count rises by 50% or more than 3 standard deviations from the mean. All tables.
`vault.metrics.collection` / `summary`	Time taken to collect usage gauges, labeled by gauge type.	If count rises by 50% or more than 3 standard deviations from the mean.
`vault.metrics.collection.error` / `counter`	Errors while collection usage gauges, labeled by gauge type.	> 0, though a period of detection and evaluation to provide protection of temporal issues/false positives is recommended.

Request handling metrics

These metrics represent counts and measurements that are provided by Vault core request handlers.

Metric/Unit/Type	What	Alert
`vault.core.handle_request` / `ms` / `summary`	Duration of requests handled by Vault core and is a key measure of Vault's response time or number of requests.	>50% or >3 standard deviations from the mean.
`vault.core.handle_login_request` / `ms` / `summary`	Duration of login requests handled by Vault core and is a key measure of Vault's user login response time or number of login requests.	>50% or >3 standard deviations from the mean.

- The `vault.core.handle_login_request` metric (using the mean aggregation) gives you a measure of how quickly Vault is responding to incoming client login requests. If you observe a spike in this metric but little to no increase in the number of tokens issued (`vault.token.creation`), you will want to immediately examine the cause of this issue. If clients cannot authenticate to Vault quickly, they may be slow in responding to the requests they receive, increasing the latency in your application.
- `vault.core.handle_request` is another good metric to track, as it can give you a sense of how much work your server is performing and whether you need to scale up your cluster to accommodate increases in traffic levels. Or, if you observe a sudden dip in throughput, it might indicate connectivity issues between Vault and its clients, which warrants further investigation.
- Look for changes to `vault.core.handle_request` that exceed 50% of baseline values, or more than 3 standard deviations above baseline. Changes for `vault.core.handle_login_request` to the count or mean fields that exceed 50% of baseline values, or more than 3 standard deviations above baseline.
- Route-Specific Metrics - Vault provides metrics about operations against specific routes, including those in use by enabled secrets engines. When using a time-series database to gather metrics for this reason over time, performance recording per API endpoint can be

understood. Route-specific metrics are use case specific and further discussion might be warranted depending on the use case.

Usage metrics

Token creation rate correlates to established normal behavior. A spike may indicate application misconfiguration.

Metric/Unit/Type	What	Alert
`vault.token.create_root` / `tokens` / `counter`	Number of created root tokens. Does not decrease on revocation.	On change
`vault.token.creation` / `token` / `counter`	Number of service or batch tokens created.	If count rises by 50% or more than 3 standard deviations from the mean.
`vault.token.create` / `ms` / `summary`	The time taken to create a token.	>50% or >3 standard deviations from the mean.
`vault.token.store` / `ms` / `summary`	Time taken to store an updated token entry without writing to the secondary index.	Establish at baseline over a week's monitoring, and alert on more than 3 standard deviations from the mean.
`vault.expire.num_leases` / `leases` / `gauge`	Number of all leases which are eligible for eventual expiry	If count rises by 50% or more than 3 standard deviations from the mean.
`vault.expire.renew` / `ms` / `summary`	Time taken to renew a lease.	Establish at baseline over a week's monitoring, and alert on more than 3 standard deviations from the mean.
`vault.expire.revoke` / `ms` / `summary`	Duration of time to revoke a token.	If count rises by 50% or more than 3 standard deviations from the mean.

Metric/Unit/Type	What	Alert
`vault.identity.num_entities` / `entities` / `gauge`	Number of identity entities	If count rises by 50% or more than 3 standard deviations from the mean.
`vault.identity.entity.count` / `entities` / `gauge`	The number of identity entities stored in Vault, grouped by namespace.	Use this integer for trend analysis. Monitor per namespace counters as well as overall count per cluster as in the next item.
`vault.identity.num_entities` / `entities` / `gauge`	The number of identity entities stored in Vault	The increase in this number over time may or may not correspond to the number of additional namespaces added to the system, but will allude to the total number of entities the system has to handle and is useful for trend analysis. Use with `vault.identity.entity.count` per namespace to identify team changes over time.

Root token creation

- The metric `vault.token.create_root` is a counter that increments any time a root token gets created, but actual usage of the root token can be identified in the audit logs.
- Root tokens have the root policy attached to them and can do anything in Vault. Only generate root tokens when absolutely necessary and avoid persisting them. We recommend using the initial root token for just enough initial setup (e.g. setting up auth methods and policies necessary to allow administrators to acquire more limited tokens).

Number of tokens

- Monitoring the volume of token creation (`vault.token.creation`) together with the login request traffic (`vault.core.handle_login_request` aggregated as a count) can help you understand how busy your system is overall. If your use case involves running many short-lived processes (e.g., serverless workloads), you might have hundreds or thousands of functions spinning up and requesting secrets at once. In this case, you will see correlated spikes in both metrics.

Number of leases

- These metrics represent lease measurements that are provided by Vault. Large and unexpected delta in count can indicate a bulk operation, load testing, or runaway client application is generating excessive leases and requires immediate investigation.
- The metric `vault.expire.num_leases` stands for the number of leases that are eligible for eventual expiry. Leases associated with tokens and dynamic secrets have a short TTL. The default setting is 32 days, and expiring leases en-masse can be problematic. What to look for: A large and unexpected delta in count can indicate a bulk operation, load testing, or runaway client application is generating excessive leases and requires immediate investigation.
- Monitoring the current number of Vault leases (`vault.expire.num_leases`) can help you spot trends in the overall level of activity of your Vault server. A rise in this metric could signal a spike in traffic to your application, whereas an unexpected drop could mean that Vault is unable to access dynamic secrets quickly enough to serve the incoming traffic.

Expiration

- The metrics `vault.expire.revoke` and `vault.expire.renew` provide the lease revoke/renewal time when a TTL is reached. Long times here indicate slow revocation/renewal.
- Vault automatically revokes the lease on a token or dynamic secret when its TTL is up. Security operators can also manually revoke a lease in the event of a suspected intrusion. When a lease is revoked, the data in the associated object is invalidated and its secret can no longer be used. A lease can also be renewed before its TTL is up if you want to continue retaining the secret or token.
- Since revocation and renewal are important to the validity and accessibility of secrets, you will want to track that these operations are completed in a timely manner. Additionally, if secrets are not properly revoked, and attackers gain access to them, they can infiltrate your system

and cause damage. If you are seeing long revocation times, review your server logs to see if there was a backend issue that prevented revocation.

Identity metrics

- These metrics represent identity entity measurements that are provided by Vault. This rises in line with onboarding teams although some existing teams may generate disproportionate numbers of additional identities depending on use case.

Replication and availability metrics

Monitoring this metric can provide insights into replication usage and saturation, while also revealing issues with storage performance. Increased time spent in Merkle diff operations can be indicative of a larger problem in determining differences between data in the two clusters. Closely monitoring this metric helps to identify potential replication breaking issues.

Metric/Unit/Type	What	Alert
`vault.merkle.flushDirty` / `ms` / `summary`	Time taken to flush dirty pages to storage. Provides insight into replication usage/saturation, and storage performance issues.	If count rises by 50% or more than 3 standard deviations from the mean. Note that it depends on system and network resources as to how long it will take to establish a mean.

Metric/Unit/Type	What	Alert
`vault.merkle.diff` / `ms` / `summary`	Time taken to perform a Merkle diff between the two clusters. Increased time spent in Merkle diff operations can be indicative of a larger problem in determining differences between data in the two clusters. Closely monitoring this metric helps to identify potential replication breaking issues.	If count rises by 50% or more than 3 standard deviations from the mean. Note that it depends on system and network resources as to how long it will take to establish a mean.
`vault.merkle.flushDirty.num_pages` / `pages` / `gauge`	Number of pages flushed.	If count rises by 50% or more than 3 standard deviations from the mean. Used with the metrics mentioned previously to determine extent of merkle sync.
`vault.merkle.saveCheckpoint` / `ms` / `summary`	Time taken to save the checkpoint.	If count rises by 50% or more than 3 standard deviations from the mean.
`vault.merkle.saveCheckpoint.num_dirty` / `pages` / `gauge`	Number of dirty pages at checkpoint.	If count rises by 50% or more than 3 standard deviations from the mean.
`vault.wal.deleteWALs` / `ms` / `summary`	Time taken to delete a Write Ahead Log (WAL).	If count rises by 50% or more than 3 standard deviations from the mean.

Metric / Unit / Type	What	Alert
`vault.merkle.remote_state_snapshot` / `ms` / `summary`	This metric represents the duration of time taken to perform a remote state snapshot on the other cluster before checking for differences.	If count rises by 50% or more than 3 standard deviations from the mean.
`vault.merkle.reindex` / `ms` / `summary`	Reindex duration. A reindex is typically IOPS intensive, affects all relevant keys, and requires updating of all sub trees. Monitoring reindex usage and saturation is key to ensuring healthy replication and detecting performance bottlenecks.	If count rises by 50% or more than 3 standard deviations from the mean.
`vault.replication.wal.last_wal` / `sequence_number` / `gauge`	Index of the last WAL.	INFO
`vault.replication.wal.last_dr_wal` / `sequence_number` / `gauge`	Index of the last DR replication WAL.	INFO
`vault.replication.wal.last_performance_wal` / `sequence_number` / `gauge`	Index of the last PR WAL.	INFO
`vault.replication.fsm.last_remote_wal` / `sequence_number` / `gauge`	Index of the last remote WAL.	INFO
`replication.rpc.client.stream_wals` / `ms` / `summary`	Duration of time taken by the client to stream WALs.	INFO

Metric/Unit/Type	What	Alert
`vault.replication.rpc.client.fetch_keys` / `ms` / `summary`	Duration of time taken by a client to perform a fetch keys request.	INFO
`vault.replication.rpc.client.conflicting_pages` / `ms` / `summary`	Duration of time taken by a client conflicting page request.	INFO
`vault.replication.merkleSync` / `ms` / `summary`	Duration of time to perform a Merkle Tree-based sync using the last delta generated between the clusters participating in replication.	INFO
`vault.replication.merkleDiff` / `ms` / `summary`	Duration of time to perform a Merkle Tree based delta generation between the clusters participating in replication.	INFO
`vault.wal_gc_total` / `WAL` / `counter`	Number of Write Ahead Logs (WAL) on disk.	INFO
`vault.wal.persistWALs` / `ms` / `summary`	Time required to persist the Vault WAL to the storage backend.	>1000 ms
`vault.wal.flushReady` / `ms` / `summary`	Time required to flush the WAL to the persist queue.	>500 ms
`vault.wal.write_controller.reject_fraction` / `number` / `gauge`	The estimated fraction of write requests that must be rejected to maintain cluster stability.	INFO
`vault.ha.rpc.client.forward` / `ms` / `summary`	Time taken to forward a request from a standby to the active node	If count rises by 50% or more than 3 standard deviations from the mean.

Metric/Unit/Type	What	Alert
`vault.ha.rpc.client.forward.errors` / `errors` / `counter`	Number of standby requests forwarding failures.	> 0. Assess the suitability of this metric in case temporal network issues cause false positives.

WAL processing

- The Vault write-ahead logs (WALs) are used to replicate Vault between clusters. WALs are kept even if replication is not currently enabled. The WAL is purged every few seconds by a garbage collector. But if Vault is under heavy load, the WAL may start to grow, putting pressure on the backend.
- `vault.wal.write_controller.reject_fraction` is connected to Vault's write overload protection. Consistently high fractions could indicate that there is a hardware failure or that the initial sizing has been outgrown by the actual workloads.

Integrated storage metrics

Monitor and trend the following metrics if you are using the Integrated Storage backend for Vault Enterprise.

Metric/Unit/Type	What	Alert
`vault.raft.leader.lastContact` / `ms` / `summary`	Measures the time since the leader was last able to contact the follower nodes when checking its leader lease.	> 0
`vault.raft.state.candidate` / `elections` / `counter`	Increments whenever raft server starts an election.	On change
`vault.raft.state.leader` / `leaders` / `counter`	Increments whenever raft server becomes a leader.	On change

Metric/Unit/Type	What	Alert
`vault.raft.get` / `ms` / `summary`	Time to retrieve a key from underlying storage.	If count rises by 50% or more than 3 standard deviations from the mean.
`vault.raft.put` / `ms` / `summary`	Time to persist a key in underlying storage.	If count rises by 50% or more than 3 standard deviations from the mean.
`vault.raft.list` / `ms` / `summary`	Time to retrieve a list of keys from underlying storage.	If count rises by 50% or more than 3 standard deviations from the mean.

13.2.2 Performance monitoring

Several factors impact the performance of Vault, ranging from the underlying hardware to resource limits. For more information on the different factors you can review the performance tuning guide. We recommend monitoring system metrics like memory, CPU, and network utilization. Some metrics in Vault can help you narrow down the underlying reason for performance issues. Configuring alerting accordingly can prevent performance issues in the first place.

Pay particular attention to the number of active leases with `vault.expire.num_leases` as described in the Usage Metrics section. An expiring lease is a write operation to Vault storage. By selecting the shortest possible TTL for your use case you can prevent a large number of leases expiring at the same time and space out those writes to ease the load on Vault storage.

Monitor the `vault.runtime.num_goroutines` metric. Blocked goroutines can slow down memory and garbage collection. More details and specific recommendations are described in the Core Metrics section.

Replication within a cluster and cross-cluster is a key factor for improving Vault's availability and performance. As such, if there is an issue with replication it can have negative effects on overall performance. Monitor those metrics as described in the replication and availability section of this guide. Hitting Vault's limits and maximums can also have a negative effect on overall performance.

Metric/Unit/Type	What	Alert
`vault.identity.upsert_entity_txn` / `ms` /`summary`	Time required to upsert an entity to the in-memory database and persist the data to storage	If count rises by 50% or more than 3 standard deviations from the mean.
`vault.identity.upsert_group_txn` / `ms` /`summary`	Time required to upsert group membership to the in-memory database and persist the data to storage	If count rises by 50% or more than 3 standard deviations from the mean.
`vault.core.fetch_acl_and_token` / `ms` /`summary`	Time required to fetch ACL and token entries	If count rises by 50% or more than 3 standard deviations from the mean.

Entities and groups

Of particular note here are usage metrics tied to Entities and Groups. As the number of entities rises (`vault.identity.num_entities`) the more taxing additions and updates to entities and groups on the overall system become. To remediate long times required for those updates, you can consider using smaller internal groups or using external groups.

Policies

Another metric that can indicate a reason for increased response times is linked to the number of policies. The higher the number of policies, the higher the time it takes to assemble the access control list for a token (`vault.core.fetch_acl_and_token`).

13.3 Audit logs

Audit devices record a detailed audit log of requests and responses from Vault, helping organizations meet their compliance requirements. Refer to the Vault audit log entry schema for the full description of request and response objects. Enable at least one audit device or Vault does not respond to requests as it cannot log them.

The general purpose of the audit log is for troubleshooting events that have adversely affected operations, rather than as an ongoing monitoring tool. This is because the audit log is an audit trail primarily designed to log requests and responses to the Vault API. In this section, we explore how to leverage the audit logs for troubleshooting.

13.3.1 General

Expect a continuous addition to the audit log endpoints. Configure an alert to notify if this stops for any reason, especially during normal business hours and maintenance windows. We cover this more in the Audit device metrics section. We also recommend monitoring the highest delta of audit log aggregation per day as a percentage of the whole stored in the `File` audit endpoint. This identifies any significant jump in activity.

13.3.2 Usage patterns

Use Vault's audit logs for auditing purposes (who did what when), to detect errors (like failed authentication or access denied), and to create metrics and visualize user patterns. Similar to operational monitoring, you can use audit logs to define baseline usage, alert on high deviations, and therefore be able to detect potential misconfigurations in applications consuming Vault. For example, if on a typical day there are 500 authentications with a token and 100 reads of a KVv2 secrets engine within a namespace, and suddenly there are 100,000 reads, there is probably an application misconfigured somewhere or a new usage pattern. If there is a high deviation in the number of operations per time period, for example, a sudden influx in key-value reads, it might be a misconfigured app or attack, or you might need to implement rate limits.

The following examples are some observable usage patterns in the Vault audit log:

- Use of the root token
- Creation of a new root token
- Increased attempts to access varying invalid paths with the same credentials
- Increased attempts to access varying paths with insufficient permissions with the same credentials
- Special tokens with extended permissions (Terraform, CI/CD, etc.) used from an unexpected IP address
- Permission denied (403) responses

- Invalid username or password
- Vault policy modifications
- Enabling a new authentication method
- Modification of an authentication method role
- Creation of a new authentication method role
- Use of Vault by people-related accounts outside regular business hours
- Vault requests originating from unrecognized subnets
- Authentication attempts with varying credentials from the same subnets
- A lot of authentication attempts in a short time
- Sealing of Vault
- Changes to the audit devices
- K/V secret actions like create, update, and delete
- PKI certificate generation and revocation
- Changes to the Transit Minimum Decryption Version Config
- Transit key deletion

We strongly recommend profiling Vault to find out what normal usage looks like in your environment and to be able to act on observed deviations.

13.3.3 Privileged endpoints

There are endpoints that are only accessed in an emergency or during maintenance windows. Configure alerts for any access to the following endpoints:

- `/sys/policy`
- `/sys/policies`
- `/sys/rekey`
- `/sys/rekey-recovery-keys`
- `/sys/replication`
- `/sys/audit`
- `/sys/audit-hash`
- `/sys/generate-root`
- `/sys/rotate`

As requests for these API endpoints go to the audit log, use a SIEM log aggregation service to detect endpoint requests and raise alerts to operational and security staff.

See also: https://developer.hashicorp.com/vault/docs/concepts/policies#root-protected-api-endpoints

13.3.4 Audit device metrics

Metric/Unit/Type	What	Alert
`vault.audit.log_request_failure` / `failures` / `counter`	Count of failed attempts to log requests to an enabled audit device.	> 0
`vault.audit.log_response_failure` / `failures` / `counter`	Count of failed attempts to log responses to an enabled audit device.	> 0
`vault.audit.file.log_request` / `ms` / `summary`	Count of requests to an enabled `file` audit device.	>50% or >3 standard deviations from the mean.
`vault.audit.file.log_response` / `ms` / `summary`	Count of responses to log requests specifically to an enabled `file` audit device.	>50% or >3 standard deviations from the mean.
`vault.audit.syslog.log_request` / `ms` / `summary`	Count of requests to an enabled `syslog` audit device.	>50% or >3 standard deviations from the mean.
`vault.audit.syslog.log_response` / `ms` / `summary`	Count of responses to log requests specifically to an enabled `syslog` audit device.	>50% or >3 standard deviations from the mean.
`vault.audit.socket.log_request` / `ms` / `summary`	Count of requests to an enabled `socket` audit device.	>50% or >3 standard deviations from the mean.
`vault.audit.socket.log_response` / `ms` / `summary`	Count of responses to log requests specifically to an enabled `socket` audit device.	>50% or >3 standard deviations from the mean.
`vault.audit.log_request` / `ms` / `summary`	Duration of time taken by all audit log requests across all audit log devices.	>50% or >3 standard deviations from the mean.

Metric/Unit/Type	What	Alert
`vault.audit.log_response` / `ms` / `summary`	Duration of time taken by audit log responses across all audit log devices.	>50% or >3 standard deviations from the mean.

Request and response failures

This is a particularly important metric. Any non-zero value here indicates that there was a failure to make an audit log request to any of the configured audit log devices.

When Vault cannot log to any of the configured audit log devices it ceases all user operations. Begin troubleshooting the audit log devices immediately if this metric continually increases.

Look for any anomalous spikes or incremental increase in `vault.audit.log_request_failure` and `vault.audit.log_response_failure` as they can indicate blocking on a device. If this occurs, inspecting your audit logs in more detail can help you identify the path to the problematic device and provide other clues as to where the issue lies. For instance, when Vault fails to write audit logs to syslog, the server publishes error logs like these:

```
2020-10-20T12:34:56.290Z [ERROR] audit: backend failed to log response: backend=
    syslog/ error="write unixgram @->/test/log: write: message too long"
2020-10-20T12:34:56.291Z [ERROR] core: failed to audit response: request_path=sys/
    mounts
 error="1 error occurred:
        * no audit backend succeeded in logging the response
```

13.3.5 Comparing sensitive values

In Vault audit logs, sensitive information written as JSON strings is HMAC'd by default. This includes error messages. When querying the audit logs for a specific error message you can use the audit-hash API to find the corresponding hash.

```
$ vault write /sys/audit-hash/file input="invalid username or password"
Key     Value
---     -----
hash    hmac-sha256:5c9b40bfb148f1247409b7c780a5c2352cd67afd40bf6ac17a48a1911dc48d75
```

13.4 API-based monitoring

13.4.1 Health API

Monitor the status of the Vault cluster and its nodes with the `sys/health` endpoint. The Vault API docs include the default status codes.

- Configure an alert when a standby node cannot connect to the active node for a prolonged time (status code 474) as this means the node is not part of the cluster. This leads to a negative impact on cluster availability.
- Configure an alert if Vault is sealed (status code 503) as it cannot serve any requests in the sealed state.

13.4.2 Transactional monitoring

Transactional monitoring is a way to mimic and test user behavior. While monitoring all other aspects of the Vault service are useful in understanding the health and performance of the service, measuring an actual user transaction is particularly of use observability-wise.

An example of transactional monitoring would be a script that does the following:

- Authenticate with Vault and retrieve a service token.
- Use the service token to write a secret.
- Use the service token to retrieve a secret.
- Compare the secret to expected values.

The following example is a basic script with the ldap and the kv secrets engine mounted to compare a secret value against an expected control value.

```
#!/bin/bash

# Variables
CONTROL_VALUE="<value you expect the secret to have>"
USERNAME="<username>"
PASSWORD="<password>"
SECRET_KEY="<key>"
SECRET_MOUNT="kv"
SECRET_PATH="test-secret"

# Authenticate to Vault
vault login -method=ldap username="$USERNAME" password="$PASSWORD"

# Write secret to Vault
vault kv put -mount="$SECRET_MOUNT" "$SECRET_PATH" "$SECRET_KEY"="$CONTROL_VALUE"

# Retrieve secret and test
TEST_VALUE=$(vault kv get -format=json -mount="$SECRET_MOUNT" "$SECRET_PATH" | jq -r
    ".data.data.$SECRET_KEY")

# Compare retrieved value with expected value
if [[ "$TEST_VALUE" == "$CONTROL_VALUE" ]]; then
    echo "SUCCESS: Retrieved value matches the expected secret value"
    exit 0
else
    echo "FAILURE: Retrieved value does not match the expected secret value"
    echo "Expected: $CONTROL_VALUE"
    echo "Got: $TEST_VALUE"
    exit 1
fi
```

At each step of the script the success or failure of that step can be measured, and the amount of time it took to execute each step can also be recorded. This allows the operator to not only understand whether the service is functioning, but also how it is performing, especially when associated with CPU, RAM, and disk utilization statistics. This helps the operator build a true understanding of service health and performance as experienced by the people consuming it. As the script matures, it can be expanded to monitor aspects such as performance replication or dynamic credentials.

We recommend executing the script at a defined interval using some form of job scheduler, such as cron. Additionally, the script is built in a way that allows long-term storage of critical data points, such as success/failure or execution time. Leveraging log aggregation tools will allow for the long-term evaluation of the data from this script.

This script would have to be maintained by a privilege/access management team in the longer term,

but the Vault API generally does not make breaking changes, and those which are introduced are surfaced as deprecation warnings at least quarters ahead of schedule - and Vault version changes would pick these up as part of prep for upgrade.

13.5 Operational logs

Vault operational logs focus on the internal processes, activities, and performance of the Vault server itself. They provide insights into how Vault operates and handles requests, errors, and system health, primarily for administrative and troubleshooting purposes within the Vault environment.

The operational logs are gathered from standard output and standard error. Configure the logging level in the Vault server configuration to either `trace`, `debug`, `info`, `warn` or `error`. Trace gives the most detailed information and error only shows error messages in the log. Configure your monitoring solution to alert on `error` log events.

HashiCorp Technical Support Engineers will often ask for Vault operational logs as a troubleshooting step, so it is extremely helpful if you provide these logs whenever relevant and especially when opening a new support issue. Please obtain the compressed log file(s) for each Vault server or as directed by HashiCorp support and share them in the ticket.

13.6 Incident response

In this section, we cover the handling of unauthorized token usage and the identification of accessed secrets. This is an essential aspect of Vault's role in enhancing organizational security. If you observe unwanted behavior, the following steps help you remediate the issue. We have added specific examples for incident responses in our tutorials.

13.6.1 General steps

1. Revoke the compromised token: Use the Vault CLI or API to revoke the compromised token

   `vault token revoke <token_id>` or `vault token revoke -accessor <token-accessor>`

2. Determine the time frame: Try to determine the time frame when the unauthorized activity occurred or when you suspect the token was compromised.

3. Search by Token ID: generate the hash of the compromised token and use tools like `grep` to search for entries in the audit logs containing the hash of the compromised token

```
$ vault write /sys/audit-hash/file input="$VAULT_TOKEN"
Key     Value
---     -----
hash    hmac-sha256:72
    d000ab78fa0036a5977e65a81841ca36a56c9c1233d550fe4e9007c5df96a5
```

4. Review the audit log entries: Review audit log entries associated with the compromised token to identify any secrets or data that were accessed or manipulated. Look for commands or actions like `read`, `write`, or `delete` that involve secret paths or keys and list the secrets or data paths that were accessed or manipulated by the compromised token.
5. Rotate affected secrets: If the compromised token accessed secrets, revoke or rotate those secrets using the Vault CLI or API.
    - Revoke the secrets via the lease ID

      `vault lease revoke database/creds/readonly/<lease-id>`
    - In case of several secrets being compromised, using the prefix for revocation can be helpful

      `vault lease revoke -prefix database/creds`
    - Generate a new secret and have application/services read the new secret in Vault

      `vault read database/creds/role-name`

13.6.2 Restrict access

Depending on the severity of the incident, you might wish to restrict access to Vault. A Vault operator can use one of two methods to restrict access:

1. Seal Vault. Sealing wipes the root key from memory, effectively shutting down all operations in Vault. In multi-tenant environments this is a highly disruptive operation.

```
vault operator seal
```

2. Lock the API of a specific namespace. This stops all operations on a specific namespace and any child namespaces.

Lock the namespace and note the `unlock_key`:

```
$ vault namespace lock hvd
Key          Value
---          -----
unlock_key   uzkzIgUvtyj3fP1bKlclk9ox
```

Unlock the namespace after remediating the incident:

```
vault namespace unlock -unlock-key uzkzIgUvtyj3fP1bKlclk9ox hvd
```

14 Resource quotas

14.1 Overview

Every interaction with Vault travels through its API. When organizations operate Vault as a shared internal platform, that API surface reaches thousands of workloads across teams, namespaces, and mounts. A single incorrectly configured application or a sudden traffic spike from one team can exhaust Vault's resources and degrade service for everyone else.

Resource quotas are Vault's primary mechanism for protecting platform stability under shared-service load. They let platform operators set enforceable limits on two distinct resource classes:

- **Rate limit quotas** — cap the number of API requests per second (RPS) reaching a given namespace, mount, path, or role.
- **Lease count quotas** — cap the total number of active leases generated against a namespace, mount, path, or role *(Enterprise only)*.

Used together, these controls give platform teams the guardrails they need to run Vault as a reliable shared service without relying solely on consumer teams to behave correctly.

14.2 Prerequisites

Before implementing resource quotas, confirm that you have:

- Vault operator or administrator access to the root or administration namespace.
- Established a baseline for normal request volume and active lease counts using Vault telemetry (see the Monitoring and Observability guidance).

- Completed the adoption and standardization phases; quotas applied before you establish stable usage patterns tend to require frequent retuning.

14.3 Choose the right quota type

Vault's resource quota types guard against different categories of operational risk. Select the quota type that matches the problem you are mitigating to protect the cluster effectively. The following table maps each quota type to the class of problem it addresses.

	Rate limit quota	Lease count quota
Available in	Community and Enterprise	Enterprise only
What it limits	API requests per second	Total active leases
Protects against	API flood, denial-of-service, noisy-neighbor traffic	Lease explosion, storage degradation, memory pressure
Algorithm	Token bucket (per unique client IP, per node)	Hard counter
Scope	Global, namespace, mount, path, role	Global, namespace, mount, path, role
Quota definition	Stored on the primary cluster and replicated to performance secondaries	Stored on the primary cluster and replicated to performance secondaries
Enforcement	Per node — each node keeps its own token buckets	Cluster-wide — all nodes in a cluster share the lease counter; each performance replication cluster counts only its own leases
Audit on rejection	Off by default; opt-in with `enable_rate_limit_audit_logging`	Always logged by enabled audit devices; no configuration required

Use rate limit quotas when the problem is high request volume — a misbehaving client hammering

an endpoint, a burst in authentication requests, or an unplanned traffic spike from a new workload.

Use lease count quotas when the problem is lease growth — dynamic secret mounts generating credentials that are never revoked, clients not reusing existing tokens, or mounts with time-to-live (TTL) values set too high. These conditions accumulate a backlog of unexpired leases.

When both problems are present, apply both quota types. They are independent controls and you can layer them at the same scope.

14.4 Rate limit quotas

14.4.1 How rate limit quotas work

Rate limit quotas use a token bucket algorithm. Each unique client IP address gets its own bucket on each Vault node. The bucket refills at the configured `rate` (requests per second). When the bucket is empty, Vault rejects requests with `HTTP 429 Too Many Requests`.

Because each node maintains its own independent buckets, rate limit quotas are **not cluster-wide aggregates**. A global rate limit quota of 500 RPS allows up to 500 RPS from each unique IP address *on each node*. Plan your `rate` values with the node count in mind.

14.4.2 Scope and precedence

You can define a rate limit quota at any of the following scopes. When multiple quotas match a request, the most specific one wins:

Precedence (highest to lowest)	Scope
1	Namespace + mount + role
2	Namespace + mount + exact path
3	Namespace + mount + glob prefix path
4	Namespace + mount
5	Namespace
6	Closest inheritable parent namespace

Precedence (highest to lowest)	Scope
7	Global (empty path)

Start with a modest global quota to establish a baseline, then add targeted mount or path quotas for mounts that carry disproportionate traffic or risk. Avoid writing highly granular path quotas at scale. Each path quota is a separate rule to tune and maintain, and when you split traffic across many narrow quotas, quota metrics make it harder to tell whether a mount or namespace as a whole is approaching its limits. Vault also evaluates quota rules on every request, so a large number of rules adds compute overhead.

Note

Create a quota only for a path that exists on the cluster where you are creating it. On Performance Replication setups, Vault stores quotas on the primary cluster and forwards all quota create, update, and delete requests to the primary. As a result, you cannot create a quota for a local mount on a performance secondary cluster.

14.4.3 Blocking behavior with `block_interval`

By default, a client that exceeds the rate limit gets throttled — its bucket runs empty and Vault rejects further requests until the bucket refills. Setting `block_interval` to a non-zero value changes this behavior: Vault blocks the client from *all* requests for the duration specified, regardless of the endpoint.

Use `block_interval` when you need to discourage retry storms. Without it, an aggressive client can sit at the rate limit boundary and continuously probe. With a block interval, the client faces a penalty for exceeding the limit and must wait before retrying.

```
$ vault write sys/quotas/rate-limit/auth-aggressive \
    path="auth/approle" \
    rate=100 \
    block_interval=60
```

The preceding example limits the AppRole auth mount to 100 RPS per client IP address and blocks any IP address that exceeds that limit for 60 seconds.

14.4.4 Identity-based and collective modes (Enterprise)

Vault Enterprise extends rate limit quotas with a `group_by` option that changes the grouping strategy for incoming requests:

`group_by` value	Behavior
`ip`	One bucket per source IP address *(default)*
`none`	One collective bucket for all requests at the target scope
`entity_then_ip`	Authenticated requests bucketed by entity ID; unauthenticated by IP
`entity_then_none`	Authenticated requests bucketed by entity ID; everything else in one collective bucket

Choose **`group_by=none`** when you want a collective cap across an entire namespace or mount — for example, a hard ceiling of 1,000 total RPS into a tenant namespace regardless of how many IP addresses the workloads originate from.

Choose **`entity_then_ip`** or **`entity_then_none`** when workloads share IP addresses (common in containerized environments with NAT) or when individual workloads scale across many IP addresses. Bucketing by entity ID gives each authenticated identity its own consistent limit regardless of how its network addresses change.

When using `entity_then_ip` or `entity_then_none`, you can set a separate `secondary_rate` for the non-entity bucket (unauthenticated or IP-grouped requests). This lets you apply a distinct limit to traffic that Vault cannot trace to a known identity:

```
$ vault write sys/quotas/rate-limit/ns-collective \
    path="tenant-a/" \
    rate=500 \
    group_by=entity_then_none \
    secondary_rate=1000 \
    inheritable=true
```

Note

`group_by`, `secondary_rate`, and `inheritable` are Vault Enterprise features. Community Edition rate limit quotas always use per-source-IP bucketing.

14.5 Lease count quotas

14.5.1 How lease count quotas work

Lease count quotas are a Vault Enterprise feature that cap the total number of active leases generated against a configured scope. When active leases reach the cap, new lease-generating requests — such as dynamic credential generation or token creation — fail until existing leases expire or operators revoke them.

Like rate limit quotas, the primary cluster stores lease count quota definitions and replicates them to performance secondaries. Unlike rate limit quotas, enforcement is not per node. All nodes in a cluster share the lease counter, so the limit is a cluster-wide aggregate regardless of which node receives the request.

Performance secondaries keep track of their own tokens and leases, so each performance replication cluster enforces the quota against its own lease count. A quota with `max_leases=5000` allows up to 5,000 leases on the primary and up to 5,000 on each performance secondary. Size `max_leases` for each cluster, not for the replication set as a whole.

14.5.2 Protecting against lease explosions

A lease explosion occurs when lease generation outpaces revocation and expiration, leading to an uncontrolled growth in the number of active leases. Left unchecked, this creates a memory drain on the active node that can cascade to other nodes and cause cluster-wide denial-of-service.

Lease count quotas are the primary guardrail against lease explosions. Apply them early — before a leak occurs — so that a misbehaving mount cannot accumulate tens of thousands of leases undetected.

Use the following checklist when designing lease count quotas for a shared Vault platform:

- **Use targeted quotas and keep a global quota as a backstop.** Most organizations get more value from mount- and role-level quotas than from a single global ceiling. New Vault Enter-

prise installations (1.16 and later) include a default global lease count quota of 300,000 leases. Keep it in place as a backstop, and lower it if your cluster has a history of lease explosions, so that a runaway mount cannot take the cluster down.

- **Add targeted quotas where lease growth is most likely.** AppRole auth mounts are the most common source of uncontrolled lease growth, followed by service tokens in general and PKI roles that issue certificates with leases. Apply quotas at the auth mount or role level for AppRole. For PKI, leave `generate_lease` at its default of **`false`** unless you need to revoke certificates by lease ID. Set each quota to the expected steady-state lease count for that mount, plus burst headroom.
- **Pair quotas with short TTL values.** A quota does not replace proper TTL configuration — it is a safety net. A database mount with a 24-hour default credential TTL hits its lease count ceiling much sooner than one with a 1-hour TTL. Tune TTL values first, then set quotas based on the resulting steady-state lease count.
- **Treat quota violations as a signal to investigate, not just a limit to raise.** If a mount consistently approaches its lease count limit, the right response is to investigate client behavior (Are clients reusing credentials? Are they revoking leases after use?) before raising the ceiling.

```
## Apply a mount-scoped lease count quota for a database secrets engine
$ vault write sys/quotas/lease-count/db-credentials \
    name="db-credentials" \
    path="database/" \
    max_leases=5000
```

14.6 Operational guidance

14.6.1 Audit logging for quota rejections

Rate limit quota rejections are **not written to the audit log by default**. This is intentional: under high rejection volume, audit logging every rejected request can itself become a performance bottleneck.

Enable rate limit audit logging only when you have a specific need to trace rejections back to individual clients — for example, when investigating an application with a suspected configuration error:

```
$ vault write sys/quotas/config enable_rate_limit_audit_logging=true
Success! Data written to: sys/quotas/config
```

Disable it again after the investigation is complete if rejection volume is high.

Lease count quota rejections are always written to enabled audit devices. Vault checks lease count quotas after it audits the incoming request, so the rejection appears in the audit response entry with a `lease count quota exceeded` error. The `enable_rate_limit_audit_logging` setting applies only to rate limit quotas. Under heavy rejection volume, this extra audit logging can affect performance, so track `vault.quota.lease_count.violation` and act on violations promptly.

14.6.2 Telemetry metrics

Vault exposes the following metrics for monitoring quota health. Each metric carries a `name` label that identifies the specific quota rule.

Metric	Type	What it signals
`vault.quota.rate_limit.violation`	counter	Number of requests rejected by a rate limit quota
`vault.quota.lease_count.counter`	gauge	Current number of active leases against a lease count quota
`vault.quota.lease_count.max`	gauge	Configured maximum for a lease count quota
`vault.quota.lease_count.violation`	counter	Number of requests rejected by a lease count quota

Alert on:

- A sustained, non-zero rate of `vault.quota.rate_limit.violation` from a specific quota indicates a mount or path is consistently at or above its limit and warrants investigation.
- `vault.quota.lease_count.counter` approaching 80–90% of `vault.quota.lease_count.max` — this is the early warning signal for lease growth before a violation. See the preceding telemetry table for metric names. Lease explosions are sudden. A mount can move from moderate use to its limit within minutes. Pair the utilization threshold with an alert on the rate of change of `vault.quota.lease_count.counter` so alerting triggers while there is still time to act.

14.6.3 Response headers

Vault can return rate limit metadata in response headers, giving clients the information they need to implement respectful backoff without relying on repeated `HTTP` 429 responses:

```
$ vault write sys/quotas/config enable_rate_limit_response_headers=true
Success! Data written to: sys/quotas/config
```

When enabled, responses include `X-Ratelimit-Limit`, `X-Ratelimit-Remaining`, and `X-Ratelimit-Reset` headers. Requests blocked by a rate limit quota also include a `Retry-After` header with the number of seconds until the quota resets. Enable this when client teams have the ability to consume and act on these headers. It reduces retry noise without changing the enforcement behavior.

14.6.4 Exempt paths

A small set of Vault paths are exempt from rate limiting by default:

- `sys/generate-recovery-token/attempt`
- `sys/generate-recovery-token/update`
- `sys/generate-root/attempt`
- `sys/generate-root/update`
- `sys/health`
- `sys/seal-status`
- `sys/unseal`

These paths remain available even when a global rate limit quota runs out, which is intentional — Vault requires these paths for operational recovery. Do not remove them from the exempt list without a clear operational reason.

You can extend the exempt list using `rate_limit_exempt_paths` in the quota configuration. Avoid adding application paths to this list casually. Every path on the exempt list bypasses all rate limit enforcement, including targeted mount or namespace quotas.

14.7 Common scenarios

14.7.1 Protect a shared auth mount

A shared AppRole or LDAP auth mount serving dozens of application teams is a common source of authentication floods. An application with an incorrect configuration that fails to cache its token and re-authenticates on every request can overwhelm the mount for all other consumers.

Apply a rate limit quota at the mount level with a `block_interval` to discourage aggressive retry behavior. Set the rate based on your observed peak-legitimate RPS from telemetry, with 20–30% headroom, provided your cluster can comfortably sustain that peak. Revisit the limit whenever request volume changes significantly — for example, after onboarding a new client application:

```
$ vault write sys/quotas/rate-limit/approle-global \
    path="auth/approle" \
    rate=500 \
    block_interval=30
```

Then add a stricter per-role quota for roles known to carry lower legitimate traffic, so a role with an incorrect configuration cannot consume the entire mount budget:

```
$ vault write sys/quotas/rate-limit/approle-webapp \
    path="auth/approle" \
    role="webapp" \
    rate=50
```

14.7.2 Protect a noisy namespace or path

When a tenant namespace or a specific secret path is generating disproportionate traffic, apply a quota directly at that scope rather than raising or tightening the global limit. This limits the impact on other namespaces and preserves headroom for legitimate traffic elsewhere.

A namespace quota already applies to every mount within that namespace. On Vault Enterprise, also set `inheritable=true` to apply the same quota to all child namespaces without having to create a quota for each one:

```
$ vault write sys/quotas/rate-limit/tenant-b-cap \
    path="tenant-b/" \
    rate=1000 \
    inheritable=true
```

14.7.3 Guard against lease growth on a dynamic secrets mount

When dynamic secret mounts begin to accumulate leases faster than expected, apply a lease count quota to create a hard ceiling while you investigate and tune client behavior. Set the quota based on the legitimate steady-state lease count, not the current inflated count:

1. Assess the current active lease volume before setting a ceiling. Use the `vault.expire.num_leases` telemetry gauge in your monitoring system to observe the cluster-wide total lease count and its trend over time. To enumerate leases for a specific role on the target mount, use the `sys/leases/lookup` API with the role prefix. This endpoint requires `sudo` capability:

   ```
   $ vault list sys/leases/lookup/database/creds/<role-name>/
   ```

   The number of lease IDs returned reflects the active lease count for that role. Repeat for each role on the mount. Use this count together with the `vault.expire.num_leases` trend data to set a quota ceiling that matches the legitimate steady-state load rather than a current inflated count.

2. Review mount-level lease telemetry and identify the expected steady state.

3. Apply a lease count quota that reflects that baseline plus headroom:

   ```
   $ vault write sys/quotas/lease-count/database-guard \
       name="database-guard" \
       path="database/" \
       max_leases=2000
   ```

4. Monitor `vault.quota.lease_count.counter` against `vault.quota.lease_count.max` and alert at 80% utilization.

5. If violations occur, treat them as the trigger to investigate client behavior — not as a prompt to raise the quota. Confirm that clients are reusing credentials, revoking after use, and operating with appropriate TTL values before adjusting the limit.

14.8 What not to do

Do not treat rate limit quotas as cluster-wide traffic shapers. Vault enforces rate limit quotas independently on each Vault node using per-source-IP buckets. A `rate=1000` quota allows 1,000

RPS per IP address per node, not 1,000 RPS across the cluster. Designing cluster-level throughput controls requires understanding your node count and traffic distribution.

Do not use `group_by=none` without deliberate intent at broad scopes. A collective (`none`) quota at the global or namespace level with a low rate blocks legitimate traffic from any source the moment the aggregate bucket empties — including traffic that would otherwise be well within individual limits. Use `group_by=none` only when a collective ceiling is the explicit intent, and at the narrowest scope that satisfies the requirement.

Do not conflate rate limiting with lease count management. These protect against different failure modes. A low rate limit does not prevent a lease explosion if the client generates one expensive credential per request at a rate well under the limit. Apply both quota types for workloads that can fail in both dimensions.

Do not raise a lease count quota as the first response to a violation. Quota violations are signals. An application hitting its lease count limit is telling you it is not cleaning up after itself. Investigate before raising the ceiling. Otherwise the ceiling provides no protection at all.

Do not confuse resource quotas with the deprecated request limiter. Vault documentation includes references to a request limiter feature that predates the current resource quota system. Resource quotas are the current, supported mechanism. Do not mix guidance from both systems.

14.9 Related resources

- Vault resource quotas concepts
- Create a rate limit quota
- Rate limit quotas - collective, by IP, by entity
- Create a lease count quota
- Prevent lease explosions
- Vault quota telemetry metrics
- Monitoring and observability

15 Backup and restore operations

15.1 Backup standard procedure

> Note
>
> On HCP Vault, snapshot functionality is managed by the HCP control plane. On starter tier and above clusters HashiCorp performs automated daily snapshots, with the option for a customer-initiated on-demand snapshot. See HCP Vault Operations Tasks.

The objective of this section is to provide a set of standard operating procedures (SOP) for backing up a Vault cluster. It protects your Vault cluster against data corruption or sabotage of which disaster recovery replication might not be able to protect against.

15.1.1 Prerequisites

The following prerequisite steps and knowledge are required in order to backup a Vault cluster. All of the following are required to understand or carry out before attempting to take a backup or restore of Vault.

- Working knowledge of Vault: Some working knowledge of Vault is required in order to follow these SOPs.
- Vault cluster configuration as defined in our Vault with Integrated Storage Reference Architecture.
- Vault is initialized: This SOP assumes you have already initialized Vault, keyholders are available with access to the unseal keys for each, that you have access to tokens with sufficient privileges for both clusters and encrypted data is stored in the storage backend.

15.1.2 Automated backup procedure for Vault with disaster recovery replication enabled

Enable automated snapshots on the primary cluster with the following command, following the instructions on the Raft Integrated Storage Documentation and using the options specified on the API Documentation.

```
vault write sys/storage/raft/snapshot-auto/config/[:name] [options]
```

15.2 Restoring a Vault Enterprise cluster

The objective of this section is to provide a set of standard operating procedures for restoring a Vault cluster from a snapshot.

> Note
>
> These procedures specifically apply to Vault clusters that use the integrated storage backend.

While typically you would rely on your DR secondary cluster to protect against data loss, there are cases that disaster recovery replication cannot safeguard against, such as data corruption or malicious data manipulation. In these cases, snapshots are the only source of data you will be able to trust when attempting to restore service.

15.2.1 Restore procedure for Vault with disaster recovery replication enabled

1. Prepare and initialize the new restored Vault cluster. Initialize the cluster, then log in with the new root token that was generated during initialization. Note that these keys and tokens are only temporary; the original unseal keys will be needed following restore.
2. Copy the snapshot file onto the new cluster. Begin the snapshot restore with the following command.

   ```
   # Replace <backup.snap> with the filename of your snapshot file
   vault operator raft snapshot restore -force <backup.snap>
   ```

   Because the snapshot you are restoring is from a different cluster, the unseal keys of the new cluster will not be consistent with the snapshot data. In cases like this, you must specify the `-force` flag for the snapshot restore to succeed.

   Caution: you can safely use `-force` when restoring a snapshot from a system using Shamir seal. After the restore, the original storage entry for the root key will be available, allowing the original cluster's unseal keys to decrypt it. However, with autounseal, it is crucial to use the same seal configuration for the new cluster as the one used for the cluster being restored. If not, data decryption will not be possible after the restore, and using `-force` will disable a valuable safeguard.
3. Once the snapshot restore is complete, unseal the Vault cluster again using the following command.

```
vault operator unseal <unseal_key>
```

16 Disaster recovery operations

> **Note**
>
> This page covers recurring disaster recovery operations. For initial disaster recovery replication setup, see the Installation Guide: Disaster recovery initial setup.

16.1 DR cluster promotion

Deciding when to initiate a DR process on Vault can be a complex decision. Due to the difficulty in accurately evaluating why a failover should or should not happen, Vault Enterprise deliberately does not support an automatic failover/promotion of a DR secondary cluster. However, It is possible to automate accurate detection with your own development/tooling, and triggering/orchestrating failover by leveraging Vault's API, specifically the Replication and DR APIs.

In this section, we will outline the identification and manual activities to failover to a DR cluster.

16.1.1 When to initiate a DR process

We might think we will be able to identify a situation that should drive a DR process at the time it occurs, but usually this is not the case. Complex network topographies, ephemeral cloud resources, and unreliable metrics/health indicators can all falsely alert to an outage or quietly suppress a true outage.

Initiating a DR process for Vault is usually part of an organization-wide issue such as connectivity or a datacenter outage, versus a siloed problem with the Vault cluster. Vault itself is designed and should be deployed in a highly-available model that is tolerable to common issues such as a server losing a disk drive.

When Vault is deployed inline with the architecture outlined in the HashiCorp Vault Enterprise Installation Guide, two availability zones can fail with Vault still maintaining quorum, operation, and performance.

In the diagram below, the Vault leader node (voter) is in Availability Zone B. If it fails, the follower node (non-voter) in Availability Zone B will be elected as the new leader node (voter) and continue operation. The specifics of how active node step-down, leadership lost, and Raft election are outside the scope of this document.

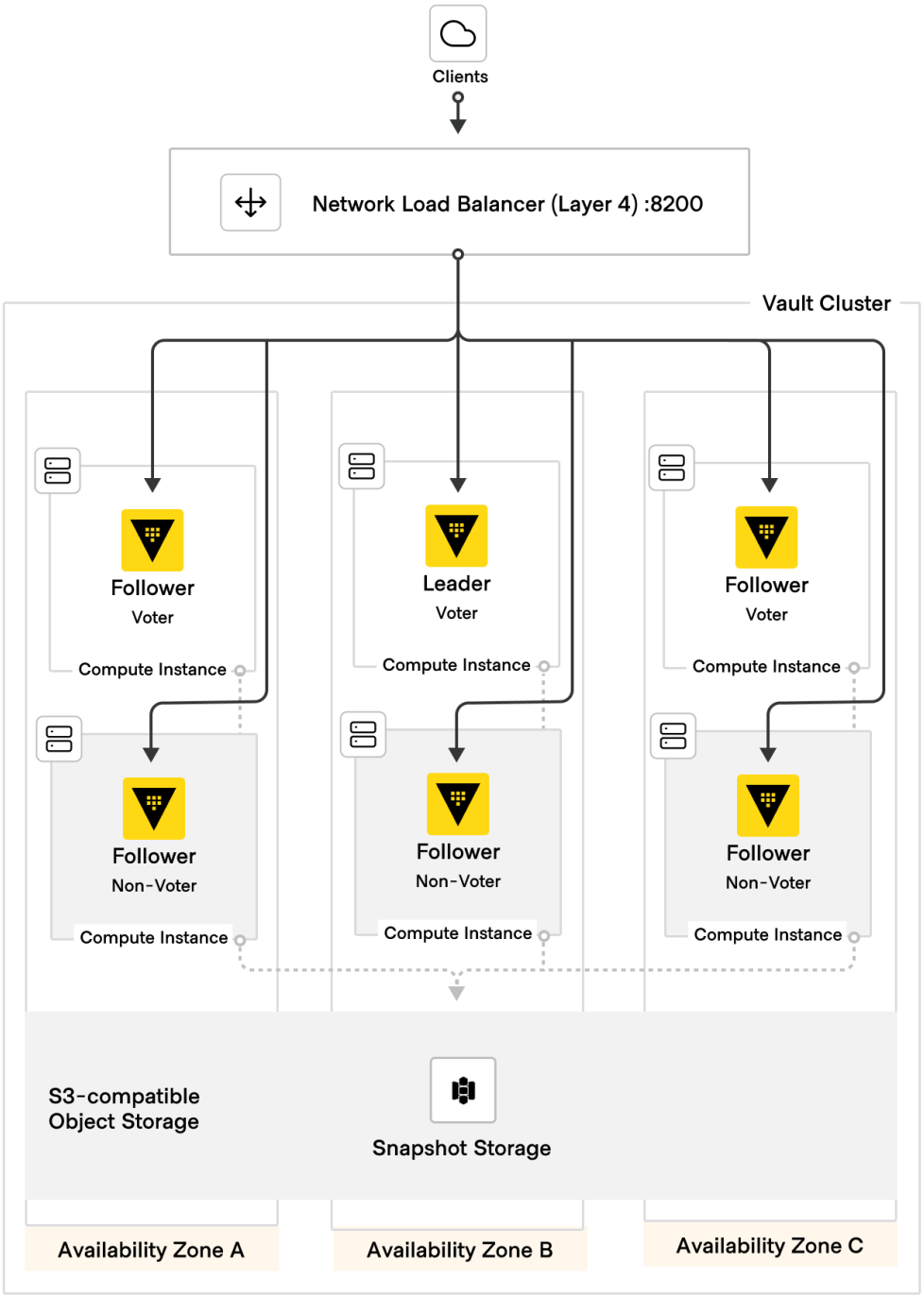


Figure 30: Vault DR Failover

Vault operators should have a runbook to handle every combination of losing a node or nodes in the above architecture. For example, if a node in Zone C fails overnight it might be acceptable to wait until the next day to replace it in the cluster. However, a loss of connectivity entirely to Zone B and C might trigger action to replace those lost nodes immediately in Zone A, as once 4 nodes are lost the cluster has no more redundancy for maintaining quorum.

In the context of DR, two scenarios should initiate an immediate process to failover to a DR cluster:

1. A rapid loss of all voter nodes such as a failure of all three availability zones.
2. Failure of more than 4 nodes across all three availability zones - a minimum of 2 voter nodes are required to maintain quorum.

Identification of issues

Reactive awareness

Initial awareness and triage of a condition that could trigger a DR move is the first step to understanding what is going on, where the breakdown is, and which team/platform needs to be involved to resolve it. A few examples of methods for this include the following.

- Application alerting (ie, application cannot connect to Vault)
- Container/Server logs
- APM/tracing tools (Zipkin, Jaeger, Dynatrace, Datadog APM, etc)
- Vault logs
- Vault telemetry
- End-user complaints

Proactive action

Outside of reactive awareness of an issue, there are situations that your business continuity plan might prescribe a move to a DR environment ahead of the actual event. Examples of this include expecting and planning for a pending disaster, such as many financial organizations failing over to non-coastal datacenters when hurricane Sandy approached the New York area in 2012.

Confirmation of issue

Upon notification that something might be wrong, your organization should take one of two paths of action depending on the criticality and urgency of the possible outage.

- Immediately begin failover procedures.
- Execute a pre-authored and pre-tested triage runbook to verify which components are functioning and which have failed. Then, staff will decide to initiate a failover, fix the system within the primary environment, or possibly accept the period of outage.

The triage runbook is a specific set of verifications and conditions to assure staff makes the best decision to perform a DR move or not. For Vault, these can include:

- Are all services that use Vault experiencing problems, or one/subset?
- Is Vault itself (logs, telemetry) indicating a problem?
- Is there current maintenance that is possibly causing false alarms?
- Is Vault the actual issue, or could it be downstream (ie, the auth Vault uses to a cloud provider has been revoked)?
- Is the Vault cluster and each node responding if accessed directly vs through the load balancer?
- Is the issue something that would have been replicated to the DR cluster already (ie, accidental mass-revocation of tokens, or configuration change)
- WIll DR resolve this issue? Or will we need to restore from backup?

Confirmation of issue is important to protect the uptime of the service. Having an automated means to failover may reduce outage time, but we do not recommend fully automated failover.

16.1.2 How to initiate a DR process

Once an outage has been confirmed, operations staff should have a standard set of procedures to address the outage in order to minimize downtime, ensure data consistency, and notify others about outages, rollbacks, or new infrastructure to employ in a DR event. Given the multi-cluster architecture below as documented in the Vault Installation Guide, in the event of a failure of DR primary cluster in Region 1, the recommended steps to promote the DR secondary cluster in Region 2 as the new primary are:

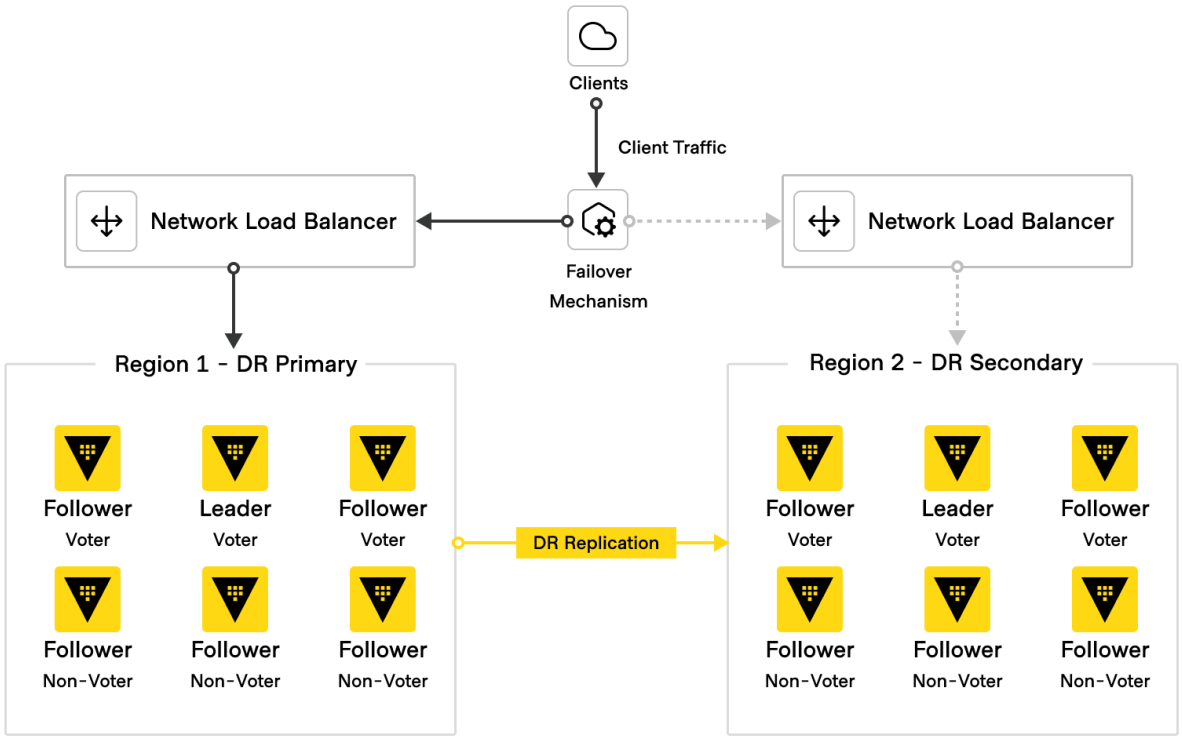


Figure 31: Vault DR Procedure

The prerequisite is to have generated a Batch DR operation token as per the advice above.

Follow the steps below.

1. Disable load balancing/routing while failover is in progress:
2. Disable the network load balancer to Region 1 Vault nodes.
3. Disable global load balancer (e.g. F5 GSLB, Amazon Route53 Failover Record, HashiCorp Consul). Recall the requirement of a failover mechanism in the Vault Installation Guide.
4. Promote Region 2 DR secondary cluster to become the new primary cluster.
5. Configure your working environment:
6. Set `VAULT_ADDR` to the address of the DR secondary cluster load balancer.
7. Verify connectivity and cluster information with `vault status`.
8. Promote DR secondary cluster from secondary to primary

9. Verify Region 2 Vault cluster is acting as a primary, that it has no secondaries, mode is primary, and its state is running:

```
vault read sys/replication/dr/status --format=json
{
"request_id": "ba05db44-2d64-174c-6857-9d7507fcf625",
...
"data": {
    "cluster_id": "78a9148d-0c9b-49e9-f025-a997811e756e",
    "known_secondaries": [],
    "last_dr_wal": 3476845,
    "last_reindex_epoch": "0",
    "last_wal": 3476845,
    "merkle_root": "83ca160168dd2609ae8be43595915ebcd4d3415a",
    "mode": "primary",
    "primary_cluster_addr": "",
    "secondaries": [],
    "state": "running"
}
```

1. Enable load balancing to the new cluster:
2. Configure global load balancer to target Region 2 Vault cluster load balancer.
3. Verify connectivity to Region 2 Vault cluster via browser.
4. Notify Vault consumers and stakeholders Vault is back online.
5. As soon as the old primary is accessible, demote it to prevent split brain scenario, since Region 2 is now Primary.
6. Set `VAULT_ADDR` to the address of a Region 1 DR primary node.
7. Demote Region 1 DR primary to secondary.

16.1.3 How to failback

After the original primary has been restored to operation, it may be necessary or desirable to revert the configuration to the preferred topology. The recommended steps to failback are in this section of the public documentation on the subject, and additional considerations are below. In the example above, Region 2 is the current Primary, and Region 1 was the old primary and will be promoted again.

The prerequisite is to have generated a Batch DR operation token as per the advice above.

Considerations

1. If you have not yet demoted the Region 1 old primary (see Step 5 above), you must do that before bringing it online and resuming replication. Demote Region 1 DR primary cluster to become secondary.
2. Monitor Vault replication to ensure that the WAL (write ahead logs) are in sync:
    1. Ensure that state of the secondary is `stream-wals`.
    2. `merkle_root` on the primary and secondary should match.
    3. The `last_dr_wal` on the primary should match the `last_remote_wal` on the secondary.
3. Notify Vault consumers and stakeholders of planned outage for failback to Region 1.
4. Disable load balancing/routing while failback is in progress:
    1. Disable load balancer to Region 2 DR primary.
    2. Disable global load balancer.
5. Demote Region 2 DR primary to a secondary:
    1. Set `VAULT_ADDR` to a Region 2 DR secondary node.
    2. Demote the Region 2 DR primary to secondary.
6. Promote Region 1 DR secondary as the new primary and verify Region 1 Vault cluster is acting as a primary, that it has no secondaries, mode is primary, and its state is running.
7. Establish replication between Region 2 DR secondary and Region 1 DR primary cluster.
    1. Generate a secondary activation token from Region 1 DR primary.
    2. Copy the generated `wrapping_token`, you will need it in the next step to update the primary for Region 2 DR secondary.
    3. Update the primary of Region 2 DR secondary using the batch DR operation token and the secondary activation token from Region 1 DR primary.
8. Enable load balancer.
    1. Enable network load balancer to Region 1 DR primary and Region 2 DR secondary.
    2. Enable global load balancer.
9. Notify Vault consumers and stakeholders Vault is back online.

17 Performance replication operations

> **Note**
>
> This page covers recurring performance replication operations. For architecture and initial setup, see the Installation Guide: Performance replication architecture and setup.

17.1 Monitoring performance replication

The health of Vault performance replication should be monitored. Monitoring Vault performance replication should include ensuring that:

1. Both the primary and secondary clusters are online and in a healthy state
2. Network communication between the primary and secondary clusters has not become disrupted
3. Replication subsystem in Vault is operating in a healthy state on primary and secondary clusters and that data sync is keeping pace between the two

For item number one, refer to the Vault observability setup to ensure that telemetry has been configured on primary and secondary clusters, and the outlined key metrics are being monitored for general cluster health.

For item number two, a synthetic check can be run periodically against the sys/replication/performance/status API endpoint on both clusters to check the value of the "state" field. If the value of the state field is "idle" then this indicates that communication may have become disrupted and replication is no longer taking place. The progress and health of synchronization between primary and secondary clusters can be checked by querying the sys/replication/performance/status API endpoint on both clusters and comparing the values of last_performance_wal on the primary and last_remote_wal on the secondary. After writing a piece of replicated data, the last_remote_wal on the secondary should match the last_performance_wal on the primary for a short period of time.

For item number three, there are specific Vault telemetry values that should be monitored to indicate the health of the replication subsystem:

Metric Name	Description
vault.wal_persistwals	Time taken to persist a WAL to storage
vault.wal_flushready	Time taken to flush a ready WAL to storage

These metrics indicate the time being taken in the replication subsystem to process write ahead logs (WALs). Both of these metrics should be monitored for a situation where the values continue to grow without returning to the expected baseline. In a high-load scenario, WALs can accumulate, putting pressure on Vault's storage backend.

18 Upgrade procedures

18.1 Vault Enterprise upgrade

> Note
>
> On HCP Vault, upgrades are managed by the HCP control plane. See HCP Vault Operations Tasks.

When it comes to upgrading HashiCorp Vault, it is essential to follow well-defined operating procedures to ensure a smooth transition and minimal disruption. Vault upgrades are crucial for security and performance enhancements, but they can also be complex. In this regard, understanding the correct steps and best practices is vital. Ensure the run book is understood by multiple staff in the Platform Team and tested by someone who is not the author to ensure it is fully understandable.

18.1.1 Autopilot

HashiCorp highly recommends that you utilize the autopilot upgrade feature, as it greatly streamlines and simplifies the upgrade process. Autopilot is purpose-built to minimize administrative involvement and reduce downtime, making upgrades smoother. It synchronizes all data to new nodes, and it accomplishes this through snapshots instead of the replication subsystem. This approach is often faster and more comprehensive. Raft seamlessly manages this process, sparing you the need to delve into the technicalities. Another benefit of utilizing autopilot is that there is

less of a need to expend resources closely monitoring traffic during upgrades, as this feature comes with many safety checks. This can lead to fewer potential losses in terms of both time and data. Additionally, you can leverage Terraform for setting up and deploying configurations and Packer for creating new nodes from templates. Autopilot ensures that Vault executes parts of the upgrade process in the correct order, resulting in minimal downtime.

When using Vault's Autopilot upgrade function, it's important to be mindful of a couple key considerations. Autopilot simplifies the safe transition of leadership to the new nodes, while demoting the old ones so as not to impact quorum. However, this ease of provisioning should be monitored to ensure it aligns with your infrastructure requirements.

Autopilot does not automatically manage the removal of old nodes. You should always manually decommission outdated nodes once an upgrade is successfully completed to prevent resource inefficiencies.

The default autopilot configurations are suitable for most scenarios. Unless you have a specific reason for doing so, it is advised against tuning the autopilot config. Picking inappropriate values can lead to difficult-to-debug problems. These default configurations are best suited for relatively static clusters, such as those deployed on manually provisioned VMs. However, if you are operating in a more dynamic environment, like Kubernetes or behind an auto scaling group or equivalent, tuning may be beneficial.

If autopilot is not available to you, please refer to the appendix.

18.1.2 How to upgrade Vault in an AWS auto scaling group with automated upgrades

Vault autopilot enables you to automatically upgrade a cluster of Vault nodes to a new version as updated nodes join the cluster. This how-to walks through the steps necessary to implement automated upgrades in an AWS auto scaling group (ASG).

Prerequisites

- A basic knowledge of AWS Auto Scaling, including scale-in protection.
- A general understanding of how Vault upgrades work.
- An AWS Auto Scaling group containing a Vault cluster running Vault Enterprise 1.11.0 or later.

- Upgrade migration must not be disabled on the cluster. Vault Enterprise enables automated upgrade migrations by default. You can check the value of this parameter by running the following command.

```
$ vault operator raft autopilot get-config | grep 'Upgrade Migration'
```

Output.

```
Disable Upgrade Migration            false
```

If upgrade migration is disabled, you can re-enable it by running the following command:

```
$ vault operator raft autopilot set-config -disable-upgrade-migration=true
```

Procedure

1. In your AWS account, create a launch template that can create new instances with the desired new version of Vault. This can be accomplished by using an AMI with Vault already installed, or via a userdata script that installs Vault at boot time.
2. Configure the ASG for your Vault cluster to use the new launch template. Once the new launch template is in place, future instances created by the ASG will be created using the new version of Vault. Existing instances will not be affected.
3. Update the ASG by changing the desired capacity count to at least double the current number of nodes. For example, if 3 nodes are currently deployed, change the desired capacity to 6.

> Note
>
> The number of new nodes running the target version of Vault must be equal to or greater than the number of existing nodes before the autopilot process will begin promoting new nodes to voters.

1. The ASG will begin creating additional instances up to the desired capacity. Once enough nodes running the target version have joined the cluster, the upgrade migration process should start. Monitor this process using the following command:

```
$ watch -n 0.5 'curl \
    -H "X-Vault-Token: $VAULT_TOKEN" \
    $VAULT_ADDR/v1/sys/storage/raft/autopilot/state \
    | jq -r ".data.upgrade_info"'
```

Continue to monitor the output until the `status` field reads `await-server-removal`. The output will look similar to the following:

```
{
    "other_version_non_voters": [
        "vault_2",
        "vault_3",
        "vault_4"
    ],
    "status": "await-server-removal",
    "target_version": "1.12.0.1",
    "target_version_voters": [
        "vault_5",
        "vault_6",
        "vault_7"
    ]
}
```

Collect the node ids from this output to refer to in subsequent steps. `other_version_non_voters` contains the list of nodes running the old version of Vault, which have been demoted to non-voting status. `target_version_voters` contains the list of nodes running the new version of Vault, which have been promoted to voting status.

1. Remove the nodes running the old version of Vault from the cluster. For each node that needs to be removed, run the following command:

```
$ vault operator raft remove-peer <node id>
```

1. Temporarily enable scale-in protection for each instance running the new version of Vault. This will prevent the ASG from destroying the new nodes during scale-in events. Follow the steps found at Change Scale-in Protection for an Instance in the AWS Documentation to enable scale-in protection. You can verify that protection is enabled by checking the `Protected From` column in the AWS management console.

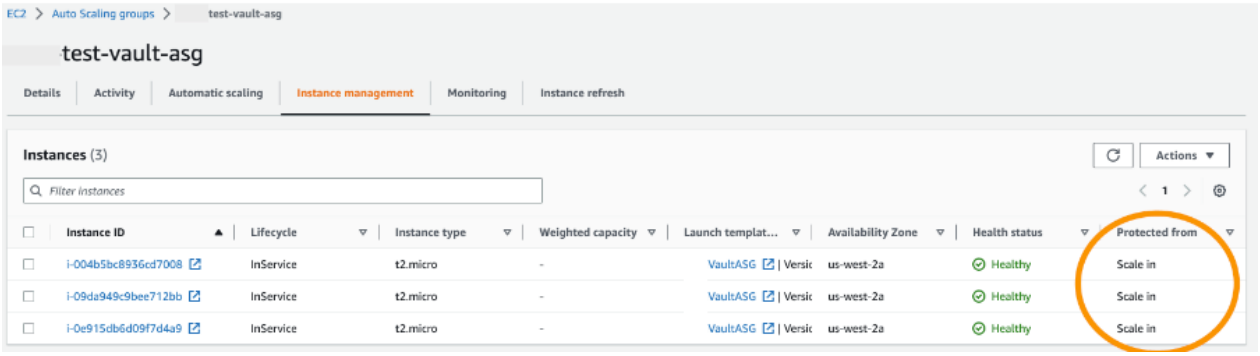


Figure 32: Autoscaling Console

1. Update the ASG again, this time reducing the desired capacity count to its original value. The ASG will begin terminating instances. Instances with scale-in protection enabled will be protected from termination, while instances without protection will be terminated. When this process is complete, all old nodes should be removed from the cluster.
2. Verify that your Vault cluster is healthy and that the upgrade was successful by running the following command:

```
$ vault status
```

Verify that the version number matches the target version of Vault, and that `Sealed` is **`false`**.

1. Remove scale-in protection from the remaining instances. Follow the steps found at Change scale-in protection for an instance in the AWS Documentation to disable scale-in protection.

19 Upgrade procedure examples

19.0.1 HA cluster in place upgrade

HA Cluster in place upgrades are the preferred choice when you must work exclusively with the machines you already have provisioned. It is important to note that this approach is recommended only when provisioning new machines or nodes is not feasible (contrast this to the preferred Autopilot upgrade recommended above).

It is worth emphasizing that HA upgrades demand strict adherence to a well-defined sequence of steps. Deviating from the prescribed order can lead to complications, especially if standby nodes

are not replaced before the active node. Such missteps can disrupt the upgrade process and potentially introduce vulnerabilities.

19.0.2 DR upgrade

Autopilot upgrades are only for use with integrated storage. Migrating from another storage backend to integrated storage would best be done with a DR-style upgrade.

Disaster recovery upgrades are necessary in situations that require a failover mechanism. This process involves replicating all data to new nodes, temporarily halting incoming traffic, promoting the DR secondary to the primary role, and subsequently reconfiguring the nodes to communicate with the DR secondary. The primary objective is to ensure seamless system continuity and minimal downtime during unexpected failures or disruptions.

20 Support readiness

To ensure smooth communication and efficient support, make sure that these guidelines are followed.

- Each team member has an account created through the HashiCorp Support portal.
- Each team member that should be permitted to create support tickets on behalf of your company must be configured as an authorized technical contact in the support portal. Please provide a list of applicable team members to your company's assigned solutions engineer so that they can configure this.
- Have your team familiarize themselves with HashiCorp support documentation on how to open a support ticket with HashiCorp Support and generate/upload a support bundle, whenever applicable.
- Be aware of the support plan level your company has purchased to manage expectations of response times. Also, understand the definitions for each severity level when opening a support ticket through our documentation.

21 Changelog

This page records notable changes to the Vault Administration Guide.

21.1 2026-08

- Restructured legacy Vault HVD content into the HVD 2.0 Vault Administration Guide as part of the Installation / Administration / User guide migration.

21.2 2026-06

- Add production hardening guide to prerequisites

21.3 2026-01

- Fixed heading formatting in tabbed content for PDF generation
- Update meta descriptions
- Updated credits and contributors

21.4 2025-12

- Updated credits and contributors

21.5 2025-07

- Added observability and monitoring guidance

21.6 2025-06

- Fixed broken links and added redirects
- Standardized operating guide naming conventions

21.7 2025-05

- Removed document structure section
- Fixed typos and documentation errors
- Fix numbered list rendering

21.8 2025-04

- Updated observability guidance
- Referenced patterns from other HVD documentation

21.9 2024-02

- GA release

22 Credits

Thank you to everyone who contributed to this validated design guidance. This cross-functional team, representing many departments within HashiCorp, authored, edited, reviewed, and iterated this content to provide prescription and recommendations for using HashiCorp software in enterprise scenarios.

- Adam Cavaliere
- Allison Broski
- Andrew Thielen
- Anna Chen
- Finn Dannin
- Hari Sankaran
- Jan Repnak
- John Duke
- Justin Clayton
- Kalen Arndt
- Mara Hammond
- Mark Lewis
- Mauricio Afiuni
- Nalini Dayma
- Neil Dahlke
- Nick Wong
- Prakash Manglanathan
- Rowan Smith

- Sara Asano
- Sean Doyle
- Theodore Marx
- Troy Fluegge